Assignment_2_Documentation

2025-12-20

Abstract

This report documents the end-to-end development of a neural network designed to predict customer repayment behavior based on an adjusted Kaggle credit dataset. The primary objective was to classify customers into one of eight distinct behavioral categories, ranging from "paid off" to "write-offs for more than 150 days," using a supervised, non-parametric learning approach.

The workflow followed a structured data science pipeline, beginning with extensive data cleaning and exploratory data analysis (EDA). Key preprocessing steps included the removal of logical anomalies (e.g., "working pensioners" with invalid employment records), the correction of impossible family structures, and the implementation of log-transformations to normalize highly skewed financial variables like income. Feature engineering further enhanced the dataset by introducing interaction terms such as "income per family member" to provide better financial context.

Model selection involved a rigorous evaluation of Multi-Layer Perceptron (MLP) architectures. A systematic grid search identified an optimal topology consisting of two hidden layers (128 and 64 neurons) with Rectified Linear Unit (ReLU) activations. While initial experiments with adaptive optimizers (Adam, AdamW) showed promise, Stochastic Gradient Descent (SGD) with Momentum (learning rate 0.0001, momentum 0.9) ultimately demonstrated superior stability and generalization. To address the dataset's severe class imbalance, various techniques including SMOTE, Random Undersampling, Focal Loss, and Class Weights were tested. However, empirical results indicated that a baseline model trained on raw data, enhanced by a custom decision threshold of 0.39 for the critical "early delinquency" class, provided the best trade-off between sensitivity and precision.

The final "Champion" model achieved a test accuracy of 84.76% and a test loss of 0.4996 on a dedicated 15% hold-out set. The evaluation revealed that while the model excels at distinguishing between standard repayment behavior (1-29 days past due) and severe delinquency, it exhibits a conservative bias, frequently categorizing uncertain cases into the majority class. Despite limitations in differentiating adjacent delinquency buckets, the model successfully isolates high-risk minority classes with high precision, making it a viable tool for behavioral risk forecasting.

1. Introduction

The goal of this assignment was to build a neural network to predict the pay-back behavior of a new customer. The prediction of this behavior is categorical in nature, having 8 different possible levels:

- no loan for the month
- paid of that month
- 1-29 days past due
- 30-59 days past due
- 60-89 days overdue

- 90-119 days overdue
- 120-149 days overdue
- Overdue or bad debts, write-offs for more than 150 days.

The method to be used is a neural network, a supervised non-parametric learning method. Supervised learning means that the data is labeled. A non-parametric method is a statistical technique that does not assume the data follows a specific probability distribution (like the "bell curve" or normal distribution), but instead relies on the data's rank or order to draw conclusions.

The data is an adjusted form of an open source kaggle-dataset, provided by the lecturers. No extra data was used to enhance this dataset.

2. Workflow

In a data science project, adhering to a structured workflow is critical. Before starting the project, a GitHub repository was initialized to ensure reproducibility and, most importantly, to maintain a transparent history of all changes and experiments.

Furthermore, a virtual environment was established to manage dependencies and ensure software consistency across different machines.

2.1. Preprocessing (Data Cleaning)

This is the most time-consuming phase of a data science project. Raw data is rarely ready for modeling immediately, and thorough cleaning is essential to ensure high-quality output. If noisy or poor-quality data is fed into a model, the outcome will inevitably be of low value (a concept known as "Garbage in, Garbage out").

Preprocessing entails handling missing values, correcting inconsistencies, removing duplicate entries, formatting data types, and splitting the data into training, validation, and testing sets to prevent data leakage.

2.2 Exploratory Data Analysis (EDA)

Before building a model, the data scientist must understand the data's behavior. This is achieved through visualization methods, statistical checks, and outlier detection. Findings in this phase often lead to a subsequent round of preprocessing. Consequently, the workflow frequently becomes an iterative back-and-forth between EDA and preprocessing, consuming the majority of the project's time.

2.3. Feature Engineering

In this phase, domain knowledge is typically applied to maximize model performance. Although the team's background in life sciences meant limited prior exposure to specific business domain cases, general data transformation principles were effectively applied. The goal is to transform the data to make it easier for the machine learning algorithm to interpret patterns. Depending on the model's initial performance, new features or different transformations are often introduced to optimize results based on emerging findings.

2.4. Model Selection and Training

For this project, the model type was predefined as a neural network. However, critical decisions remained regarding the specific architecture (e.g., MLP, CNN, RNN) and hyperparameter configuration. The process begins with a baseline model—a simple architecture used to establish a performance benchmark. By testing different architectures and tuning hyperparameters, one attempts to optimize the applicable evaluation metrics. Crucially, this optimization process relies exclusively on the training and validation sets created during preprocessing.

2.5. Evaluation

Finally, the model's performance is assessed using metrics relevant to the problem—specifically loss and accuracy, as defined by the project requirements. This assessment is conducted using the test data set, which was set aside at the beginning of the workflow to ensure an unbiased evaluation of the model's generalization capability.

```
# Set personal working directory
# Change directory as needed
#setwd("~/Documents/Repos/GRP-6_DS-Project")
#renv::activate()
```

3. Preprocessing and EDA

In this chapter, the preprocessing pipeline is documented step-by-step. Crucially, this section reflects the iterative nature of the project. It details not only the final code but also the initial attempts that were later discarded or refined due to poor model accuracy.

```
# Load necessary libraries
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.0     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.2.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

```
library(ggplot2)
library(dplyr)
library(fastDummies)
library(Hmisc)
```

```
##
## Attaching package: 'Hmisc'
##
## The following objects are masked from 'package:dplyr':
```

```
##
##     src, summarize
##
## The following objects are masked from 'package:base':
##
##     format.pval, units
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(tfruns)
library(tictoc)
library(beepr)
library(keras3)
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

3.1. Data Import and Initial Inspection

The first thing we do is load the csv into a dataframe and inspect the amount of rows and columns, as well as the datatypes needed for each column.

```
# Load the dataset
path <- "/Users/Jujou/Documents/Repos/GRP-6_DS-Project/DS-Project_data/Dataset-part-2.csv"
original <- read.csv(path)

# Display the dimensions of the dataset
dim(original)
```

```
## [1] 67614    19
```

```r
cat("Number of rows:", nrow(original), "\n")
```

```
## Number of rows: 67614
```

```r
cat("Number of columns:", ncol(original), "\n")
```

```
## Number of columns: 19
```

```r
# Display the structure of the dataset
str(original)
```

```
## 'data.frame':    67614 obs. of  19 variables:
##  $ ID                 : int  6449070 6539777 5403595 6759049 5409565 6485747 5852647 5361434 6152734
##  $ CODE_GENDER        : chr  "M" "F" "F" "F" ...
##  $ FLAG_OWN_CAR       : chr  "Y" "N" "N" "N" ...
##  $ FLAG_OWN_REALTY    : chr  "Y" "Y" "Y" "N" ...
##  $ CNT_CHILDREN       : int  2 0 0 1 0 0 1 0 0 0 ...
##  $ AMT_INCOME_TOTAL   : num  81000 225000 54000 157500 81000 ...
##  $ NAME_INCOME_TYPE   : chr  "Pensioner" "Commercial associate" "Working" "Working" ...
##  $ NAME_EDUCATION_TYPE: chr  "Secondary / secondary special" "Secondary / secondary special" "Seconda
##  $ NAME_FAMILY_STATUS : chr  "Married" "Separated" "Married" "Married" ...
##  $ NAME_HOUSING_TYPE  : chr  "House / apartment" "Rented apartment" "House / apartment" "House / apa
##  $ DAYS_BIRTH         : int  -18416 -13964 -15328 -14850 -23191 -15708 -14034 -19755 -21499 -24164 .
##  $ DAYS_EMPLOYED      : int  365243 -664 -855 -1416 365243 -5409 -4590 365243 -1504 365243 ...
##  $ FLAG_MOBIL         : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ FLAG_WORK_PHONE    : int  0 0 0 1 0 1 0 0 0 0 ...
##  $ FLAG_PHONE         : int  0 0 0 1 0 1 1 0 0 1 ...
##  $ FLAG_EMAIL         : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE    : chr  NA "Sales staff" "Laborers" "Managers" ...
##  $ CNT_FAM_MEMBERS    : int  4 1 2 3 1 2 3 2 2 2 ...
##  $ status             : chr  "0" "0" "0" "2" ...
```

```r
# Display the first few rows of the dataset
head(original)
```

```
##        ID CODE_GENDER FLAG_OWN_CAR FLAG_OWN_REALTY CNT_CHILDREN
## 1 6449070           M            Y               Y            2
## 2 6539777           F            N               Y            0
## 3 5403595           F            N               Y            0
## 4 6759049           F            N               N            1
## 5 5409565           F            N               Y            0
## 6 6485747           F            N               Y            0
##   AMT_INCOME_TOTAL     NAME_INCOME_TYPE           NAME_EDUCATION_TYPE
## 1            81000            Pensioner Secondary / secondary special
## 2           225000 Commercial associate Secondary / secondary special
## 3            54000              Working Secondary / secondary special
## 4           157500              Working Secondary / secondary special
## 5            81000            Pensioner              Higher education
## 6           126000              Working Secondary / secondary special
##   NAME_FAMILY_STATUS NAME_HOUSING_TYPE DAYS_BIRTH DAYS_EMPLOYED FLAG_MOBIL
## 1            Married House / apartment     -18416        365243          1
```

```
## 2          Separated  Rented apartment     -13964          -664          1
## 3            Married House / apartment     -15328          -855          1
## 4            Married House / apartment     -14850         -1416          1
## 5              Widow House / apartment     -23191        365243          1
## 6     Civil marriage House / apartment     -15708         -5409          1
##   FLAG_WORK_PHONE FLAG_PHONE FLAG_EMAIL OCCUPATION_TYPE CNT_FAM_MEMBERS status
## 1               0          0          0            <NA>               4      0
## 2               0          0          0     Sales staff               1      0
## 3               0          0          0        Laborers               2      0
## 4               1          1          0        Managers               3      2
## 5               0          0          0            <NA>               1      0
## 6               1          1          0  Medicine staff               2      0
```

The dataset has an initial of 67'614 rows and 19 columns (18 features and 1 label). We reload the dataset with the correct datatypes using the knowledge on the columns found on kaggle. It is crucial to load datatypes like ID as a string since sometimes the programm would change the value due to its formatting.

Flag columns are changed to integers with TRUE being 1 and FALSE being 0.

```r
# 1. Define Data Types (The 'dtype dictionary' equivalent in R)
col_types <- c(
  ID = "character",                  # Prevent ID from being parsed as math number
  CODE_GENDER = "factor",
  FLAG_OWN_CAR = "character",
  FLAG_OWN_REALTY = "character",
  CNT_CHILDREN = "integer",
  AMT_INCOME_TOTAL = "numeric",
  NAME_INCOME_TYPE = "character",
  NAME_EDUCATION_TYPE = "factor",
  NAME_FAMILY_STATUS = "factor",
  NAME_HOUSING_TYPE = "factor",
  DAYS_BIRTH = "integer",
  DAYS_EMPLOYED = "integer",
  FLAG_MOBIL = "integer",
  FLAG_WORK_PHONE = "integer",
  FLAG_PHONE = "integer",
  FLAG_EMAIL = "integer",
  OCCUPATION_TYPE = "factor",
  CNT_FAM_MEMBERS = "integer",
  status = "factor"
)

# 2. Reload the CSV with strict types
# na.strings includes empty strings to catch missing values in factors
data <- read.csv(path, colClasses = col_types, na.strings = c("NA", ""))

# 3. Post-Load Logic (Conversions that cannot be done directly in read.csv)
# Manually convert "Y" to 1 and "N" to 0
data$FLAG_OWN_CAR <- ifelse(data$FLAG_OWN_CAR == "Y", 1, 0)
data$FLAG_OWN_REALTY <- ifelse(data$FLAG_OWN_REALTY == "Y", 1, 0)

# (Optional) Verify that all flags are now technically integers
flags_list <- c("FLAG_MOBIL", "FLAG_WORK_PHONE", "FLAG_PHONE", "FLAG_EMAIL", "FLAG_OWN_CAR", "FLAG_OWN_
```

```r
# Force them all to be integers to be safe
data[flags_list] <- lapply(data[flags_list], as.integer)

str(data)
```

```
## 'data.frame':    67614 obs. of  19 variables:
##  $ ID                 : chr  "6449070" "6539777" "5403595" "6759049" ...
##  $ CODE_GENDER        : Factor w/ 2 levels "F","M": 2 1 1 1 1 1 1 1 2 1 ...
##  $ FLAG_OWN_CAR       : int  1 0 0 0 0 0 1 0 1 0 ...
##  $ FLAG_OWN_REALTY    : int  1 1 1 0 1 1 1 1 1 1 ...
##  $ CNT_CHILDREN       : int  2 0 0 1 0 0 1 0 0 0 ...
##  $ AMT_INCOME_TOTAL   : num  81000 225000 54000 157500 81000 ...
##  $ NAME_INCOME_TYPE   : chr  "Pensioner" "Commercial associate" "Working" "Working" ...
##  $ NAME_EDUCATION_TYPE: Factor w/ 5 levels "Academic degree",..: 5 5 5 5 2 5 5 5 5 5 ...
##  $ NAME_FAMILY_STATUS : Factor w/ 5 levels "Civil marriage",..: 2 3 2 2 5 1 2 2 2 2 ...
##  $ NAME_HOUSING_TYPE  : Factor w/ 6 levels "Co-op apartment",..: 2 5 2 2 2 2 2 2 2 2 ...
##  $ DAYS_BIRTH         : int  -18416 -13964 -15328 -14850 -23191 -15708 -14034 -19755 -21499 -24164 .
##  $ DAYS_EMPLOYED      : int  365243 -664 -855 -1416 365243 -5409 -4590 365243 -1504 365243 ...
##  $ FLAG_MOBIL         : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ FLAG_WORK_PHONE    : int  0 0 0 1 0 1 0 0 0 0 ...
##  $ FLAG_PHONE         : int  0 0 0 1 0 1 1 0 0 1 ...
##  $ FLAG_EMAIL         : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE    : Factor w/ 18 levels "Accountants",..: NA 15 9 11 NA 12 12 NA 9 NA ...
##  $ CNT_FAM_MEMBERS    : int  4 1 2 3 1 2 3 2 2 2 ...
##  $ status             : Factor w/ 8 levels "0","1","2","3",..: 1 1 1 3 1 1 1 1 1 1 ...
```

After each transformation a visual check using str(data) is used to check whether changes have been done as wished.

Next, we check for duplicated rows or rows with the same ID

```r
# Check for duplicated rows
duplicated_rows <- data[duplicated(data),]
cat("Number of duplicated rows:", nrow(duplicated_rows), "\n") # 0
```

```
## Number of duplicated rows: 0
```

```r
# Check for duplicated IDs
duplicated_ids <- data[duplicated(data$ID),]
cat("Number of duplicated IDs:", nrow(duplicated_ids), "\n") # 0
```

```
## Number of duplicated IDs: 0
```

```r
# Check for duplicated rows when excluding ID-column
duplicates_mask <- duplicated(data[, !names(data) %in% "ID"])

# Count the number of duplicates
num_duplicates <- sum(duplicates_mask)

cat("Number of duplicate rows (excluding ID):", num_duplicates, "\n")
```

```
## Number of duplicate rows (excluding ID): 23
```

```r
cat("Percentage of total dataset:", round((num_duplicates / nrow(data)) * 100, 2), "%\n")
```

```
## Percentage of total dataset: 0.03 %
```

Since ID is an identifier of a customer, it will not be fed into the model. Furthermore, it is highly unlikely that two distinct people have the same age, years worked, etc. We find 23 rows with similar information (excluding the ID), which can be dropped.

```r
# Optional: Remove the duplicates found above
data <- data[!duplicates_mask, ]
```

3.2. Data summary

Now we can check the summary of all columns to identify missing values and get an overview of the data.

```r
# Summary of the dataset
summary(data)
```

```
##       ID            CODE_GENDER  FLAG_OWN_CAR    FLAG_OWN_REALTY
##  Length:67591       F:43910     Min.   :0.0000   Min.   :0.0000
##  Class :character   M:23681     1st Qu.:0.0000   1st Qu.:0.0000
##  Mode  :character               Median :0.0000   Median :1.0000
##                                 Mean   :0.3625   Mean   :0.6881
##                                 3rd Qu.:1.0000   3rd Qu.:1.0000
##                                 Max.   :1.0000   Max.   :1.0000
##
##   CNT_CHILDREN     AMT_INCOME_TOTAL  NAME_INCOME_TYPE
##  Min.   : 0.0000   Min.   :  26100   Length:67591
##  1st Qu.: 0.0000   1st Qu.: 112500   Class :character
##  Median : 0.0000   Median : 157500   Mode  :character
##  Mean   : 0.4206   Mean   : 178864
##  3rd Qu.: 1.0000   3rd Qu.: 225000
##  Max.   :19.0000   Max.   :6750000
##
##                      NAME_EDUCATION_TYPE            NAME_FAMILY_STATUS
##  Academic degree              :   38    Civil marriage      : 6014
##  Higher education             :16885    Married             :44889
##  Incomplete higher            : 2305    Separated           : 4125
##  Lower secondary              :  715    Single / not married: 9526
##  Secondary / secondary special:47648    Widow               : 3037
##
##
##            NAME_HOUSING_TYPE   DAYS_BIRTH     DAYS_EMPLOYED      FLAG_MOBIL
##  Co-op apartment    :  227   Min.   :-25201   Min.   :-17531   Min.   :1
##  House / apartment  :60287   1st Qu.:-19439   1st Qu.: -2886   1st Qu.:1
##  Municipal apartment: 2303   Median :-15593   Median : -1305   Median :1
##  Office apartment   :  586   Mean   :-15915   Mean   : 62044   Mean   :1
##  Rented apartment   : 1019   3rd Qu.:-12348   3rd Qu.:  -321   3rd Qu.:1
##  With parents       : 3169   Max.   : -7489   Max.   :365243   Max.   :1
##
##  FLAG_WORK_PHONE    FLAG_PHONE       FLAG_EMAIL        OCCUPATION_TYPE
```

```
##  Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Laborers   :12421
##  1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.0000   Sales staff: 6897
##  Median :0.0000   Median :0.0000   Median :0.0000   Core staff : 6058
##  Mean   :0.2027   Mean   :0.2742   Mean   :0.1005   Managers   : 4949
##  3rd Qu.:0.0000   3rd Qu.:1.0000   3rd Qu.:0.0000   Drivers    : 4425
##  Max.   :1.0000   Max.   :1.0000   Max.   :1.0000   (Other)    :12147
##                                                     NA's       :20694
##  CNT_FAM_MEMBERS      status
##  Min.   : 1.000   0      :52133
##  1st Qu.: 2.000   1      : 6491
##  Median : 2.000   X      : 5790
##  Mean   : 2.174   C      : 1805
##  3rd Qu.: 3.000   2      :  699
##  Max.   :20.000   5      :  368
##                   (Other):  305
```

The`summary()`function acts as a critical diagnostic tool in the early stages of preprocessing by providing a five-number summary for numerical variables and data type information for categorical ones. This quick overview helps identify scale differences between variables, such as the vast difference between`CNT_CHILDREN`and`AMT_INCOME_TOTAL`, which indicates a need for normalization. Furthermore, it allows for the immediate detection of outliers or physically impossible values, as well as the identification of constant variables with zero variance that add no predictive value to the model.

The summary output reveals several critical data quality issues that require handling. Most notably,`DAYS_EMPLOYED` contains a significant artifact with a maximum value of 365'243, which is roughly 1,000 years. This is likely a placeholder for pensioners or unemployed individuals and severely skews the mean compared to the median, requiring correction before training. Additionally,`FLAG_MOBIL`shows a minimum, mean, and maximum of 1, indicating that every row contains the same value; this feature offers no discriminative information and should be removed. It is also observed that `DAYS_BIRTH` and `DAYS_EMPLOYED` are recorded as negative values, which should be converted to positive figures for better interpretability. Finally, many categorical variables like `CODE_GENDER` and `NAME_EDUCATION_TYPE` are currently stored as characters and must be converted to factors to properly utilize them in the modeling phase.

3.3. Univariate columns

As we already seen in the summary(), FLAG_MOBIL shows a mean, minimum and max of 1, indicating that there is only one value present. We check this with the table() function. We do this simultaneously for the other FLAG-columns to see whether there is a strong imbalance in the column.

```
unique(data$FLAG_OWN_CAR)
```

```
## [1] 1 0
```

```
unique(data$FLAG_OWN_REALTY)
```

```
## [1] 1 0
```

```
table(data$FLAG_OWN_CAR)
```

```
##
##     0     1
## 43090 24501
```

```r
table(data$FLAG_OWN_REALTY)
```

```
##
##     0     1
## 21082 46509
```

```r
table(data$FLAG_MOBIL) # Only True
```

```
##
##     1
## 67591
```

```r
table(data$FLAG_WORK_PHONE)
```

```
##
##     0     1
## 53891 13700
```

```r
table(data$FLAG_PHONE)
```

```
##
##     0     1
## 49057 18534
```

```r
table(data$FLAG_EMAIL)
```

```
##
##     0     1
## 60798  6793
```

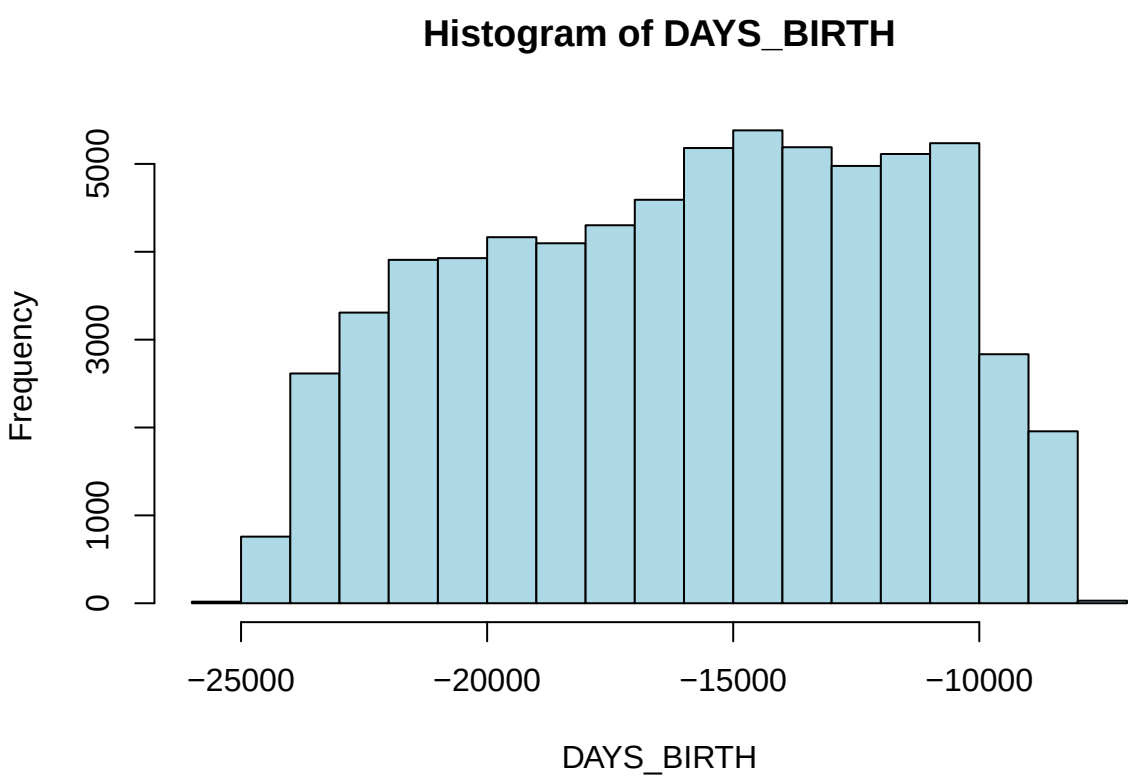
```r
# Drop columns with no variance
data <- data %>%
  select(-FLAG_MOBIL)
```

The column FLAG_MOBIL is redundant since it only has one distinct value and can therefore be dropped.

3.4. Analysis of days_of_birth and days_employed

Both of these columns show in the summary negative values. In the datadictionary we learn that DAYS_BIRTH is the age of the customer. It is a backwards count from the current day (0) where -1 would mean yesterday. DAYS_EMPLOYED is the start date of employment, it holds the same logic as DAYS_BIRTH. If the value is positive, it means that the person is currently unemployed.

```r
# Histogram of DAYS_BIRTH
hist(data$DAYS_BIRTH,
     main="Histogram of DAYS_BIRTH",
     xlab="DAYS_BIRTH",
     col="lightblue")
```




```
# Using the summary we can see the distribution of the data
summary(data$DAYS_BIRTH)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -25201  -19439  -15593  -15915  -12348   -7489
```

On the first glance we do not see any outliers. However, we can see that the age is displayed as a negative value. We correct this by dividing by 365 and taking the absolute value. This computes the age in years, which is easier to understand whether or not the data makes sense. Furthermore, we will compute rounded numbers. The model most likely won't benefit from a day-level precision, furthermore most financial relationships are tied to age ranges, which is why this method is sufficient.

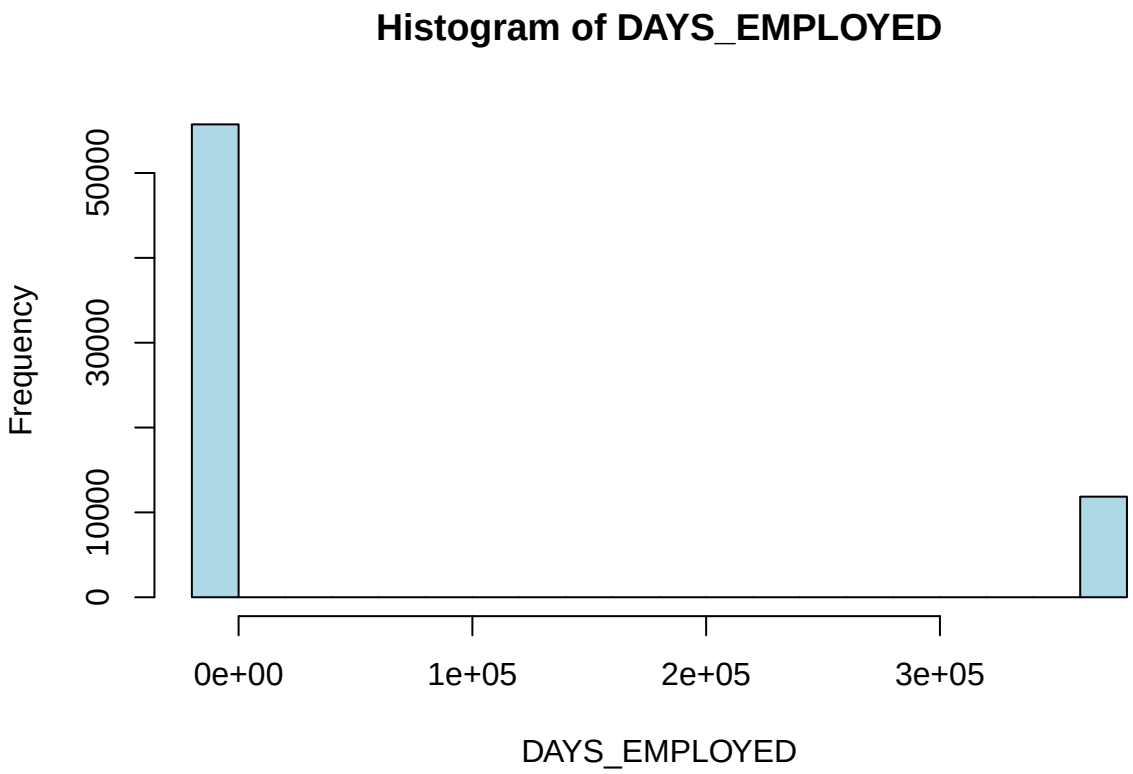
```
# Correct DAYS_BIRTH to positive years
data <- data %>%
  mutate(AGE = round(abs(DAYS_BIRTH) / 365.25)) %>%
  select(-DAYS_BIRTH)
# Histogram of AGE_YEARS
hist(data$AGE,
     main="Histogram of AGE",
     xlab="AGE (years)",
     col="lightgreen")
```


```
# Using the summary we can see the distribution of the data
summary(data$AGE)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   21.00   34.00   43.00   43.58   53.00   69.00
```

The second strange column is 'DAYS_UNEMPLOYED'. We will make a histogram to understand the distribution.

```
# Histogram of DAYS_EMPLOYED
hist(data$DAYS_EMPLOYED,
     main="Histogram of DAYS_EMPLOYED",
     xlab="DAYS_EMPLOYED",
     col="lightblue")
```




```
# Using the summary we can see the distribution of the data
summary(data$DAYS_EMPLOYED)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -17531   -2886   -1305   62044    -321  365243
```

We know that the unemployed have the sentinel value of 365'243. We therefore would expect the same amount of NA-values in OCCUPATION_TYPE.

```
# Number of rows where days_employed == 365243
print(nrow(data[data$DAYS_EMPLOYED == 365243, ])) # 11'855 rows
```

```
## [1] 11855
```

```
# Look for occupation type where days_employed is 365243
unique(data$OCCUPATION_TYPE[data$DAYS_EMPLOYED == 365243]) # They are ALL NA meaning that they do not w
```

```
## [1] <NA>
## 18 Levels: Accountants Cleaning staff Cooking staff Core staff ... Waiters/barmen staff
```

```
# Count how many rows have NA in occupation type
nrow(data[is.na(data$OCCUPATION_TYPE), ]) # 20'699 rows --> not all are pensioned
```

```
## [1] 20694
```

```r
# Show name_income_type where there is an NA in occupation type
table(data$NAME_INCOME_TYPE[is.na(data$OCCUPATION_TYPE)])
```

```
##
## Commercial associate           Pensioner       State servant
##                 2499               11877                 878
##              Student             Working
##                    2                5438
```

```r
# Show all occupation_type where name_income_type is Pensioner
table(data$OCCUPATION_TYPE[data$NAME_INCOME_TYPE == 'Pensioner'])
```

```
##
##           Accountants       Cleaning staff         Cooking staff
##                     3                    0                     4
##            Core staff              Drivers High skill tech staff
##                    24                    8                     6
##              HR staff           IT staff              Laborers
##                     0                    0                    28
##    Low-skill Laborers             Managers       Medicine staff
##                     1                   13                     7
## Private service staff        Realty agents           Sales staff
##                     0                    0                     8
##           Secretaries       Security staff  Waiters/barmen staff
##                     1                    2                     0
```

```r
# Show the amount of name_income_type where days_employed is 365243
table(data$NAME_INCOME_TYPE[data$DAYS_EMPLOYED == 365243])
```

```
##
## Pensioner
##     11855
```

```r
# Total amount of pensioners in the dataset
table(data$NAME_INCOME_TYPE == 'Pensioner')
```

```
##
## FALSE  TRUE
## 55609 11982
```

We see that there are 11'855 rows with positive values, indicating 11'855 unemployed which happen all to be pensioners. Looking at the OCCUPATION_TYPE we can see that those with the sentinel value have an NA-value in OCCUPATION_TYPE as well.

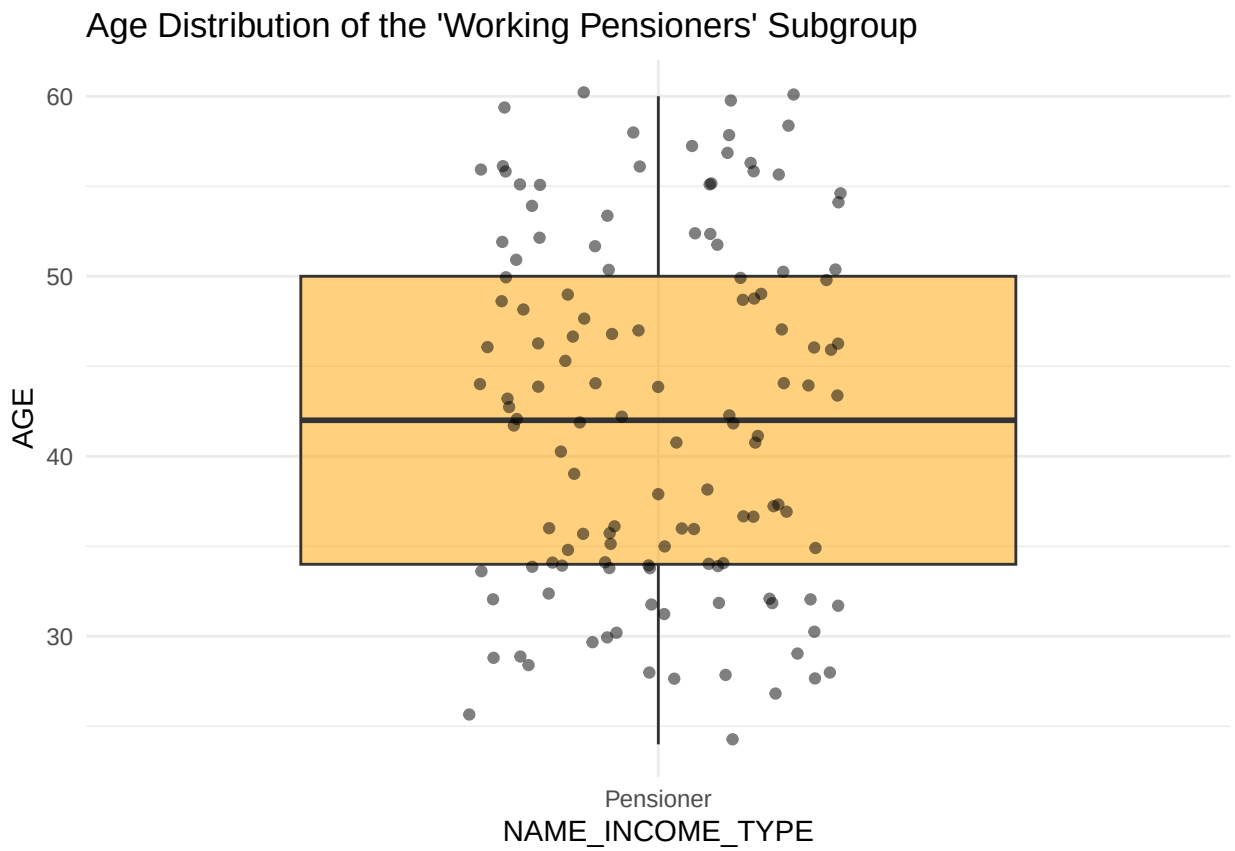
- Sentinel Group (11'855 rows)
  - There are exactly 11'855 rows where DAYS_EMPLOYED is 365'243.
  - Every single one of these rows has NAME_INCOME_TYPE as 'Pensioner'.
  - Every single one of these rows has OCCUPATION_TYPE as NA.
  - These represent the 'standard' retirees in the dataset.

- Missing Occupation Group (20'694 rows)
  - There are 20'694 rows total where OCCUPATION_TYE is NA.
  - 11'877 of these are Pensioners. This is slightly higher than the 11'855 sentinel count, implying that 22 pensioners have valid days but no job title.
  - The remaining 8'817 rows are active workers who simply did not fill in their specific job title.
- Hidden subgroup (127 rows)
  - Total Pensioners (11'982) - Sentinel Pensioners (11'855) = 127 cases..
  - These rows are labeled 'Pensioner' but have valid employment days and often specific job titles.

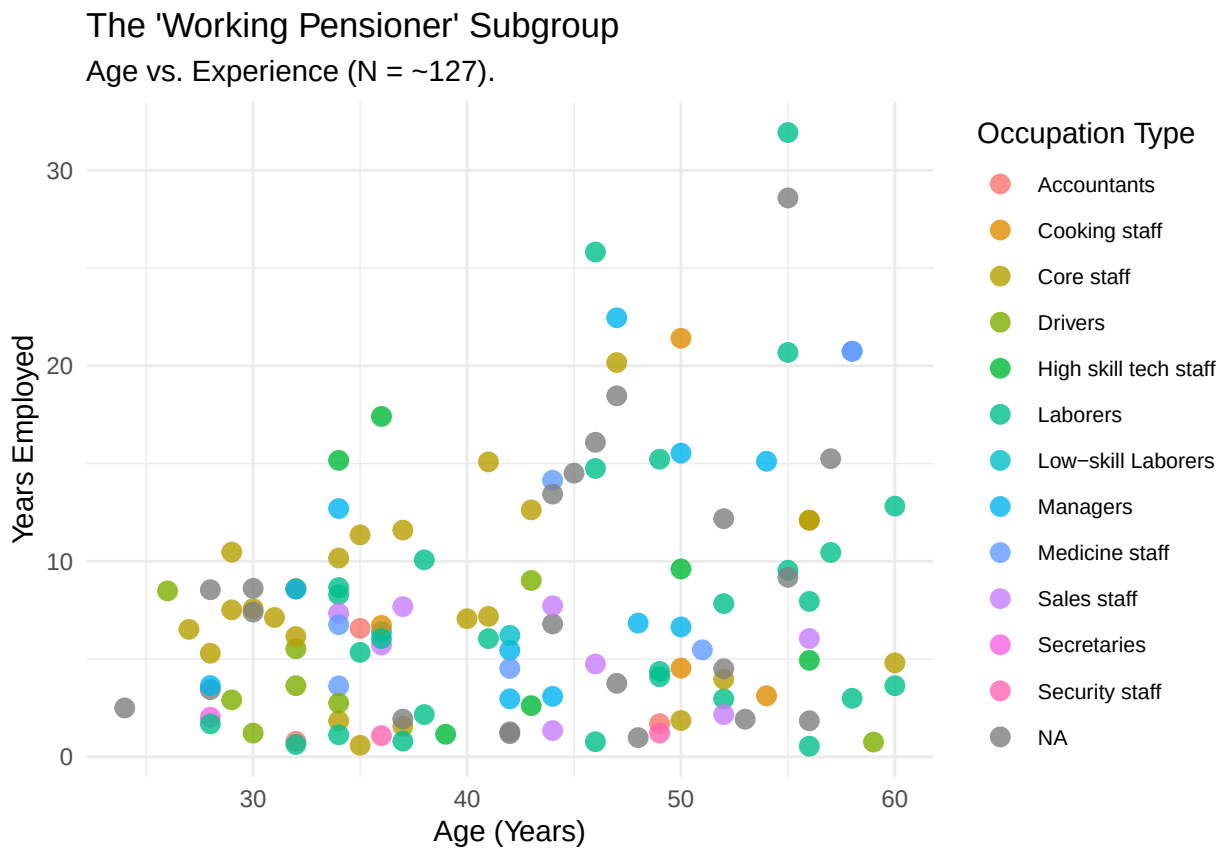
These inconsistencies necessitate further investigation into their age and employment history to determine if they are valid 'working retirees' or data entry errors.

NOTE: In the first 2 iterations of this sentinel value these specific findings were not identified. In the first iteration all NA-values in OCCUPATION_TYPE were changed to 'Unknown' and a Flag was made for those who were retired. In a second iteration the flag was dropped due to it's redundancy and the rows were imputed with 'Retired' instead of 'Unknown' where applicable. Due to a final data check and mediocre accuracy in the trained model, these new findings were identified.

```
# 1. Prepare the specific subgroup data
working_pensioners_data <- data %>%
  filter(NAME_INCOME_TYPE == "Pensioner", DAYS_EMPLOYED != 365243) %>%
  mutate(
    EXPERIENCE_YEARS = abs(DAYS_EMPLOYED) / 365.25
  )

# Boxplot of Age for Working Pensioners specifically
# To see if they are actually 'Old' or if they are younger people on disability pension
working_pensioners <- working_pensioners_data %>% filter(NAME_INCOME_TYPE == 'Pensioner')

ggplot(working_pensioners, aes(x = NAME_INCOME_TYPE, y = AGE)) +
  geom_boxplot(fill = "orange", alpha = 0.5) +
  geom_jitter(width = 0.2, alpha = 0.5) +
  labs(title = "Age Distribution of the 'Working Pensioners' Subgroup") +
  theme_minimal()
```




```
# Scatter Plot: Focused on the Anomaly
ggplot(working_pensioners_data, aes(x = AGE, y = EXPERIENCE_YEARS, color = OCCUPATION_TYPE)) +
  geom_point(size = 3, alpha = 0.8) +
  labs(
    title = "The 'Working Pensioner' Subgroup",
    subtitle = "Age vs. Experience (N = ~127).",
    x = "Age (Years)",
    y = "Years Employed",
    color = "Occupation Type"
  ) +
  theme_minimal() +
  theme(
    legend.position = "right",
    legend.text = element_text(size = 8)
  )
```



The boxplot above isolates the age distribution of the 'working pensioner' subgroup. Here we see a median age of approximately 42 years old, with a range from 25 to 60 years old. There is a complete absence of individuals over the standard retirement age (60+), which is statistically impossible for a genuine random sample of pensioners.

The age profile of this group is identical to that of the active workforce. It does not reflect a demographic of retirees or even typical disability pensioners (who would likely show a wider age variance including older individuals).

The scatterplot maps age against years employed, coloring points by there occupation type. T he subgroup is not limited to 'consultancy' or 'specialized' post-retirement roles. Instead, we see physically demanding and standard roles such as laborers, drivers, core staff, and security staff. Furthermore, the years-employed scales naturally with age, mimicking the standard accumulation of experience found in regular employees.

The "Working Pensioners" do not form a distinct cluster. They are buried within the main distribution of the "Working" class. If they were a distinct demographic (e.g., wealthy early retirees), we would expect them to cluster differently in terms of experience-to-age ratio. Their perfect alignment with the standard workforce further suggests they are standard workers, simply mislabeled.

Based on the evidence above, we conclude that these 127 rows represent classification errors. Due to the small amount of rows, we decide to drop them.

NOTE: This was done in the final iteration of data cleaning. The previous iteration including these rows got a maximum validation accuracy in the neural network of 84%.

```
print(paste("Total rows before:", nrow(data)))
```

```
## [1] "Total rows before: 67591"
```

```r
# 2. Filter out the specific anomaly
# Logic: Keep rows that are NOT (Pensioner AND have Valid Employment Days)
data <- data %>%
  filter(!(NAME_INCOME_TYPE == "Pensioner" & DAYS_EMPLOYED != 365243))

# 3. Verify the drop
print(paste("Total rows after:", nrow(data)))
```

```
## [1] "Total rows after: 67464"
```

```r
# Double check: Should be 0
remaining_anomalies <- nrow(data[data$NAME_INCOME_TYPE == "Pensioner" & data$DAYS_EMPLOYED != 365243, ])
print(paste("Anomalies remaining:", remaining_anomalies))
```

```
## [1] "Anomalies remaining: 0"
```

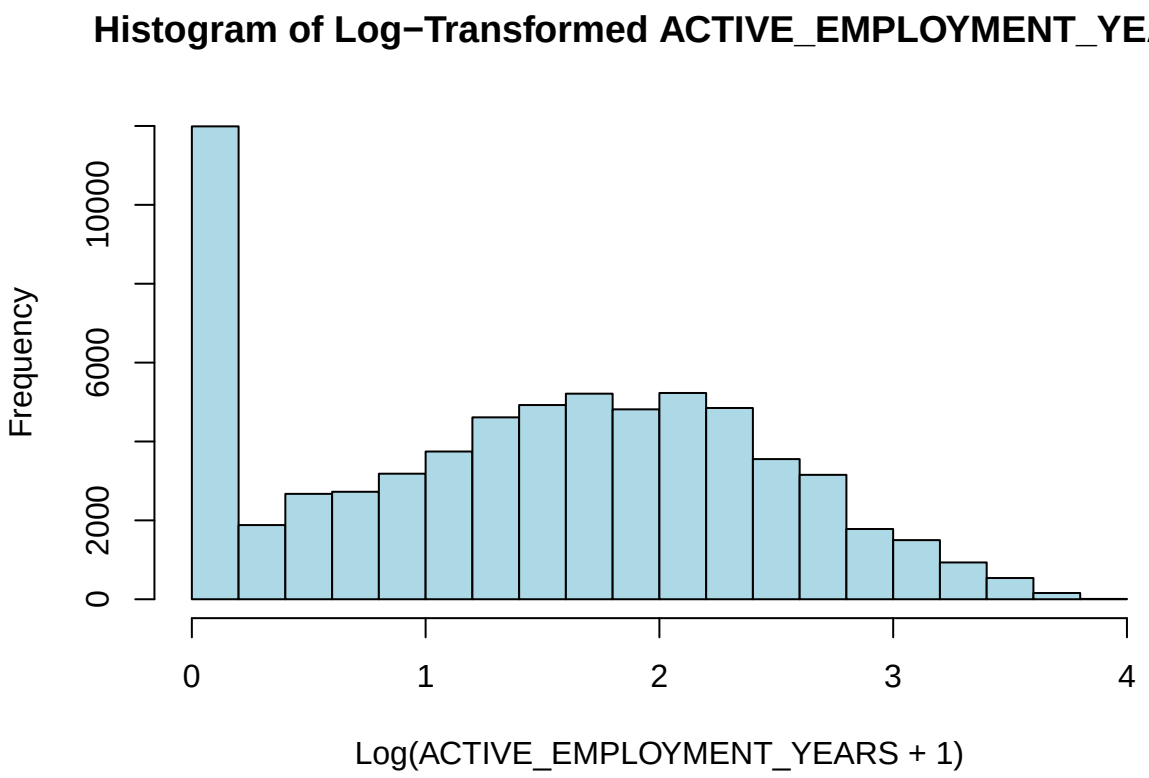
We can now handle the remaining NA-values in OCCUPATION_TYPE.

```r
# 1. Reclassify OCCUPATION_TYPE
data <- data %>%
  mutate(
    OCCUPATION_TYPE = as.character(OCCUPATION_TYPE),
    OCCUPATION_TYPE = case_when(
      DAYS_EMPLOYED == 365243 ~ "Retired", # Only the valid pensioners remain now
      is.na(OCCUPATION_TYPE)  ~ "Unknown", # The missing workers
      TRUE ~ OCCUPATION_TYPE
    ),
    OCCUPATION_TYPE = as.factor(OCCUPATION_TYPE)
  )

# 2. Calculate ACTIVE_EMPLOYMENT_YEARS cleanly in one step
# We set sentinel values (365243) directly to 0 to avoid the "1000 year" glitch
data <- data %>%
  mutate(
    ACTIVE_EMPLOYMENT_YEARS = ifelse(
      DAYS_EMPLOYED == 365243,
      0, # in a previous iteration this was set so that the years_worked was 60
      abs(DAYS_EMPLOYED) / 365.25
    )
  )

# 3. Histogram
hist(data$ACTIVE_EMPLOYMENT_YEARS,
     breaks = seq(0, max(data$ACTIVE_EMPLOYMENT_YEARS, na.rm=TRUE) + 1, by=1),
     main = "Histogram of Employment Duration (Pensioners = 0)",
     xlab = "Years Employed",
     col = "lightblue",
     border = "white")
```

As we see a very skewed distribution we will make a log-transformation to use in the Neural Network.

Note: Empirical results from our previous neural network iterations confirmed that preserving 'Unknown' as a distinct category, rather than applying statistical imputation, significantly improved the model's ability to differentiate risk. This suggests that the absence of occupation data is Missing Not At Random (MNAR) and contains a latent 'informational signal', representing a specific risk profile among applicants who choose not to disclose their professional details. By avoiding methods like KNN or Mode imputation, we prevent the dilution of this signal and allow the network to learn the predictive weight of non-disclosure itself.

```
# Log-transform ACTIVE_EMPLOYMENT_YEARS and drop the original
data <- data %>%
  mutate(ACTIVE_EMPLOYMENT_YEARS_LOG = log1p(ACTIVE_EMPLOYMENT_YEARS)) %>%
  select(-c(DAYS_EMPLOYED, ACTIVE_EMPLOYMENT_YEARS))
# Histogram of log-transformed ACTIVE_EMPLOYMENT_YEARS
hist(data$ACTIVE_EMPLOYMENT_YEARS_LOG,
     main="Histogram of Log-Transformed ACTIVE_EMPLOYMENT_YEARS",
     xlab="Log(ACTIVE_EMPLOYMENT_YEARS + 1)",
     col="lightblue")
```



3.5. Analysis of family members and children

We next examine CNT_FAM_MEMBERS and CNT_CHILDREN.

```
# Identify extreme outliers
outlier_fam_members <- data[data$CNT_FAM_MEMBERS > 10, ]
print(outlier_fam_members)
```

```
##            ID CODE_GENDER FLAG_OWN_CAR FLAG_OWN_REALTY CNT_CHILDREN
## 23533 5105054           F            0               1           19
## 58806 5931568           F            0               1           12
##       AMT_INCOME_TOTAL     NAME_INCOME_TYPE           NAME_EDUCATION_TYPE
## 23533           112500              Working Secondary / secondary special
## 58806           337500 Commercial associate Secondary / secondary special
##         NAME_FAMILY_STATUS NAME_HOUSING_TYPE FLAG_WORK_PHONE FLAG_PHONE
## 23533 Single / not married House / apartment               1          1
## 58806              Married House / apartment               0          0
##       FLAG_EMAIL      OCCUPATION_TYPE CNT_FAM_MEMBERS status AGE
## 23533          0 Waiters/barmen staff              20      0  30
## 58806          0           Core staff              14      0  39
##       ACTIVE_EMPLOYMENT_YEARS_LOG
## 23533                    1.803892
## 58806                    2.130559
```

A record showing a 38-year-old with 12 children (and a family size of 14) is statistically improbable within this demographic context and likely represents a data entry error. Given the extreme rarity (2 rows), these were removed to ensure the model is not biased by impossible family-to-income ratios.

```
data <- data[data$CNT_FAM_MEMBERS <= 10, ]
```

A mathematical constraint must exist where CNT_FAM_MEMBERS >= CNT_CHILDREN + 1. A validation check identified 31 instances where the total family count was strictly less than the number of children reported.

```
# Identify anomalous rows where children outnumber total family members
anomaly_rows <- data[data$CNT_FAM_MEMBERS < data$CNT_CHILDREN, ]
print(nrow(anomaly_rows)) # 31 rows
```

```
## [1] 31
```

To rectify inconsistencies without losing data, we applied a deterministic correction based on NAME_FAMILY_STATUS. This assumes the child count is the "source of truth" and adjusts the total family count to include the necessary adults.

- If 'married' or 'civil marriage', the formula is CNT_CHILDREN + 2
- if 'separated', 'widowed', or 'single', the formula is CNT_CHILDREN + 1

```
# Identify logic mask
anomaly_mask <- data$CNT_FAM_MEMBERS < data$CNT_CHILDREN

# Apply corrections
data <- data %>%
  mutate(CNT_FAM_MEMBERS = case_when(
    anomaly_mask & NAME_FAMILY_STATUS %in% c("Married", "Civil marriage") ~ CNT_CHILDREN + 2,
    anomaly_mask & NAME_FAMILY_STATUS %in% c("Separated", "Widow", "Single / not married") ~ CNT_CHILDR
    TRUE ~ CNT_FAM_MEMBERS
  ))

# Final verification
nrow(data[data$CNT_FAM_MEMBERS < data$CNT_CHILDREN, ]) # Should be 0
```

```
## [1] 0
```

This correction resolved the logical anomalies in all 31 cases. By using the family status to determine the number of adults, we maintain the integrity of the family-unit data which is often a strong predictor for debt-to-income ratios in credit scoring.

3.6. Analysis of income amount

This chapter outlines the analysis and treatment of AMT_INCOME_TOTAL variable, focusing on outlier detection, logical consistency with occupation, and data normalization.

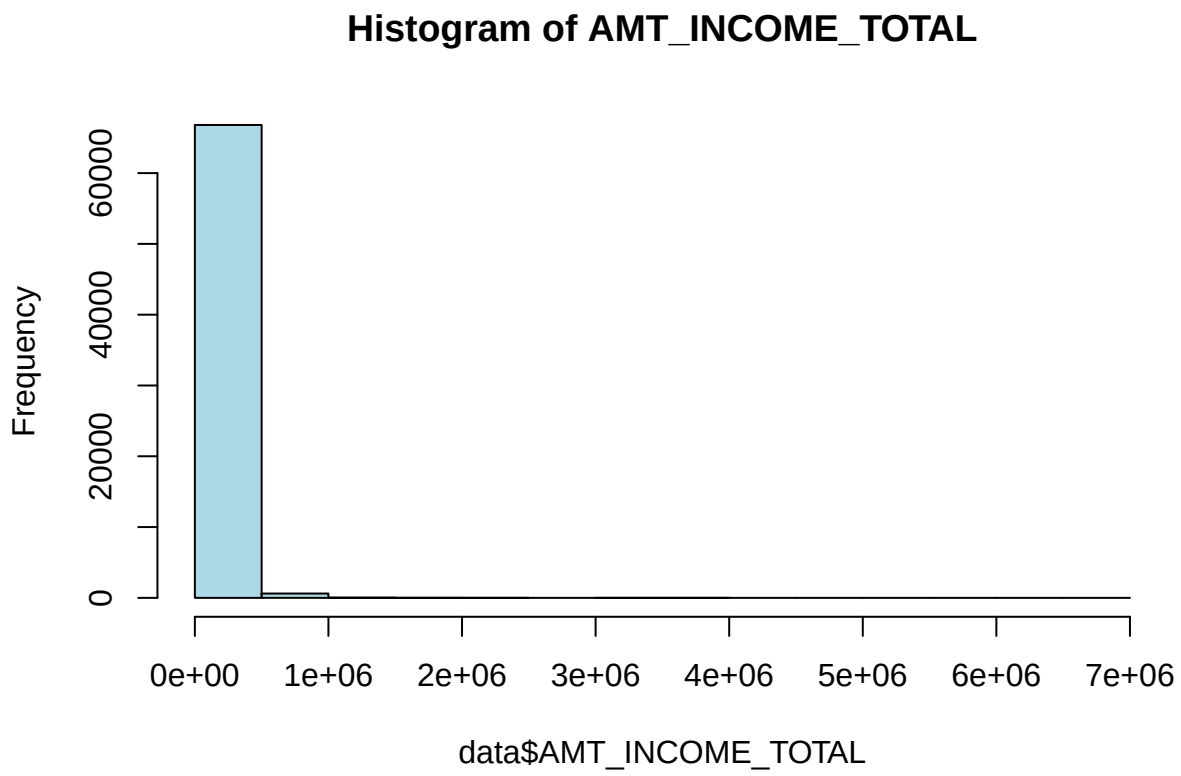
```r
summary(data$AMT_INCOME_TOTAL)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   26100  112500  157500  178808  225000 6750000
```

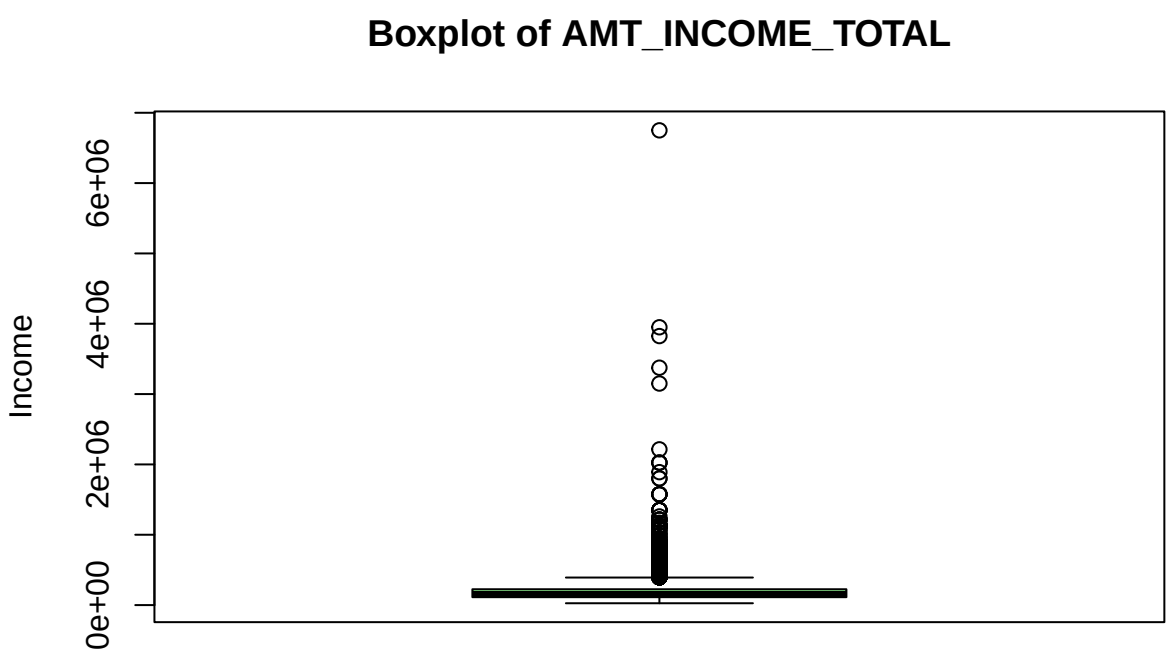
The initial summary of 'AMT_INCOME_TOTAL' reveals a highly skewed distribution with a significant gap between the median ($157'500) and the maximum ($6'750'000).

Visual inspection through histograms and boxplots confirms a heavy right-tail distribution dominated by extreme outliers.

```r
# Standard Histogram
hist(data$AMT_INCOME_TOTAL, main="Histogram of AMT_INCOME_TOTAL", col="lightblue")
```

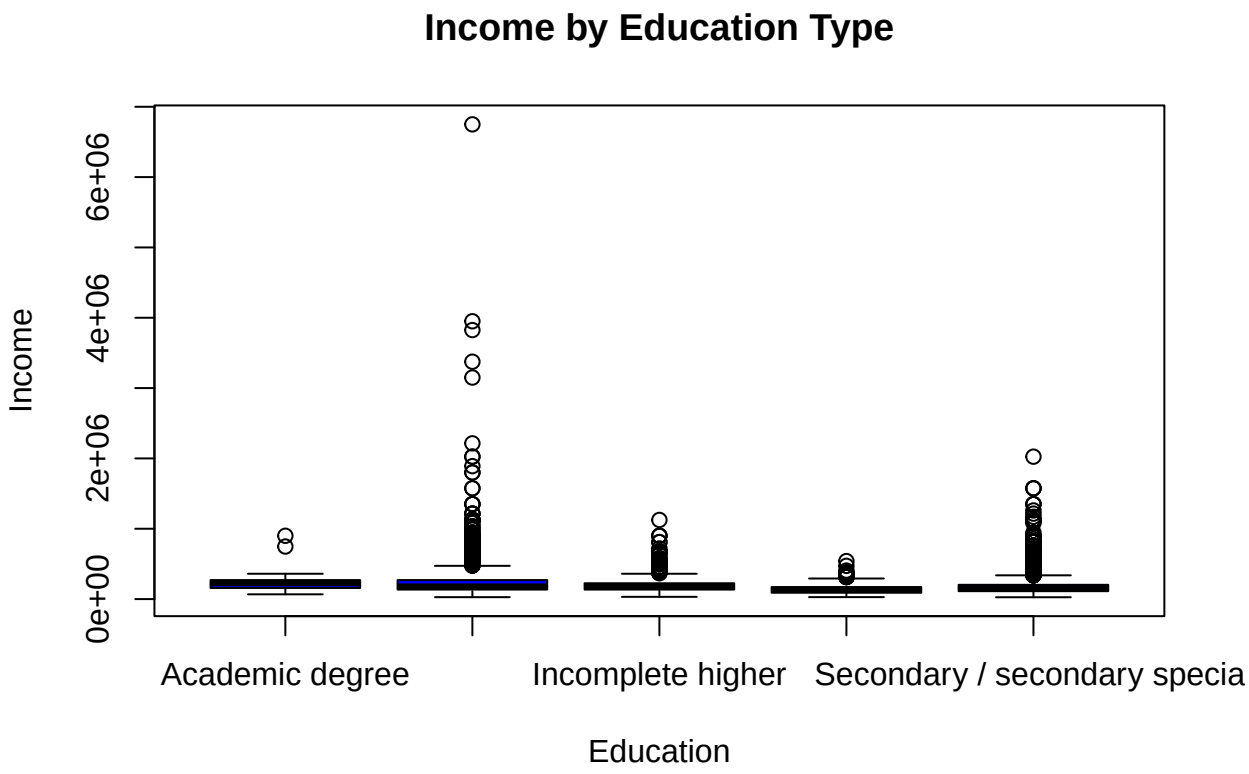



```r
# Boxplot to visualize the distance of outliers from the interquartile range
boxplot(data$AMT_INCOME_TOTAL, main="Boxplot of AMT_INCOME_TOTAL", ylab="Income", col="lightgreen")
```

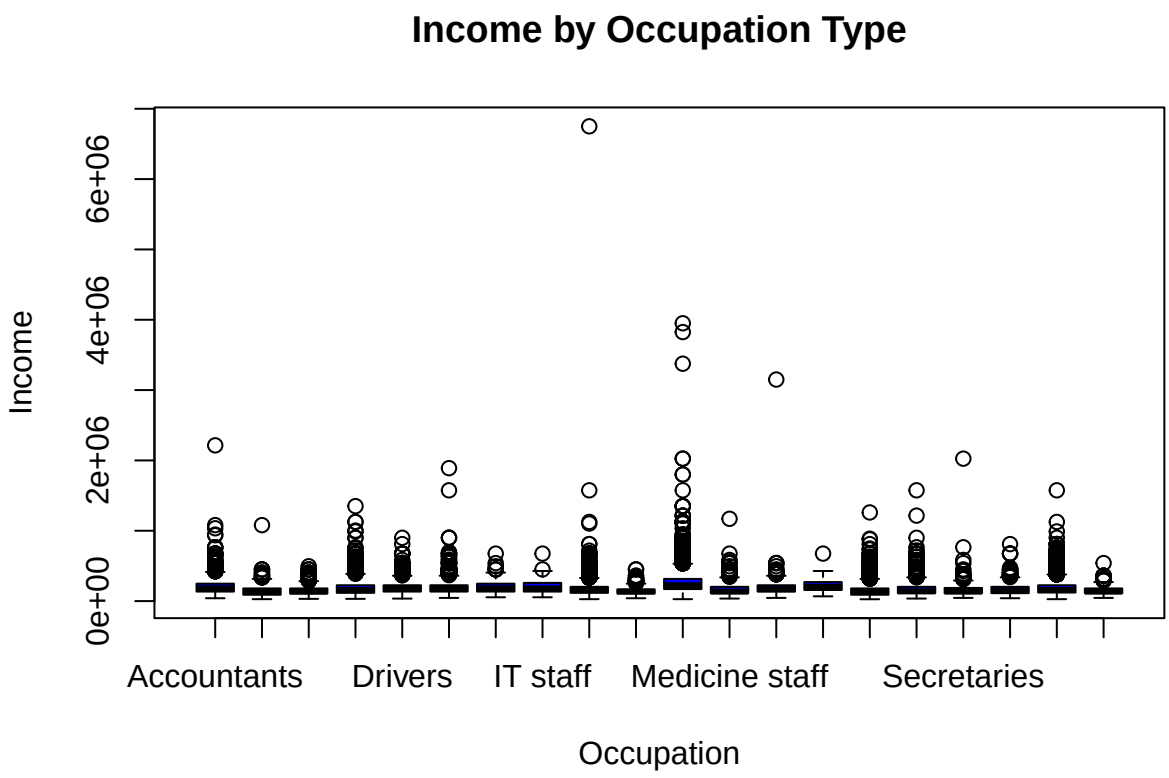


To determine if the high-income values are legitimate or data entry errors, we analyzed income against NAME_EDUCATION_TYOE and OCCUPATION_TYPE.

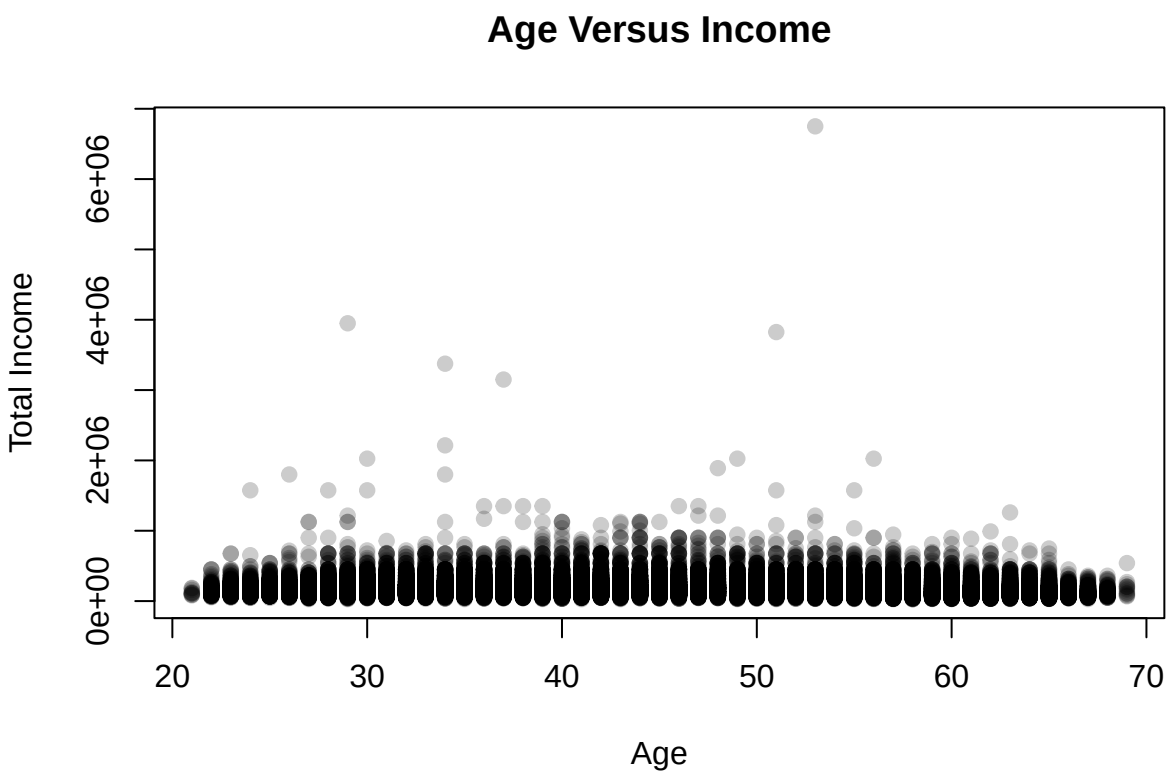
```
# Income by Education
boxplot(data$AMT_INCOME_TOTAL ~ data$NAME_EDUCATION_TYPE,
        main="Income by Education Type", ylab = "Income", xlab = "Education", col="blue")
```




```
# Income by Occupation
boxplot(data$AMT_INCOME_TOTAL ~ data$OCCUPATION_TYPE,
        main="Income by Occupation Type", ylab = "Income", xlab = "Occupation", col="blue")
```




```
# Age vs Income
plot(data$AMT_INCOME_TOTAL ~ data$AGE, main="Age Versus Income",
     ylab="Total Income", xlab="Age", pch=19, col=rgb(0,0,0,0.2))
```



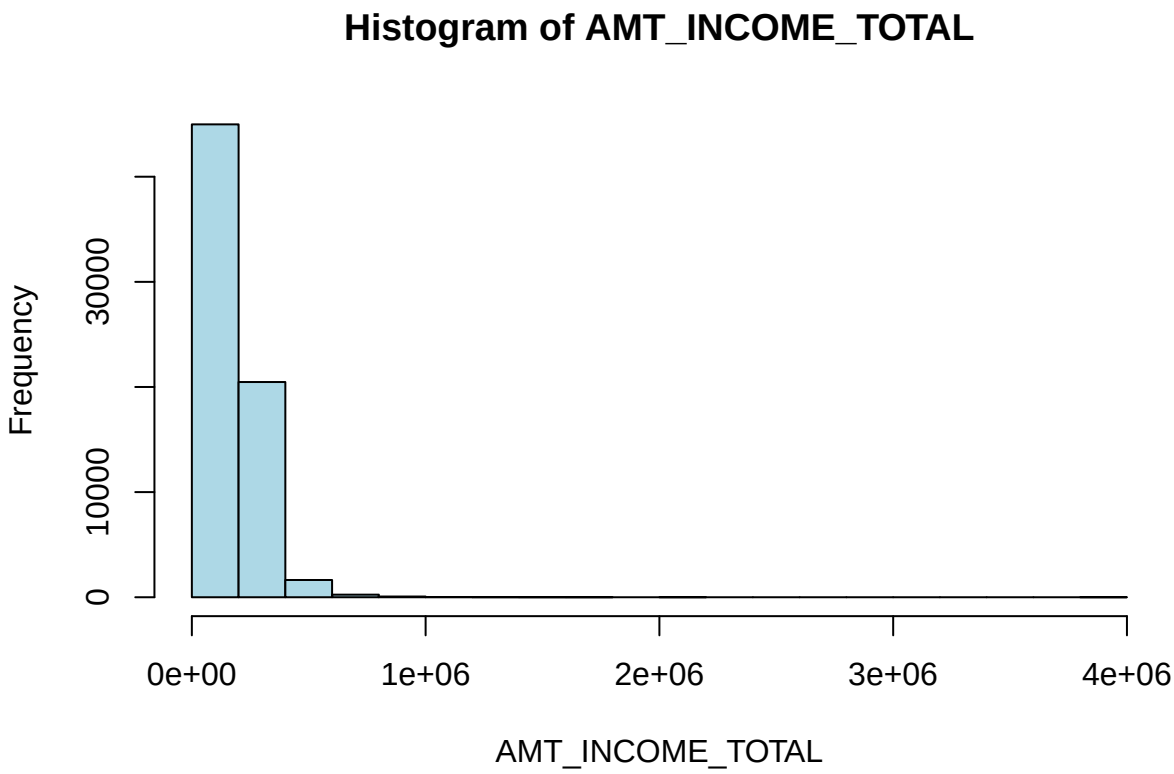

```
data %>%
  arrange(desc(AMT_INCOME_TOTAL)) %>%
  head(10)
```

```
##         ID CODE_GENDER FLAG_OWN_CAR FLAG_OWN_REALTY CNT_CHILDREN
## 1  5987963           M            1                0            0
## 2  5680511           M            1                1            1
## 3  5163386           M            1                0            0
## 4  5873449           M            1                1            1
## 5  6544043           F            1                1            1
## 6  5993384           F            1                1            2
## 7  6022090           F            1                1            0
## 8  6651834           M            1                0            0
## 9  5996392           M            1                1            0
## 10 6423704           M            0                1            0
##    AMT_INCOME_TOTAL     NAME_INCOME_TYPE           NAME_EDUCATION_TYPE
## 1           6750000              Working              Higher education
## 2           3950060 Commercial associate              Higher education
## 3           3825000 Commercial associate              Higher education
## 4           3375000              Working              Higher education
## 5           3150000        State servant              Higher education
## 6           2214117 Commercial associate              Higher education
## 7           2025000              Working Secondary / secondary special
## 8           2025000              Working              Higher education
## 9           2025000 Commercial associate              Higher education
## 10          1890000              Working              Higher education
```

```
##      NAME_FAMILY_STATUS NAME_HOUSING_TYPE FLAG_WORK_PHONE FLAG_PHONE FLAG_EMAIL
## 1               Married House / apartment               1          1          0
## 2               Married      With parents               1          1          0
## 3         Civil marriage House / apartment               0          0          1
## 4               Married House / apartment               0          1          0
## 5         Civil marriage House / apartment               1          0          0
## 6               Married House / apartment               1          0          1
## 7               Married House / apartment               1          0          0
## 8               Married House / apartment               1          1          0
## 9               Married House / apartment               0          0          0
## 10 Single / not married House / apartment               0          0          0
##           OCCUPATION_TYPE CNT_FAM_MEMBERS status AGE ACTIVE_EMPLOYMENT_YEARS_LOG
## 1               Laborers               2      0  53                    0.7942894
## 2               Managers               3      0  29                    2.2679752
## 3               Managers               2      1  51                    1.3857809
## 4               Managers               3      0  34                    0.8319266
## 5  Private service staff               3      0  37                    1.6585865
## 6             Accountants              4      0  34                    0.7238151
## 7             Secretaries              2      0  49                    1.3007752
## 8               Managers               2      1  30                    1.6385780
## 9               Managers               2      0  56                    2.4501432
## 10 High skill tech staff               1      1  48                    1.9923368
```

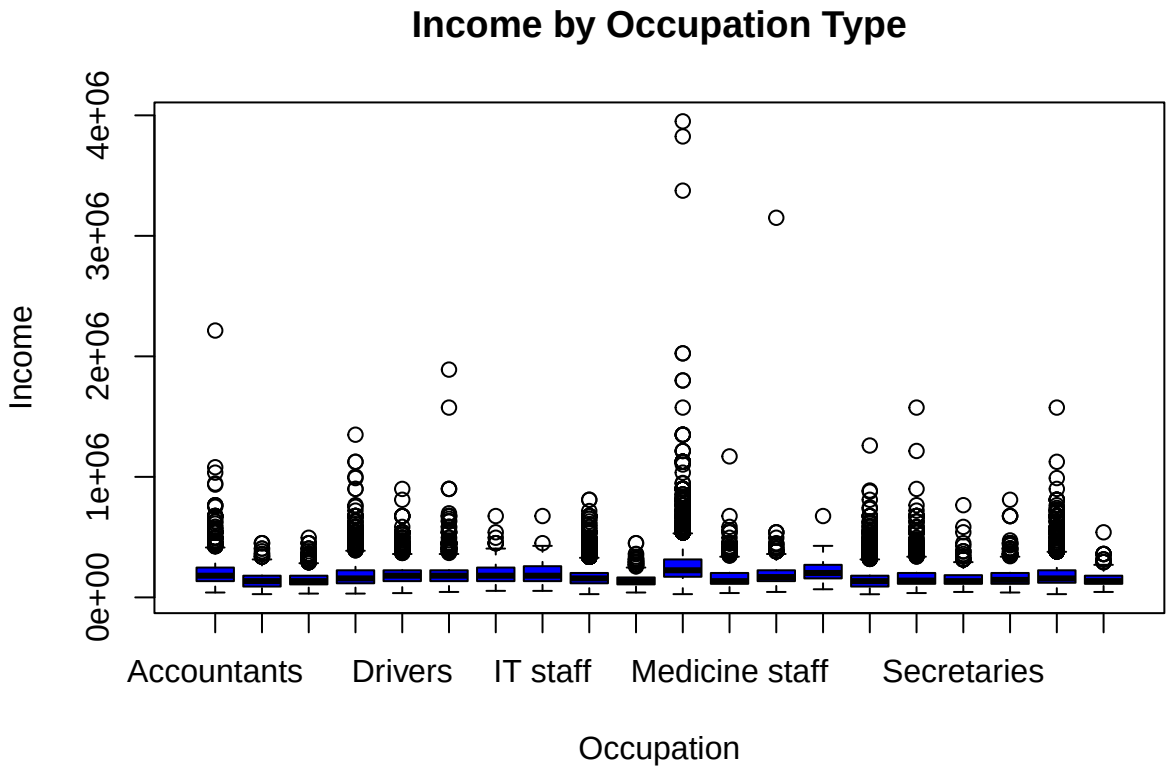
Examination of the top earners reveals critical inconsistencies. In a standard economic context, it is highly improbable for low-skill or service-oriented roles to command multi-million dollar annual salaries. Records where individuals in low-skill roles (e.g., Laborers, Cleaning staff) earn over $1,000,000 are treated as entry errors. Removing these ensures that the model does not learn false correlations between low-skill occupations and extreme wealth.

```r
impossible_jobs <- c("Laborers", "Cleaning staff", "Secretaries",
                     "Low-skill Laborers", "Drivers", "Waiters/barmen staff")

# Filter logic: Remove high earners in low-skill categories
data <- data %>%
  filter( !(AMT_INCOME_TOTAL > 1000000 & OCCUPATION_TYPE %in% impossible_jobs) )
# Visualizing the normalized distribution
hist(data$AMT_INCOME_TOTAL,
     main="Histogram of AMT_INCOME_TOTAL",
     xlab="AMT_INCOME_TOTAL", col="lightblue")
```

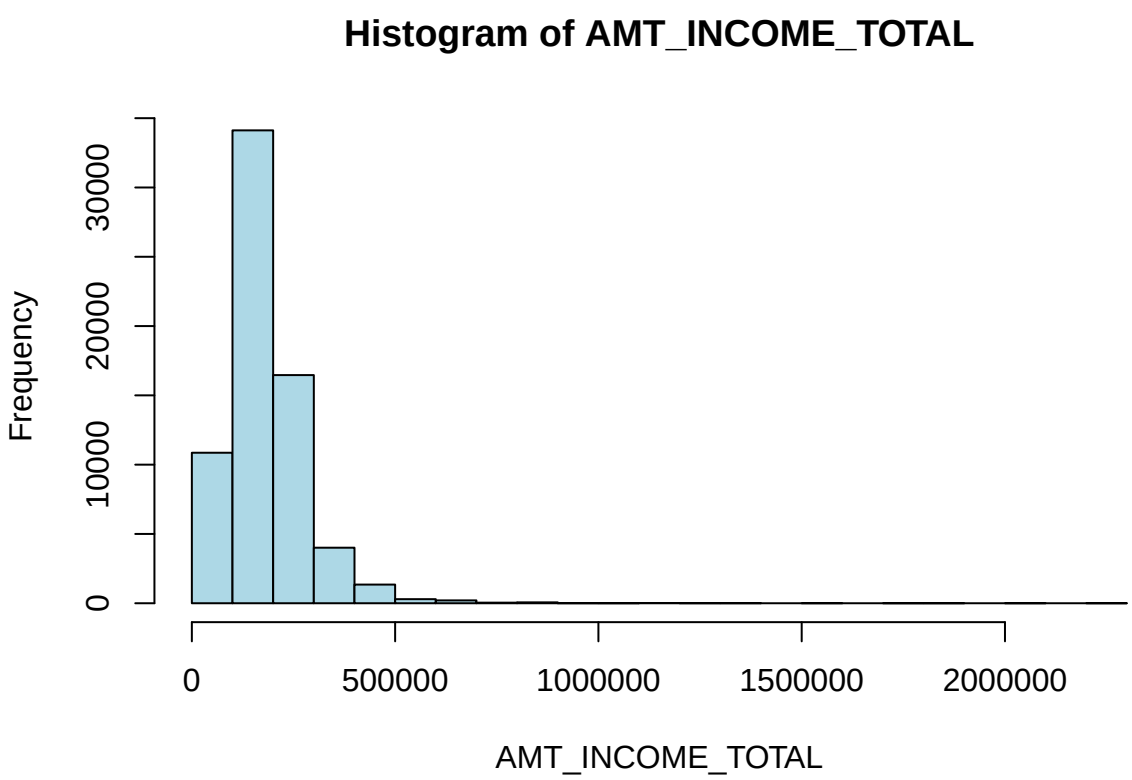


After removing the high earners in low-skill categories, we still see outliers for managers, which should be removed.

```
# Income by Occupation
boxplot(data$AMT_INCOME_TOTAL ~ data$OCCUPATION_TYPE,
        main="Income by Occupation Type", ylab = "Income", xlab = "Occupation", col="blue")
```




```
data <- data %>%
  filter(AMT_INCOME_TOTAL <= 3000000)

# Visualizing the normalized distribution
hist(data$AMT_INCOME_TOTAL,
     main="Histogram of AMT_INCOME_TOTAL",
     xlab="AMT_INCOME_TOTAL", col="lightblue")
```



We check the people who earn more than 1 million. Yet looking at their profile, we do not see any strange behavior in the data.

```
# Count rows with Income > 1,000,000
count_high_earners <- data %>%
    filter(AMT_INCOME_TOTAL > 1000000) %>%
    nrow()

print(paste("Number of customers earning > 1M:", count_high_earners))
```

```
## [1] "Number of customers earning > 1M: 42"
```

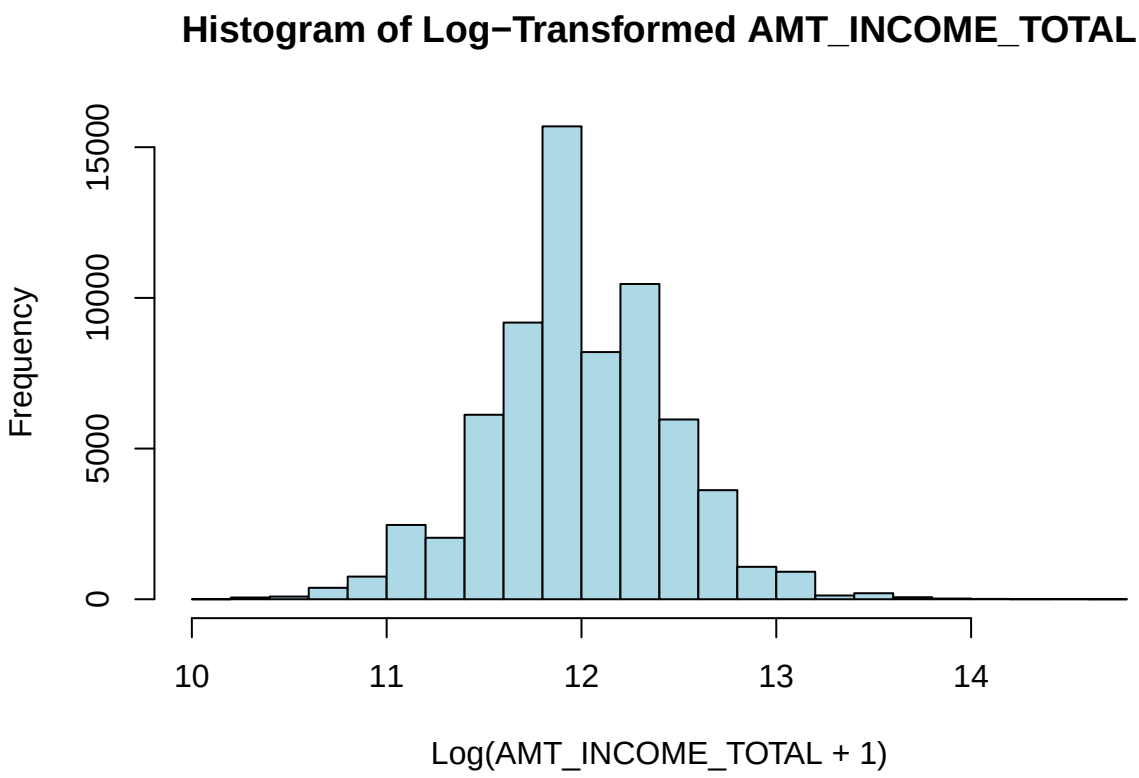
```
# view them to see if they look like valid data or errors
data %>%
    filter(AMT_INCOME_TOTAL > 1000000) %>%
    select(ID, AMT_INCOME_TOTAL, NAME_INCOME_TYPE, OCCUPATION_TYPE, AGE) %>%
    arrange(desc(AMT_INCOME_TOTAL))
```

```
##         ID AMT_INCOME_TOTAL     NAME_INCOME_TYPE       OCCUPATION_TYPE AGE
## 1  5993384          2214117 Commercial associate           Accountants  34
## 2  6651834          2025000              Working              Managers  30
## 3  5996392          2025000 Commercial associate              Managers  56
## 4  6423704          1890000              Working High skill tech staff  48
## 5  6695460          1800000 Commercial associate              Managers  34
## 6  6683694          1800000 Commercial associate              Managers  26
```

```
## 7  6832248          1575000              Working           Sales staff  30
## 8  6210090          1575000        State servant              Unknown  55
## 9  5884716          1575000 Commercial associate High skill tech staff  51
## 10 5143231          1575000 Commercial associate             Managers  28
## 11 5270822          1350000              Working             Managers  38
## 12 5889951          1350000 Commercial associate             Managers  37
## 13 6604643          1350000 Commercial associate            Core staff  47
## 14 5009694          1350000 Commercial associate             Managers  36
## 15 6430121          1350000 Commercial associate             Managers  39
## 16 5832856          1350000 Commercial associate             Managers  46
## 17 6087607          1260000            Pensioner              Retired  63
## 18 5730241          1215000 Commercial associate             Managers  53
## 19 6037082          1215000              Working             Managers  47
## 20 6074818          1215000 Commercial associate             Managers  29
## 21 6743545          1215000              Working           Sales staff  48
## 22 5981319          1170000        State servant        Medicine staff  36
## 23 6119705          1125000 Commercial associate             Managers  40
## 24 5839466          1125000 Commercial associate             Managers  29
## 25 5290155          1125000 Commercial associate             Managers  38
## 26 5709823          1125000 Commercial associate             Managers  43
## 27 5770751          1125000              Working             Managers  29
## 28 5436145          1125000              Working              Unknown  44
## 29 5343905          1125000              Working             Managers  40
## 30 5736055          1125000 Commercial associate             Managers  44
## 31 5112948          1125000              Working             Managers  45
## 32 6620577          1125000 Commercial associate            Core staff  27
## 33 5242490          1125000        State servant             Managers  53
## 34 5090777          1125000 Commercial associate             Managers  39
## 35 5025201          1125000 Commercial associate             Managers  44
## 36 5703492          1125000              Working            Core staff  34
## 37 6100356          1125000 Commercial associate             Managers  40
## 38 5354909          1102500 Commercial associate             Managers  43
## 39 5680396          1080000 Commercial associate           Accountants  42
## 40 5236562          1035000 Commercial associate           Accountants  55
## 41 6111383          1035000 Commercial associate             Managers  40
## 42 6415817          1001826 Commercial associate            Core staff  44
```

Even after removing logical outliers, the income variable remains highly skewed. To stabilize variance and minimize the impact of legitimate high-income earners, a Log Transformation (log1p) was applied.

```r
# Applying log transformation to achieve a more Gaussian distribution
data <- data %>%
  mutate(AMT_INCOME_TOTAL_LOG = log1p(AMT_INCOME_TOTAL))

# Visualizing the normalized distribution
hist(data$AMT_INCOME_TOTAL_LOG,
     main="Histogram of Log-Transformed AMT_INCOME_TOTAL",
     xlab="Log(AMT_INCOME_TOTAL + 1)", col="lightblue")
```



The resulting log-transformed distribution is much closer to a normal distribution, providing a more stable input for the neural network and preventing the gradient from being dominated by extreme values.

Note: In previous iterations all values above the 99% percentile were dropped without checking any domain logic.

3.7. Correlation

In this chapter we check for mutlicollinearity. Thfis is for different reasons than we would for a linear model. While Neural Networks are famously capable of handling non-linear relationships and redundant features, a correlation matrix is still av ital diagnostic tool during the data cleaning phase.

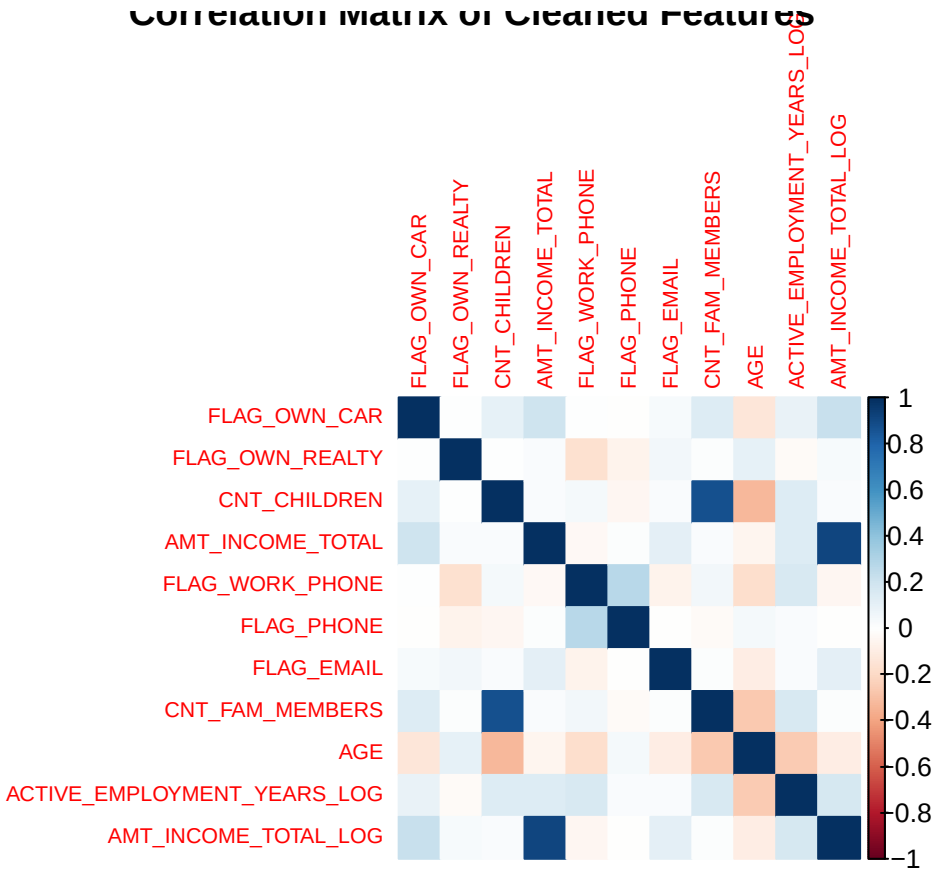
Even though NNs can technically learn to ignore redundant features, high multicollinearity can make the model's loss landscape 'flat' or unstable.

In terms if data-leakage this is only allowed to be done using the train data. Throughout this entire project we use seed 42 for obvious reasons. The data is splitted into train (70%), validation (15%) and test (15%).

```
set.seed(42) # ANSWER TO EVERYTHING
# Ensure target_col is defined
target_col <- "status"

# Create splits based on the target
y_factor <- factor(data[[target_col]])
test_idx <- caret::createDataPartition(y_factor, p = 0.15, list = FALSE)
test_data <- data[test_idx, ]
remain_data <- data[-test_idx, ]
```

```r
val_p <- 0.15 / 0.85
val_idx <- caret::createDataPartition(factor(remain_data[[target_col]]), p = val_p, list = FALSE)
val_data   <- remain_data[val_idx, ]
train_data <- remain_data[-val_idx, ]
```

```r
# Calculate correlation matrix for numerical columns
# Note: Use 'pearson' for linear and 'spearman' for non-linear rank correlation
cor_matrix <- cor(train_data %>% select_if(is.numeric), use="complete.obs", method="pearson")

# Visualize with a heatmap
corrplot(cor_matrix, method="color", tl.cex=0.7, title="Correlation Matrix of Cleaned Features")
```



In the provided Correlation Matrix, we see a heatmap representing the strength and direction of linear relationships between numerical variables. The graph uses a color scale where dark blue indicates a strong positive correlation, dark red indicates a strong negative correlation, and white indicates no linear correlation.

The most prominent dark blue square occurs at the intersection of CNT_CHILDREN and CNT_FAM_MEMBERS. This indicates that as athe number of children increases, the total family size increases almost in lockstep.

In traditional statistics, high intercorrelation is problematic because it suggests that one feature is a linear combination of another. Dropping a redundant feature reduces the dimensionality of the problem, making the model faster to train and easier to interpret.

While Neural Networks are more robust to multicollinearity than linear models, it still influences the training process. High correlation can make the "loss landscape" flat in certain directions, which can cause the optimizer to oscillate or converge more slowly toward the best solution. The network might assign very large

weights to one feature and very small weights to the other, even though they carry the same signal. This can make the model less stable when faced with new, unseen data.

Despite the high correlation between CNT_FAM_MEMBERS and CNT_CHILDREN seen in the matrix, we keep both for the following reasons:

- Latent Information:By keeping both, we allow the Neural Network to implicitly calculate the number of adults (Adults=Family Members-Children). This distinguishes between a single-parent household and a dual-parent household of the same total size.
- Interaction Effects:The network can learn complex, non-linear interactions between these two variables that a simple correlation coefficient cannot capture, such as how the financial burden of children might change depending on the total number of adults present.
- Data Integrity:Since we have already performed deterministic imputation to ensure these columns are logically consistent (Total Family>=Children + 1), providing both features gives the network the "full picture" of the household structure.

3.8. Summary of Features

The table below provides the final audit of transformed variables:

Original Variable	Cleaned/Engineered Variable	AGE
DAYS_BIRTH	AGE	Absolute value conversion to positive years.
DAYS_EMPLOYED	ACTIVER_EMPLOYMENT_YEARS_LOG	Retired values set to 0; log1p transformation applied
OCCUPATION_TYPE	OCCUPATION_TYPE	NA values imputed as 'Retired' for pensioners and 'Unknown' for workers.
CNT_FAM_MEMBERS	CNT_FAM_MEMBERS	Deterministic correction where FAM_MEMBERS < CHILDREN
AMT_INCOME_TOTAL	AMT_INCOME_TOTAL_LOG	Removal of 'impossible' job/income pairs and log1p scaling

Dataset Dimensions

```
dim(data)
```

```
## [1] 67452    19
```

3.9. Feature Engineering

The feature engineering phase was initiated during the third iteration of the data cleaning and modeling process. Despite significant efforts in fine-tuning the Neural Network architecture, including adjustments to layer depth, dropout rates, and activation functions, validation accuracy remained persistently low and stagnant.

Analysis of the misclassifications suggested that the raw features lacked sufficient "signal" to distinguish between target classes, particularly for minority classes. To overcome this, we transitioned from basic data cleaning to active feature engineering. The goal was to create interaction terms and domain-specific ratios that provide the model with better context regarding a borrower's financial stability and life stage.

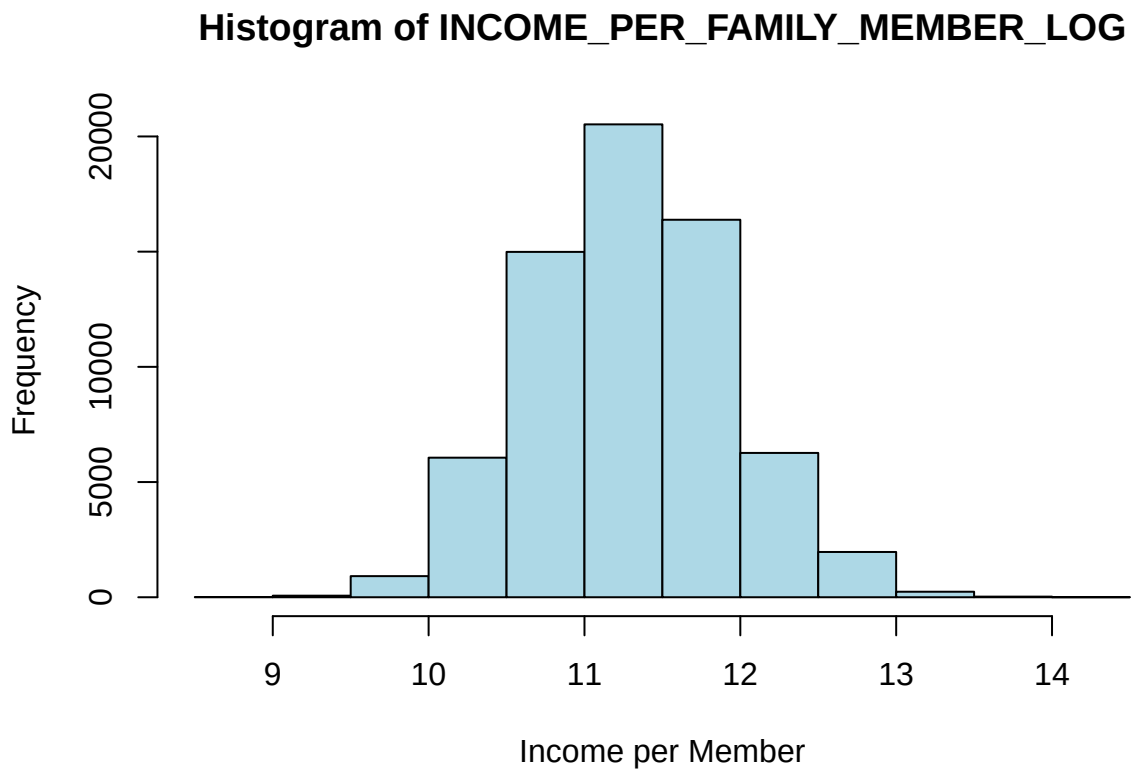
3.9.1. Financial context: family and life stage ratios

A borrower's raw income is less informative than their "disposable" income or earning power relative to their age and dependents. We created interaction terms to provide the model with this missing context.

Income per Family Member

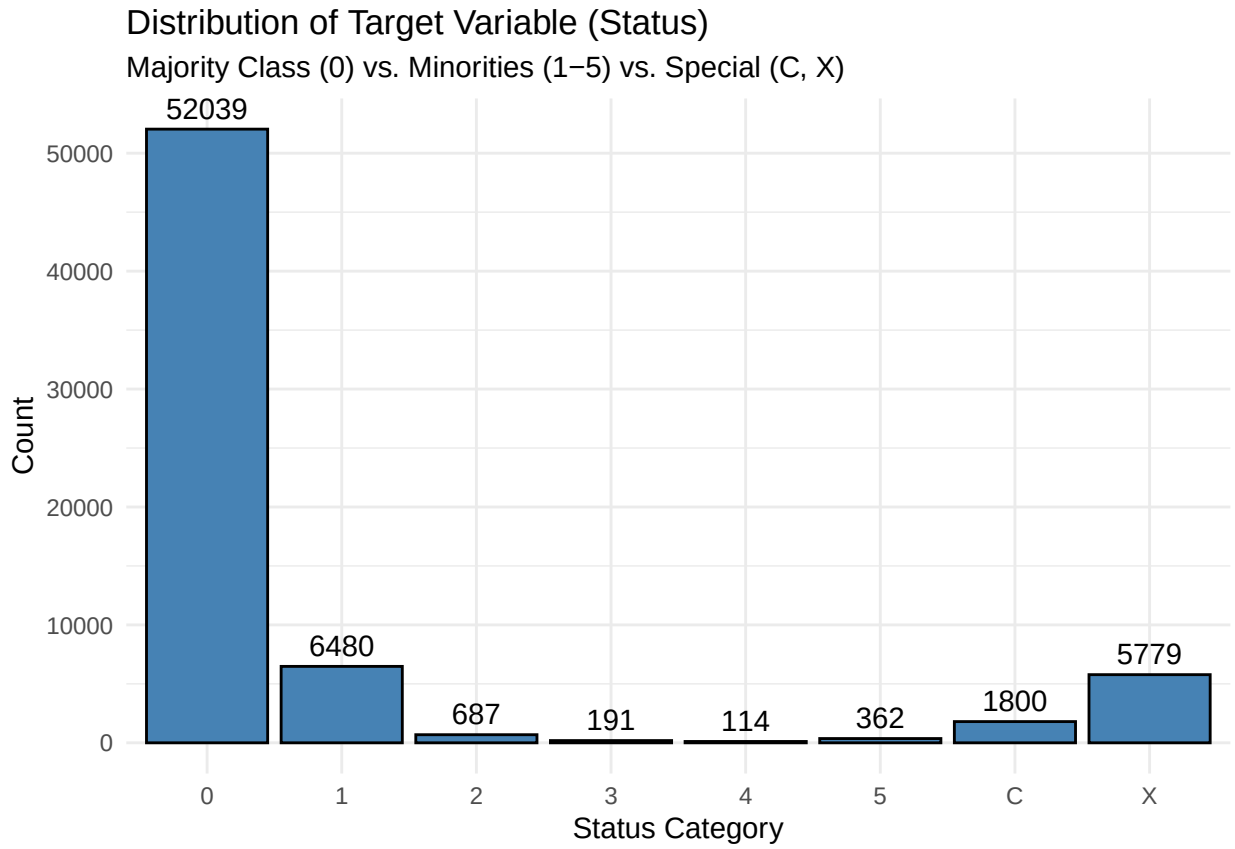
Total income does not account for the financial burden of dependents. We created INCOME_PER_FAMILY_MEMBER to capture the per-capita resource availability within the household. Given the high skewness of financial data, we applied a log transformation to normalize the distribution.

```
# Interaction term for family-adjusted income
data$INCOME_PER_FAMILY_MEMBER <- data$AMT_INCOME_TOTAL / data$CNT_FAM_MEMBERS

# Log transformation to stabilize variance
data <- data %>%
  mutate(INCOME_PER_FAMILY_MEMBER_LOG = log1p(INCOME_PER_FAMILY_MEMBER)) %>%
  select(-c(AMT_INCOME_TOTAL,INCOME_PER_FAMILY_MEMBER))


# Histogram of INCOME_PER_FAMILY_MEMBER
hist(data$INCOME_PER_FAMILY_MEMBER_LOG,
     main="Histogram of INCOME_PER_FAMILY_MEMBER_LOG",
     xlab="Income per Member",
     col="lightblue")
```



3.10. Target Distributions

```r
# Look at the distribution of the target variable histogram
ggplot(data, aes(x = status)) +
  geom_bar(fill = "steelblue", color = "black") +
  geom_text(
    stat = "count",
    aes(label = after_stat(count)),
    vjust = -0.5
  ) +
  labs(
    title = "Distribution of Target Variable (Status)",
    subtitle = "Majority Class (0) vs. Minorities (1-5) vs. Special (C, X)",
    x = "Status Category",
    y = "Count"
  ) +
  theme_minimal()
```



The target variable distribution reveals a severe class imbalance, with the majority class (0) overwhelmingly outnumbering minority delinquency categories (1–5) and special status codes (C, X). Such extreme skewness can cause a neural network to develop a strong majority-class bias, where it achieves high overall accuracy by simply predicting the most frequent class while failing entirely to identify high-risk borrowers. To mitigate this, common sampling techniques like oversampling minority instances via SMOTE or undersampling the majority class can be employed to balance the training signal. Alternatively, algorithmic adjustments such as applying class weights or utilizing Focal Loss can penalize errors in minority classes more heavily, forcing

the network to prioritize difficult, underrepresented examples. Beyond sampling, architectural strategies like delayed generalization or monitoring for grokking, where the model suddenly learns to generalize long after traditional convergence, may help the network move beyond simple rote memorization of the majority distribution toward meaningful pattern recognition across all risk tiers.

3.11. Preprocessing 2

This final phase of data preparation transitions the cleaned dataset into a machine-ready format. The primary objective is to transform categorical strings and boolean flags into numerical representations that the neural network can process, while removing redundant original variables to prevent multi-collinearity.

Target Recoding

```
# TARGET RECODING
# Mapping status levels to class IDs 0..7
data$status <- as.character(data$status)
data$target_class <- recode(data$status,
                            "X" = 0, "C" = 1, "0" = 2, "1" = 3,
                            "2" = 4, "3" = 5, "4" = 6, "5" = 7) |> as.numeric()

data <- data %>% select(-status)
```

Categorical Encoding: Ordinal vs. Nominal

We distinguish between two types of categorical data to preserve as much information as possible:

- Ordinal Encoding: Education level is treated as a ranked variable (EDUCATION_ENCODED), as there is a clear logical progression from secondary education to an academic degree.
- One-Hot Encoding: Nominal variables like CODE_GENDER, NAME_INCOME_TYPE, and our engineered OCCUPATION_GROUP have no inherent ranking. These are expanded into binary dummy columns to ensure the model does not assume a false mathematical relationship between categories.

```
# EDUCATION (ORDINAL ENCODING)
data$EDUCATION_ENCODED <- case_when(
    data$NAME_EDUCATION_TYPE == "Lower secondary" ~ 0,
    data$NAME_EDUCATION_TYPE == "Secondary / secondary special" ~ 1,
    data$NAME_EDUCATION_TYPE == "Incomplete higher" ~ 2,
    data$NAME_EDUCATION_TYPE == "Higher education" ~ 3,
    data$NAME_EDUCATION_TYPE == "Academic degree" ~ 4,
    TRUE ~ NA_real_
)
data <- data %>% select(-NAME_EDUCATION_TYPE)

# NOMINAL ENCODING (ONE-HOT)
nominal_cols <- c("CODE_GENDER", "NAME_INCOME_TYPE", "NAME_FAMILY_STATUS",
                  "NAME_HOUSING_TYPE", "OCCUPATION_TYPE")

data[nominal_cols] <- lapply(data[nominal_cols], as.factor)
```

```r
data <- dummy_cols(
  data,
  select_columns = nominal_cols,
  remove_selected_columns = TRUE,
  remove_first_dummy = FALSE
)
```

Drop ID-column

Since ID is just an identifier and not a real feature, it should be dropped.

```r
data <- data %>%
  select(-ID)

# Final Structure Check
str(data)
```

```
## 'data.frame':    67452 obs. of  51 variables:
##  $ FLAG_OWN_CAR                          : int  1 0 0 0 0 0 1 0 1 0 ...
##  $ FLAG_OWN_REALTY                       : int  1 1 1 0 1 1 1 1 1 1 ...
##  $ CNT_CHILDREN                          : int  2 0 0 1 0 0 1 0 0 0 ...
##  $ FLAG_WORK_PHONE                       : int  0 0 0 1 0 1 0 0 0 0 ...
##  $ FLAG_PHONE                            : int  0 0 0 1 0 1 1 0 0 1 ...
##  $ FLAG_EMAIL                            : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ CNT_FAM_MEMBERS                       : num  4 1 2 3 1 2 3 2 2 2 ...
##  $ AGE                                   : num  50 38 42 41 63 43 38 54 59 66 ...
##  $ ACTIVE_EMPLOYMENT_YEARS_LOG           : num  0 1.04 1.21 1.58 0 ...
##  $ AMT_INCOME_TOTAL_LOG                  : num  11.3 12.3 10.9 12 11.3 ...
##  $ INCOME_PER_FAMILY_MEMBER_LOG          : num  9.92 12.32 10.2 10.87 11.3 ...
##  $ target_class                          : num  2 2 2 4 2 2 2 2 2 2 ...
##  $ EDUCATION_ENCODED                     : num  1 1 1 1 3 1 1 1 1 1 ...
##  $ CODE_GENDER_F                         : int  0 1 1 1 1 1 1 1 0 1 ...
##  $ CODE_GENDER_M                         : int  1 0 0 0 0 0 0 0 1 0 ...
##  $ NAME_INCOME_TYPE_Commercial associate : int  0 1 0 0 0 0 0 0 0 0 ...
##  $ NAME_INCOME_TYPE_Pensioner            : int  1 0 0 0 1 0 0 1 0 1 ...
##  $ NAME_INCOME_TYPE_State servant        : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ NAME_INCOME_TYPE_Student              : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ NAME_INCOME_TYPE_Working              : int  0 0 1 1 0 1 1 0 1 0 ...
##  $ NAME_FAMILY_STATUS_Civil marriage     : int  0 0 0 0 0 1 0 0 0 0 ...
##  $ NAME_FAMILY_STATUS_Married            : int  1 0 1 1 0 0 1 1 1 1 ...
##  $ NAME_FAMILY_STATUS_Separated          : int  0 1 0 0 0 0 0 0 0 0 ...
##  $ NAME_FAMILY_STATUS_Single / not married: int  0 0 0 0 0 0 0 0 0 0 ...
##  $ NAME_FAMILY_STATUS_Widow              : int  0 0 0 0 1 0 0 0 0 0 ...
##  $ NAME_HOUSING_TYPE_Co-op apartment     : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ NAME_HOUSING_TYPE_House / apartment   : int  1 0 1 1 1 1 1 1 1 1 ...
##  $ NAME_HOUSING_TYPE_Municipal apartment : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ NAME_HOUSING_TYPE_Office apartment    : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ NAME_HOUSING_TYPE_Rented apartment    : int  0 1 0 0 0 0 0 0 0 0 ...
##  $ NAME_HOUSING_TYPE_With parents        : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Accountants           : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Cleaning staff        : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Cooking staff         : int  0 0 0 0 0 0 0 0 0 0 ...
```

```
##  $ OCCUPATION_TYPE_Core staff             : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Drivers                : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_High skill tech staff  : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_HR staff               : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_IT staff               : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Laborers               : int  0 0 1 0 0 0 0 0 1 0 ...
##  $ OCCUPATION_TYPE_Low-skill Laborers     : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Managers               : int  0 0 0 1 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Medicine staff         : int  0 0 0 0 0 1 1 0 0 0 ...
##  $ OCCUPATION_TYPE_Private service staff  : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Realty agents          : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Retired                : int  1 0 0 0 1 0 0 1 0 1 ...
##  $ OCCUPATION_TYPE_Sales staff            : int  0 1 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Secretaries            : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Security staff         : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Unknown                : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ OCCUPATION_TYPE_Waiters/barmen staff   : int  0 0 0 0 0 0 0 0 0 0 ...
```

Note on Feature Scaling

Throughout the iterative development of the neural network, the strategy for feature scaling evolved to better align with the Rectified Linear Unit (ReLU) activation function and the specific distribution of our engineered features.

Initial Iterations: Robust Scaler In the first two iterations, Robust Scaler was employed as the primary normalization tool.

- Mechanism: Unlike standard scaling which uses the mean and variance, the Robust Scaler removes the median and scales the data according to the Interquartile Range (IQR).
- ustification: This method is specifically designed to handle datasets with significant outliers, such as the initial versions of our income and employment data. It ensures that extreme values do not exert undue influence on the feature's scale, preserving the relative relationships within the majority of the data.
- Compatibility with ReLU: Robust Scaler typically produces a centered distribution with both positive and negative values. This works with ReLU because the network can learn to utilize the "active" positive region for significant signals while the negative region is zeroed out, effectively acting as a natural threshold for the neurons.

Transition to Min-Max Scaling In the final iteration, the scaling strategy was shifted to Min-Max Scaling to optimize performance with the ReLU activation function.

- Mechanism: Min-Max Scaling transforms all features into a fixed range, typically [0, 1], by subtracting the minimum value and dividing by the range.
- Superiority with ReLU:
  - Avoiding Dead Neurons: ReLU outputs zero for any negative input ($f(x) = \max(0, x)$). By shifting the entire feature set to a positive [0, 1] range, we ensure that every feature starts within the active gradient region of the ReLU function. This reduces the risk of "dying ReLU" problems during the initial phases of training.
  - Consistent Input Space: Standardizing all inputs to the same positive bounds allows for more stable weight initialization and prevents certain features from dominating the gradient updates simply due to their numerical magnitude.

- Sparsity Preservation: Min-Max scaling is particularly effective at preserving the zero-values in sparse data, which is essential given our use of one-hot encoding for the OCCUPATION_GROUP.

Leakage Prevention and Implementation While Min-Max Scaling is a required step for neural network convergence, it is intentionally excluded from this preprocessing script.

Data Leakage Prevention: To prevent data leakage, the scaling parameters (minimum and maximum values) must be calculated solely on the training split. Implementation Flow: If parameters were calculated using the entire dataset, information from the test set would "leak" into the training process. Consequently, scaling is implemented within the neural network training script immediately following the data split.

4. Neural Network

4.1. Neural Network Type: The Multi-Layer Perceptron (MLP)

For this classification task, we selected a Multi-Layer Perceptron (MLP), the canonical form of a feed-forward neural network. While modern deep learning often focuses on specialized architectures like Convolutional Neural Networks (CNNs) for image data or Recurrent Neural Networks (RNNs) for sequential data, the MLP remains the optimal choice for tabular datasets for several key reasons:

1. Data Independence: Our dataset consists of independent customer records. Unlike time-series data (e.g., stock prices), there is no temporal dependency between Row i and Row i+1. Unlike image data, there is no spatial correlation between Column A and Column B. CNNs and RNNs rely on these dependencies to function; applying them here would introduce unnecessary computational overhead without theoretical benefit.
2. Universal Approximation: The Universal Approximation Theorem states that a feed-forward network with a single hidden layer containing a finite number of neurons can approximate any continuous function. Given the complex, non-linear relationships between financial variables (e.g., the interaction between Age, Income, and Family Size), an MLP provides the necessary capacity to map these inputs to the 8 target classes.
3. Feature Interaction: Unlike linear models (e.g., Logistic Regression), which assume additive relationships, an MLP learns to combine features. A neuron in the second hidden layer might activate only when Income is Low AND Family Size is High, effectively automating the feature engineering process.

4.2. Defining the Architecture

Designing the network topology is a balancing act between Capacity (the ability to learn complex patterns) and Generalization (the ability to perform well on unseen data). Our final architecture was derived through a systematic Grid Search, but guided by the following principles:

4.2.1. Input Layer

The input layer size corresponds to the number of features after preprocessing. Since we employed One-Hot Encoding for categorical variables (like Occupation and Gender), our input vector is sparse and high-dimensional.
We tried grouping the column OCCUPATION_TYPE. The goal was to ensure that, when looking at the training set, occupations with the same risk-category could be found in all target classes. Yet grouping efforts actually lost important information for the model, leading to lower class-specific accuracy.

```r
dim(data) # We have 50 features and 1 target
```

```
## [1] 67452    51
```

The size of the input layer of the neural network is the amount of columns, in this case 50.

4.2.2. Hidden Layers (Depth and Width)

We experimented with depths ranging from 1 to 3 hidden layers.

The Risk of Depth: The deeper a network gets, the more complex interactions become. Yet the signal is diluted in every layer. Similarly, the signal from the loss function dilutes as it propagates back through many layers. Furthermore, deep networks on small datasets (like ours) are prone to overfitting, memorizig the noise in the training set rather than the signal.

The Width Strategy (Cubic vs. Funnel): We tested two shape configurations. Funnel (Pyramid): Layers decrease in size (e.g., 512 to 256 to 128). This assumes the network should compress information into higher-level abstractions. Cubic (Rectangular): Layers maintain constant width (e.g., 256 to 256 to 256).

```r
# Define the Search Grid
runs_grid <- list(
  # --- 1. ARCHITECTURE ---
  # Search Size: Wide (1024) vs. Narrow (256)
  units1 = c(128, 256, 512, 1024),

  # Search Funneling: Tapered (128) vs. Cylindrical (256)
  units2 = c(64, 128, 256, 512),

  # Search Depth: 2 Layers (0) vs. 3 Layers (64)
  units3 = c(0, 64, 128, 256),

  # --- 2. OPTIMIZATION ---
  # Search Learning Speed: Standard (1e-3) vs. Precise (3e-4)
  learning_rate = c(0.01),

  # Search Gradient Noise: Noisy (128) vs. Stable (256)
  batch_size = c(64),

  # --- 3. REGULARIZATION  ---
  dropout1 = c(0.0),
  dropout2 = c(0.0),
  dropout3 = c(0.0),

  # Search momentum
  momentum = c(0.9),
  # --- CONSTANTS ---
  patience = c(100),
  epochs = c(1000)
)
```

The goal of this grid search is to only see how high validation accuracy can get depending solely on the network architecture.

```
tic()
tuning_run("Assignment_2_MLP_minmax.r",
           runs_dir = "tuning_architecture_sgd",
           flags = runs_grid,
           sample = 1.0, # Run ALL combinations (set to e.g. 0.5 to run random 50%)
           echo = TRUE)

beep(sound=5)
toc()
```

From all the models, we will search for the model with the lowest validation loss since this is the most stable model. A model with higher accuracy and higher loss score is more unstable. While it gets more class labels correct, it is very unconfident or very wrong on the outliers. A high loss usually signals that the model has overfit the cross-entropy objective and is essentially "memorizing" the decision boundary. Therefore it leaves no room to improve with hyperparameter tuning later because the gradients are already exploding.

```
# Load all runs from the directory
all_results <- ls_runs(runs_dir = "tuning_architecture_sgd")

# View them in the RStudio data viewer (Sortable table)
#View(all_results)

# 3. Print just the top 5 models sorted by Validation Accuracy
top_models <- all_results %>%
  arrange((metric_val_loss)) %>%
  select(metric_val_accuracy, metric_val_loss, flag_units1, flag_units2, flag_units3) %>%
  head(5)

print(top_models)
```

```
## Data frame: 5 x 5
##   metric_val_accuracy metric_val_loss flag_units1 flag_units2 flag_units3
## 1              0.8439          0.5572         128          64           0
## 2              0.8185          0.5676          64          64           0
## 3              0.8408          0.5776         128          64           0
## 4              0.8444          0.5915         128          64          64
## 5              0.8504          0.6147         128         128           0
```

We see that a simpler model is the more stable model. The danger of going too small though is underfitting (low accuracy and low loss). A microgrid with one-layer experiments was made to see if even simpler models still can find the patterns in the data. Then we would base the decision on Razor's principle 'The simplest model is the best'.

Based on multiple experiments, always in combination with different scalers and optimisers due to the iterative process. The best architectures found was:

- Number of hidden layers: 2 layers
- Number of units per layer: 128/64

Activation Functions

We utilized the Rectified Linear Unit for our hidden layers. ReLU is the standard for deep learning because it introduces non-linearity without suffering from the vanishing gradient problem common in Sigmoid or Tanh

functions. However, we discovered a critical dependency between our choice of scaler and this activation function. Initial Failure (Robust Scaler): In our early iterations, we used a Robust Scaler, which centers data and often produces negative values for inputs below the median. Since ReLU defines the output of any negative input as exactly 0, a significant portion of our input signal was immediately zeroed out (deactivated) upon entering the first hidden layer. This led to dead neurons and significant information loss. Note: These random searches using this scaling method can be found in the git-commit history as well as in the tfruns folder 'tuning_architecture_search'. Due to the very bad results and high overfitting of the models, these results will not be displayed in this documentation. The Fix (Min-Max Scaler): To resolve this, we switched to Min-Max Scaling, which forces all input features into the range [0, 1]. By ensuring all input values are positive, we guarantee that they fall into the active linear region of the ReLU function ($x > 0$) rather than the dead region. This alignment between our preprocessing and our activation function resulted in an immediate and significant increase in model accuracy.

Output Layer (Softmax): Since this is a multi-class classification problem with 8 mutually exclusive classes, the final layer uses the Softmax function. This converts the raw logits into a probability distribution summing to 1.0, allowing us to interpret the output as confidence percentages.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Here we demonstrate the difference of both scalers with the relu activation function. In the following experiment a robustscaler was used with the same architecture as above.

```
options(tfruns.runs_dir = "tuning_scaler")
tic()
training_run("Assignment_2_MLP_robustscaler.r")

beep(sound=5)
toc()
# Reset options
options(tfruns.runs_dir = NULL)
```

```
run_path <- "tuning_architecture_minmax/2025-12-22T10-12-20Zs"

#view_run(run_path)
```

4.3. Class Imbalance Handling

Our target variable is severely imbalanced (approx. 99% Good vs 1% Bad/Late). A standard neural network minimizes global error by predicting the majority class for every instance, achieving 99% accuracy but 0% utility.

```
tic()
# Set the "Bucket"
options(tfruns.runs_dir = "tuning_imbalance")

#  Run SMOTE Script
training_run("Assignment_2_MLP_SMOTE.r")

# Run Undersampling Script
training_run("Assignment_2_MLP_undersampling.r")
```

```r
# Run Weighted Class Scripts
training_run("Assignment_2_MLP_weighted.r")
training_run("Assignment_2_MLP_man_weight.r")

# Run Focal Loss Script
training_run("Assignment_2_MLP_focal_loss.r")


# 4. Finish
beep(sound=5)
toc()

# 5. Reset options (good practice)
options(tfruns.runs_dir = NULL)
```

Note: These tests were made with a different amount of layers and depth, compared to the optimal chosen architecture. Still, these tests can be compared among eachother to show whether or not the new method is superior to the raw-data.

4.3.1. SMOTE

Random oversampling, which involves simply duplicating rows, frequently leads to overfitting because the model memorizes specific examples rather than learning general patterns. To address this, we employed SMOTE. Instead of duplication, SMOTE creates new synthetic examples by interpolating between existing minority samples in the feature space. This forces the model to learn a more generalized decision boundary. We noted that ADASYN (Adaptive Synthetic Sampling) could represent a potential enhancement. ADASYN functions similarly to SMOTE but prioritizes generating synthetic data for examples located near the decision boundary, which are harder to classify. However, this variation was not implemented in this project.

Analysis of Results

The model achieved an overall accuracy of 85.9%, significantly outperforming the No Information Rate (NIR) of 77.1%. The NIR is the baseline accuracy in machine learning classification. It represents how well a model performs if it always guesses the most frequent class (majority class 2) in the dataset, without learning anything. The substantial gap between Accuracy and NIR, along with a Kappa score of 0.61 (indicating "substantial agreement"), confirms that the SMOTE-enhanced model has successfully learned distinctive patterns beyond just memorizing the majority class distribution.

The goal of using SMOTE was to improve the recognition of minority classes (Classes 4, 5, 6 and 7), which are often ignored during standard training.

The most telling metric here is Balanced Accuracy, which averages recall across classes. While the standard sensitivity (recall) for minority classes ranges from roughly 41% to 53%, the balanced accuracy remains consistently high, between 70% and 76% for these rare groups. this indicates that SMOTE successfully pushed the decision boundary to "pay attention" to these classes, preventing them from dropping to near-zero recognition.

For the rarest groups, the model exhibits a conservative bias. For example, in class 5, the model only identified 41% of the actual cases (sensitivity), but when it did predict class 5, it was correct 80% of the time (Precision). This suggests that SMOTE helped create high-confidence regions for these classes, even if those regions don't cover the entire spread of the minority data.

The confusion matrix reveals where the model still struggles despite oversampling:

- Class 2 remains the dominant anchor of the model. However, significant "leakage" occurs between Class 2 and it's neighbors.

- Classes 4 and 6 show lower Precision compared to Class 5. Specifically, Class 6 suffers from a very small sample size (only ~17 true instances total), yet the model successfully captured 9 of them.

In conclusion, the application of SMOTE resulted in a Macro F1 score of 0.616.

4.3.2. Data Augmentation with Cherry picking

```
# Set the "Bucket"
options(tfruns.runs_dir = "tuning_imbalance")

# 1. Clean the Data
print(">>> Step 1/3: Running Data Cleaning...")
source("Kaggle_dataset_clean.r")
print(">>> Data Cleaning Complete.")

# 2. Run Exploratory Data Analysis
print(">>> Step 2/3: Running EDA...")
source("Kaggle_EDA.r")
print(">>> EDA Complete.")

# 3. Train Neural Network (Cherry Picking Strategy)
print(">>> Step 3/3: Running Neural Network (Cherry Picking)...")
source("NN_Cherrypicking_Kaggle.r")
print(">>> Pipeline Finished Successfully.")

# Reset options (good practice)
options(tfruns.runs_dir = NULL)
```

To address the severe class imbalance without relying solely on synthetic data (like SMOTE), we implemented a strategy of Selective Data Augmentation. We leveraged the original Kaggle source dataset to find "organic" examples of minority classes that were excluded from the assignment subset.

First, we reconstructed the target labels for the full Kaggle dataset by joining application_record.csv with credit_record.csv and identifying the worst delinquency status for each unique ID.

A critical challenge was ensuring no data leakage occurred between this new source and our validation sets. Since the assignment IDs were scrambled, we performed a rigorous content-based exclusion. We compared the two datasets based on their full feature vectors (dropping the ID column) and filtered the Kaggle dataset to retain only those rows that did not exist in the assignment data. This effectively created a "Rest of World" dataset containing unique, unseen customers.

From this pool of unique rows, we "cherry-picked" only the samples belonging to underrepresented classes (Status 1, 2, 3, 4, 5, X, and C), deliberately excluding the majority class (Status 0). This allowed us to inject real-world patterns of delinquency into the training set without further inflating the majority baseline.

This approach significantly enriched the training data with genuine minority examples. However, it introduced a distribution shift: by artificially boosting the prevalence of defaulters in the training set, we altered the prior probabilities compared to the validation set (which retained the natural, imbalanced distribution). Consequently, while the model's Sensitivity (ability to detect risk) improved, the overall Accuracy decreased, as the model became more aggressive in predicting defaults than the natural base rate would suggest.

Analysis of Results

Contrary to the hypothesis that adding real-world minority examples would improve detection rates, the Selective Data Augmentation strategy resulted in a degradation of model performance across nearly all critical metrics.

- The overall validation accuracy dropped from 86.04% (in the baseline Raw Model) to 84.77%. While a slight drop in accuracy is often an acceptable trade-off for higher sensitivity in imbalanced datasets, we did not observe the corresponding gain in minority detection.
- The most significant failure occurred in the sensitivity (recall) of the high-risk classes, which the strategy was specifically designed to boost.
  - Class 3 (Status 1): Sensitivity dropped precipitously from 61.0% to 49.1%.
  - Class 4 (Status 2): Sensitivity declined from 51.5% to 44.7%.
  - Class 0 (Status X): Sensitivity decreased from 55.8% to 53.6%.
- The confusion matrix reveals that the model became less discriminative and more prone to classifying actual defaulters as "Good" clients (Class 2/Status 0). Specifically, the number of Class 3 (Status 1) defaulters misclassified as Class 2 rose from 335 in the raw model to 448 in this augmented model.

These results strongly suggest the presence of a covariate shift between the University assignment dataset and the raw Kaggle source. Although the external rows represented the same target labels (e.g., "Status 1"), their underlying feature distributions likely differed from the "slightly altered" University data. Consequently, the augmented data acted as noise rather than signal, blurring the decision boundaries and causing the model to revert to predicting the majority class more frequently. Due to this negative impact, this external augmentation strategy was discarded in favor of the baseline model.

4.3.3. Undersampling majority classes

To counter the dominance of the majority class, we implemented Random Undersampling. Unlike SMOTE, which adds synthetic data, undersampling balances the dataset by randomly removing examples from the majority class in the training set until its size matches that of the minority classes, or untill a threshold is reached. The theoretical advantage of this approach is that it forces the neural network to treat every class as, more or less, equally probable, preventing the optimization algorithm from getting stuck in a local minimum where it simply predicts "class 2" for everything to minimize loss.

However, this technique introduces a significant risk known as distribution shift. By artificially equalizing the class counts, we fundamentally alter the "prior probability" the model sees during training. In our training set, a rare class (like Class 5) might appear 12.5% of the time, but in the real-world test set, it only appears 0.2% of the time. Consequently, the model learns to expect these minority classes far more often than they actually occur. This leads to a high rate of False Positives: the model becomes "trigger-happy," guessing minority classes too frequently because it assumes they are common. Furthermore, by discarding a vast amount of majority class data, we risk losing valuable information about the variance and sub-clusters within the majority group.

Analysis of Results

The results of the undersampling experiment strictly confirm the theoretical risks described above. The most immediate observation is the drop in Overall Accuracy to 72.3%, which is notably lower than the No Information Rate (NIR) of 77.1%. Statistically, this model performs worse on a raw accuracy basis than a "dumb" model that simply predicts Class 2 for every instance.

However, accuracy is misleading here. The goal was to improve minority detection, and by that metric, the model succeeded, albeit at a heavy cost:

- The distribution shift caused the model to drastically over-predict minority classes.
  - For Class 0, the model predicted it 1'618 times, but only 644 were correct. A massive 846 instances of Class 2 were incorrectly classified as Class 0. This results in a very low Precision of 39.8%.

  - Similarly, for Class 4, the Precision is only 33.8%, meaning two-thirds of the time the model predicted Class 4, it was wrong.

- Conversely, the model is excellent at finding the minority classes when they do exist.
  - Balanced Accuracy is exceptionally high across the board, ranging from 74% to 82%.
  - Class 0 Sensitivity rose to 74.3%, and even the extremely rare Class 6 achieved a Sensitivity of 64.7%.

- The trade-off is clear in the Class 2 column. The model only correctly identified 73.2%of the Class 2 instances (Sensitivity), whereas standard models usually achieve over 95%.

In summary, undersampling successfully "woke up" the model to the existence of rare classes, achieving high balanced accuracy. However, it did so by aggressively hallucinating minority labels on majority class data, resulting in a model that is sensitive but not precise.

4.3.4. Algorithm Level: Class Weights

To address the imbalance problem without altering the training data itself (as in SMOTE or Undersampling), we applied Cost-Sensitive Learning. This technique modifies the loss function during training, assigning a higher penalty (cost) to misclassifications of minority classes. By telling the model that "missing a rare class is 50 times worse than missing a common class," we force the optimization algorithm to pay attention to the underrepresented groups.

We explored two strategies for assigning these weights:

1. Computed Weights (Inverse Frequency): We first calculated weights mathematically to perfectly counterbalance the class frequencies. The weight wj for class j was calculated as

$$w_j = \frac{N_{samples}}{N_{classes} \times N_j}$$

   This resulted in extreme penalties for the rarest classes (e.g., Class 6 received a weight of ca. 74.7, while the majority Class 2 received ca. 0.16).

2. Manual Tuning: We hypothesized that the computed weights might be too aggressive, leading to overfitting or excessive false positives. Therefore, we performed a second experiment with Manually Tuned Weights, capping the maximum penalty at 5.0 and slightly down-weighting the majority class (0.5) to find a more stable "business-logic" balance.

Analysis of Results

The comparison between the two weighting strategies highlights a clear trade-off between detection sensitivity (catching the minority) and precision (avoiding false alarms).

Computed Weights: The "Aggressive" Approach The mathematically computed weights (ranging up to 74.7) succeeded in forcing the model to recognize minority classes but did so indiscriminately.

- High Sensitivity, Low Precision: The model became "trigger-happy." For the rarest group, Class 6, the Sensitivity was 64.7%, which is excellent. However, the Precision was only 17.7%. This means that for every 100 times the model predicted Class 6, it was wrong 82 times.
- Impact on Accuracy: Because the model was aggressively predicting rare classes where they didn't exist, the Overall Accuracy (78.5%) barely exceeded the No Information Rate (77.1%).
- High Balanced Accuracy: The Balanced Accuracy of ~81% was the highest among methods, proving the model was strictly prioritizing average recall over total accuracy.

Manual Tuning: The "Balanced" Approach By dampening the weights (setting Class 6/7 to 5.0 and Class 2 to 0.5), we achieved a far more robust model.

- Restored Accuracy: The Overall Accuracy jumped to 84.9%, a significant improvement over the computed weights. This indicates the model stopped "hallucinating" minority labels on clear majority cases.
- Precision Recovery: The precision for minority classes improved drastically. For Class 6, Precision rose from 17.7% to 42.9%, and for Class 5, it rose from 25.4% to 68.2%.
- The Trade-off: We accepted a slight drop in Balanced Accuracy (from ~81% down to ~78%). The Sensitivity for Class 6 dropped from 64.7% to 52.9%, meaning we miss a few more actual cases. However, the cases we do flag are much more likely to be correct.

While the computed weights provided the highest theoretical recall for minority groups, the manual weights offered a superior practical solution. The manual approach (Accuracy 84.9%, Macro F1 0.61) successfully identifies underrepresented classes without overwhelming the user with false positives, making it the more viable candidate for deployment.

4.3.5. Loss Function: Focal Loss

In our final experiments, we replaced standard Categorical Cross-Entropy with Focal Loss.

Standard Cross-Entropy loss struggles with extreme imbalance because it is overwhelmed by the sheer volume of "easy" examples. In our dataset, Class 2 (Non-Default) comprises 77% of the data. Even if the model assigns a decent probability (e.g., 0.6) to these easy examples, the cumulative loss from thousands of them drowns out the valuable signal from the few misclassified minority examples.

Focal Loss adds a modulating factor, (1-pt) , to the standard loss equation.

- Effect: If the model is 99% confident in a prediction (an "easy" example), the modulating factor approaches 0. This effectively silences the loss contribution from the majority class.
- Result: The network is forced to focus its weight updates almost entirely on the "hard" examples—specifically, the minority class errors where the model is currently failing.

Analysis of Results

Focal Loss emerged as the most "surgical" of all the methods tested. While Undersampling used a sledgehammer (deleting data) and Class Weights used a megaphone (amplifying penalties), Focal Loss acted as a scalpel, ignoring what was already learned to focus on what was difficult.

1. Best-in-Class Precision The most striking result is the Macro F1 Score of 0.631, the highest among all tested methods (compared to ~0.61 for SMOTE and Manual Weights). This improvement is driven by superior Precision.

- Unlike Undersampling, which hallucinated minority classes everywhere, Focal Loss remained conservative and accurate.
- Class 7: Precision of 83.3% (it rarely guessed Class 7, but when it did, it was right).
- Class 5: Precision of 75.0%.
- Class 0: Precision of 70.1%. This indicates that Focal Loss successfully learned the distinctive features of these minority classes rather than just guessing them based on inflated priors.

2. Preserving the Majority: Crucially, Focal Loss did not sacrifice the majority to save the minority.

   - Class 2 Sensitivity remained extremely high at 94.4%.
   - Overall Accuracy was 85.9%, effectively tying with SMOTE and significantly beating Undersampling (72%) and Computed Weights (78%).

3. The Limit of "Hard" Examples (Class 6) However, the method hit a wall with Class 6.

   - Sensitivity: Only 35.3% (detecting 6 out of 17 instances).
   - Balanced Accuracy: Dropped to 67.6% for this class. Because Class 6 is so incredibly rare (only ~0.1% of data), it is possible these examples were treated as "outliers" rather than "hard examples," or simply that the model requires more representative features to distinguish them.

Focal Loss provided the best balance of the imbalance rectifying attempts. It achieved the highest F1 score, maintained high overall accuracy, and delivered reliable predictions (high precision) for the minority classes. It proves that intelligently re-weighting the loss gradients is often superior to artificially manipulating the training data.

Conclusion and Recommendation for Class Imbalance We systematically evaluated four techniques to address the class imbalance against a baseline model trained on raw data. Our hypothesis was that the baseline would fail to detect the minority classes (late payers). However, the empirical results refuted this.

Model	**Overall Accuracy**	**Macro F1 Score**	**Class 6 Sensitivity (Recall)**	**Class 6 Precision**
Baseline (Raw Data)	86.04%	0.648	58.8%	62.5%
Focal Loss	85.88%	0.631	35.3%	50.0%
SMOTE	85.86%	0.616	52.9%	40.9%
Manual Class Weights	84.94%	0.612	52.9%	42.9%
Cherry Picking	84.77%	0.606	52.9%	64.3%
Undersampling	72.34%	0.518	64.7%	0.1% (Approx)

The table below summarizes the key performance metrics across all experiments:

- **Undersampling** achieved the highest "detection rate" for rare classes but failed catastrophically on precision, generating so many false alarms that the model became practically unusable.
- **Focal Loss** was excellent at increasing confidence (Precision) but surprisingly conservative, missing too many of the "hard" examples (lowest Recall for Class 6).
- **The Baseline Model** struck the best balance. It not only achieved the highest Overall Accuracy and F1 Score but, crucially, it outperformed SMOTE and Focal Loss in detecting the rarest instances (Class 6 Recall of 58.8%).

Business Implications: The Cost of a Miss vs The cost of a False Alarm

For this specific business case, identifying payment behavior, the two critical errors have different costs:

1. **False Negative (Missed Late Payer):** The bank fails to intervene, leading to a financial loss (Default). This is the most expensive error.

2. **False Positive (False Alarm):** The bank flags a good customer as risky. This incurs a small administrative cost (checking the file) or a customer service risk (annoyance).

While we want to maximize the detection of late payers (high Recall), we cannot afford a model like Undersampling, where ~80-90% of the flagged customers are actually innocent. This would overwhelm the collections department and damage customer relationships.

The Baseline Model offers the most viable path for deployment. It catches the majority of late payers (high sensitivity) without drowning the operations team in false positives.

Final Decision for Hyperparameter Tuning

Based on these findings, we will proceed with the Baseline (Raw Data) approach for the final hyperparameter tuning phase.

The data suggests that the "imbalance" is not the primary blocker; the minority classes have distinctive enough features that a standard neural network can separate them if tuned correctly. Adding complexity via SMOTE or Focal Loss provided diminishing returns. Therefore, our final optimization efforts will focus on maximizing the architecture (layers, neurons, learning rate) of the standard model to squeeze out the remaining performance.

Note: The different methods to resolve the imbalance were only tested with one architecture due to time constraints (since all models were trained on one machine). It is possible that with different architectures, the results could have been different. We acknowledge that a specific grid-search for each class-imbalance handling would have been the correct solution.

4.4 Choosing the Optimizer: The Shift from Adam to SGD

A significant portion of the initial model exploration involved unstructured random searches. While these experiments provided early insights, they are not detailed in this documentation. The raw logs and scripts for these exploratory runs, including our initial attempts at "delayed regularization" and long-duration training, can be found in the tuning_architecture_search directory. For the sake of clarity and reproducibility, this chapter documents only the final, systematic grid searches.

Initially, we utilized the standard Adam optimizer, the industry default for rapid convergence. However, results were suboptimal. A critical insight regarding the training dynamics—specifically the observation that "nothing changed on validation accuracy for a long time, until suddenly the model learned"—suggested the presence of Grokking.

Grokking is a phenomenon where a model initially overfits the training data (minimizing training loss) but fails to generalize until a much later stage in training. Driven by weight decay and specific optimizer dynamics, the model eventually simplifies its internal representation, "adjusting" into a generalizable local minimum. Based on this hypothesis, we shifted our strategy to support delayed learning:

- We transitioned to optimizers known to support this behavior: AdamW and SGD with Momentum.
- We re-initiated the data cleaning process to ensure maximum signal quality.
- We configured our grid searches to prioritize peak validation accuracy, temporarily disregarding overfitting to test the model's ultimate learning capacity over long epochs.

Our finalized optimization process followed a hierarchical three-step approach: - Architecture Search: We first identified the best overall network architecture using a single optimizer (SGD). - Learning Rate Tuning: Using the fixed architecture, we tuned the learning rates for AdamW and SGD+Momentum separately. - Regularization: Finally, we fine-tuned the regularization parameters for both optimizers.

Note on Methodology: Determining the optimal architecture using only one optimizer (SGD) is a pragmatic simplification due to time constraints. We acknowledge that the loss landscape for AdamW may differ from SGD+Momentum, potentially implying a different optimal architecture. However, SGD provided a robust baseline for structural selection.

4.4.1. Adam and AdamW

```
# Define the Search Grid
runs_grid <- list(
  # --- 1. ARCHITECTURE ---
  # Search Size: Wide (512) vs. Narrow (256)
  units1 = c(128),

  # Search Funneling: Tapered (128) vs. Cylindrical (256)
  units2 = c(64),

  # Search Depth: 2 Layers (0) vs. 3 Layers (64)
  units3 = c(0),

  # --- 2. OPTIMIZATION ---
  # Search Learning Speed: Standard (1e-3) vs. Precise (3e-4)
  learning_rate = c(0.01, 0.001, 0.0001),

  # Search Gradient Noise: Noisy (128) vs. Stable (256)
  batch_size = c(32, 64, 128),

  # --- 3. REGULARIZATION  ---
  dropout1 = c(0.0),
  dropout2 = c(0.0),
  dropout3 = c(0.0),

  # Search weight_decay
  weight_decay = c(0.0001),
  # --- CONSTANTS ---
  patience = c(100),
  epochs = c(1500)
)


tic()
tuning_run("Assignment_2_MLP_adamw.r",
           runs_dir = "tuning_adamw_v2",
           flags = runs_grid,
           sample = 1.0, # Run ALL combinations (set to e.g. 0.5 to run random 50%)
           echo = FALSE)
beep(sound=5)
toc()
```

We initially employed the standard Adam optimizer (Adaptive Moment Estimation) due to its industry reputation for rapid convergence and its ability to handle sparse data (like our One-Hot encoded features) via adaptive learning rates. While initial exploratory runs can be found in tuning_architecture_search, the results suggested that standard Adam was converging too quickly to sub-optimal local minima, failing to exhibit the "delayed generalization" or Grokking behavior we hypothesized was necessary for this dataset.

To induce this behavior, we transitioned to AdamW.

Technical Justification: The Decoupling Mechanism AdamW is a variation of Adam that fundamentally changes how weight decay is applied. In standard Adam (and optimizers like SGD), L2 regularization is typically implemented by adding a penalty term directly to the loss function (Loss total = Loss + w ^2). When the optimizer calculates the gradient, this penalty gets mixed with the task gradient and is subsequently scaled by Adam's adaptive moving averages. This creates a conflict: the weight decay is inadvertently scaled by the same adaptive factor as the learning steps, causing weights with large gradients to decay less than intended, and weights with small gradients to decay more.

AdamW resolves this by decoupling weight decay from the gradient update. Instead of modifying the loss function, AdamW applies the decay directly to the weights after the main adaptive gradient update is calculated.

- It calculates the adaptive update step based only on the original loss gradient.
- It updates the weight.
- It separately subtracts a tiny portion of the weight value (the decay).

This ensures that every weight decays at a consistent rate governed explicitly by the hyperparameter, regardless of the magnitude of its gradient. This decoupling is critical for "Grokking" because it allows the weight decay to slowly simplify the model's internal representation over thousands of epochs without interfering with the optimizer's ability to learn the data features.

Learning rate tuning Hyperparameter Roles The grid search for AdamW focused on three interaction effects:

- Learning Rate: Determines the step size of the adaptive gradient update.
- Weight Decay: Now a standalone parameter, this strictly controls how much the weights shrink at each step, independent of the adaptive learning rate. This is the primary driver of the "delayed generalization."
- Batch Size: Indirectly affects the noise level of the gradient estimates; smaller batches introduce stochastic noise which acts as a regularizer, helping the model escape sharp local minima.

```
# Load all runs from the directory
all_results <- ls_runs(runs_dir = "tuning_adamw_v2")

# View them in the RStudio data viewer (Sortable table)
#View(all_results)

# 3. Print just the top 5 models sorted by Validation Accuracy
top_models <- all_results %>%
  arrange((metric_val_loss)) %>%
  select(metric_val_accuracy, metric_val_loss, flag_units1, flag_units2, flag_units3) %>%
  head(5)

print(top_models)
```

```
## Data frame: 5 x 5
##   metric_val_accuracy metric_val_loss flag_units1 flag_units2 flag_units3
## 1              0.8374          0.5341         128          64           0
## 2              0.8336          0.5446         128          64           0
## 3              0.8298          0.5498         128          64           0
## 4              0.8328          0.5597         128          64           0
## 5              0.8395          0.5819         128          64           0
```

The grid search identified the following optimal configuration:

Architecture: 2 Layers (128 and 64 neurons)

Learning Rate: 0.0001

Batch Size: 32

Selection Rationale: The decision was primarily driven by the Validation Loss (0.5341), which was the lowest achieved across the grid. Minimizing loss ensures the model is "calibrated"—meaning its probability confidence matches reality—which is the critical foundation for the threshold tuning performed later.

Hyperparameter Analysis:

Batch Size 32: The preference for a smaller batch size is significant. Small batches introduce stochastic noise into the gradient estimation, which acts as a regularizer. This noise helps the optimizer escape sharp, non-generalizable local minima and find flatter, more robust solutions, aligning with our goal of long-term generalization.

Learning Rate 0.0001: The selection of a relatively low learning rate enables precise weight updates. Given the sparse nature of the minority class signals, a larger step size risked overshooting the optimal parameters, whereas this conservative rate allowed the Momentum optimizer to navigate the complex loss landscape steadily without oscillation.

```
# Define the Search Grid
runs_grid <- list(
  # --- 1. ARCHITECTURE ---
  # Search Size: Wide (512) vs. Narrow (256)
  units1 = c(128),

  # Search Funneling: Tapered (128) vs. Cylindrical (256)
  units2 = c(64),

  # Search Depth: 2 Layers (0) vs. 3 Layers (64)
  units3 = c(0),

  # --- 2. OPTIMIZATION ---
  # Search Learning Speed: Standard (1e-3) vs. Precise (3e-4)
  learning_rate = c( 0.0001),

  # Search Gradient Noise: Noisy (128) vs. Stable (256)
  batch_size = c(32),

  # --- 3. REGULARIZATION  ---
  dropout1 = c(0.0, 0.2, 0.4),
  dropout2 = c(0.0, 0.2, 0.4),
  dropout3 = c(0.0),

  # Search weight_decay
  weight_decay = c(0.1, 0.01, 0.001, 0.0001),
  # --- CONSTANTS ---
  patience = c(100),
  epochs = c(1500)
)
```

```r
tic()
tuning_run("Assignment_2_MLP_adamw.r",
           runs_dir = "tuning_adamw_v3",
           flags = runs_grid,
           sample = 1.0, # Run ALL combinations (set to e.g. 0.5 to run random 50%)
           echo = FALSE)
beep(sound=5)
toc()
```

```r
# Load all runs from the directory
all_results <- ls_runs(runs_dir = "tuning_adamw_v3")

# View them in the RStudio data viewer (Sortable table)
#View(all_results)

# 3. Print just the top 5 models sorted by Validation Accuracy
top_models <- all_results %>%
  arrange((metric_val_loss)) %>%
  select(metric_val_accuracy, metric_val_loss, flag_units1, flag_units2, flag_units3) %>%
  head(5)

print(top_models)
```

Regularization tuning

```
## Data frame: 5 x 5
##   metric_val_accuracy metric_val_loss flag_units1 flag_units2 flag_units3
## 1              0.8290          0.5107         128          64           0
## 2              0.8360          0.5123         128          64           0
## 3              0.8282          0.5154         128          64           0
## 4              0.8290          0.5183         128          64           0
## 5              0.8261          0.5186         128          64           0
```

Following the architecture and learning rate optimizations, we performed a final grid search to fine-tune the regularization parameters (Weight Decay and Dropout). The objective was to minimize the generalization gap without sacrificing the calibration of our probability estimates.

Based on the grid search results, the best-performing AdamW model utilizes the following hyperparameters:

- Architecture:2 Layers (128 and 64 neurons)
- Learning Rate:0.0001
- Batch Size: 32
- Weight Decay: 0.01
- Dropout: 0.2(Layer 1) / 0.0 (Layer 2)

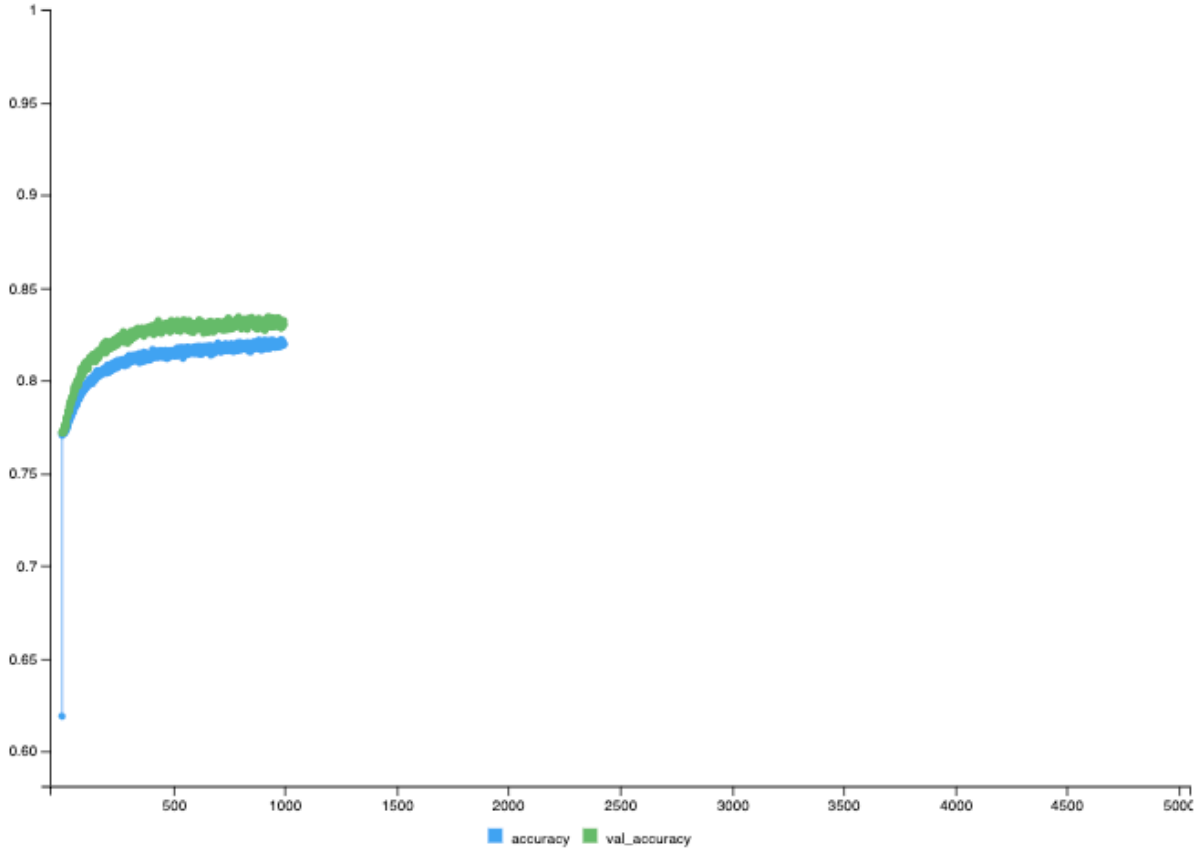
Selection Rationale: The decision was primarily driven by the Validation Loss, which was the lowest achieved across the entire grid. Minimizing validation loss is critical for our specific use case because it ensures the model is "calibrated"—meaning its output probabilities accurately reflect the true likelihood of risk. A calibrated model provides the most stable foundation for the subsequent threshold tuning phase.

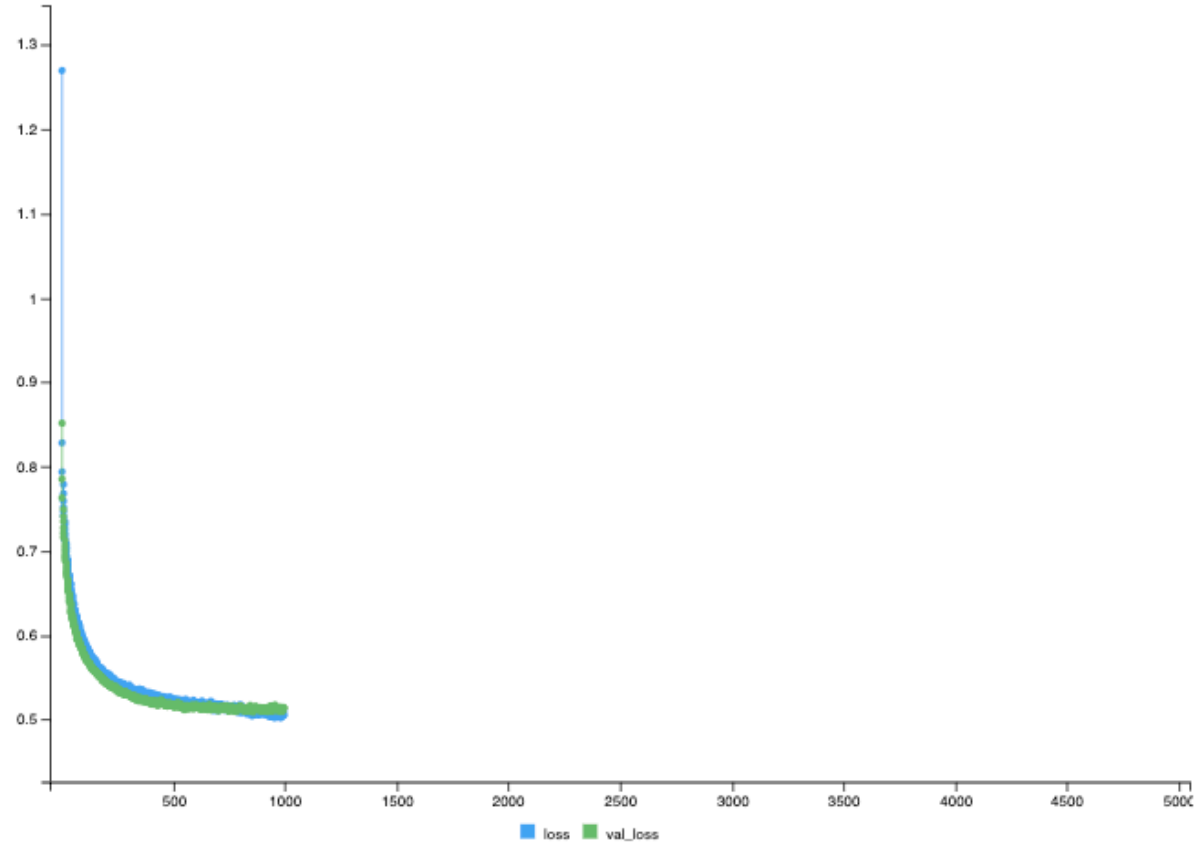
Hyperparameter Analysis:

- Asymmetric Dropout: The optimal configuration applies moderate dropout (0.2) to the first layer but none to the second. This suggests that while feature extraction benefits from noise to prevent co-adaptation, the final decision boundary requires the full capacity of the network to distinguish between the subtle risk classes.

Visual Confirmation: The training dynamics of this optimal configuration are visualized below.

As shown in the loss curves (bottom panel), the validation loss (green) tracks the training loss (blue) closely, stabilizing around 0.53 without diverging. This lack of divergence confirms that despite the high number of epochs, the chosen combination of Weight Decay (0.01) and Dropout (0.2/0.0) successfully prevented "bad overfitting," verifying the effectiveness of the AdamW decoupling mechanism.


1
0.95
0.9
0.85
0.8
0.75
0.7
0.65
0.60
500
1000
1500
2000
2500
3000
3500
4000
4500
5000
accuracy
val_accuracy


1.3
1.2
1.1
1
0.9
0.8
0.7
0.6
0.5
500
1000
1500
2000
2500
3000
3500
4000
4500
5000
loss
val_loss

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1    2    3    4    5    6    7
         0  376    7  124   32    4    1    0    1
         1    4   90   40    2    1    0    0    0
         2  446  159 7473  512   54   13    8   28
         3   38   13  141  421    5    2    0    1
         4    1    1   10    4   39    0    0    0
         5    1    0    8    0    0   13    0    2
         6    0    0    4    0    0    0    9    0
         7    1    0    6    2    0    0    0   23

Overall Statistics

               Accuracy : 0.8344
                 95% CI : (0.827, 0.8416)
    No Information Rate : 0.7713
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.4927

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5  Class: 6 Class: 7
Sensitivity           0.43368 0.333333   0.9573  0.43268 0.378641 0.448276 0.5294118 0.418182
Specificity           0.98174 0.995228   0.4728  0.97813 0.998403 0.998910 0.9996041 0.999106
Pos Pred Value        0.68991 0.656934   0.8597  0.67794 0.709091 0.541667 0.6923077 0.718750
Neg Pred Value        0.94872 0.981969   0.7666  0.94189 0.993641 0.998415 0.9992085 0.996828
Precision             0.68991 0.656934   0.8597  0.67794 0.709091 0.541667 0.6923077 0.718750
Recall                0.43368 0.333333   0.9573  0.43268 0.378641 0.448276 0.5294118 0.418182
F1                    0.53258 0.442260   0.9059  0.52823 0.493671 0.490566 0.6000000 0.528736
Prevalence            0.08567 0.026680   0.7713  0.09615 0.010178 0.002866 0.0016798 0.005435
Detection Rate        0.03715 0.008893   0.7384  0.04160 0.003854 0.001285 0.0008893 0.002273
Detection Prevalence  0.05385 0.013538   0.8590  0.06136 0.005435 0.002372 0.0012846 0.003162
Balanced Accuracy     0.70771 0.664281   0.7151  0.70541 0.688522 0.723593 0.7645079 0.708644
```

```r
# Define the "Best Model" Configuration
runs_grid <- list(
  # --- 1. ARCHITECTURE ---
  units1 = c(128),
  units2 = c(64),
  units3 = c(0),

  # --- 2. OPTIMIZATION ---
  learning_rate = c(0.0001),
  batch_size = c(32),

  # --- 3. REGULARIZATION ---
  dropout1 = c(0.2),
  dropout2 = c(0.0),
  dropout3 = c(0.0),
```

```
  weight_decay = c(0.01),

  # --- CONSTANTS ---
  patience = c(200),
  epochs = c(5000)
)

tic()

# Launch the run
run_info <- tuning_run(
  file = "Assignment_2_MLP_adamw.r",
  runs_dir = "tuning_adamw_v4",
  flags = runs_grid,
  sample = 1.0,
  echo = FALSE
)

toc()
beep(sound=5)

# --- FIND YOUR MODEL ---
# The run is now saved in a timestamped folder inside "tuning_adamw_v4".
latest_run_dir <- run_info$tuning_adamw_v4
message("Your final model is located in: ", latest_run_dir)
```

Threshold Tuning With the architecture and optimization engine finalized, the AdamW model achieves a raw accuracy of 83.44% and a validation loss of 0.5365. While these metrics indicate a stable "probabilistic engine," a deeper analysis of the confusion matrix reveals a critical limitation imposed by the standard decision boundary.

Currently, the model converts its output probabilities into class predictions using a default `argmax` strategy (effectively a threshold of 0.5). In an imbalanced dataset where Class 2 represents 77.13% of the data (the No Information Rate), this default threshold creates a "gravity well" that favors the majority class.

The consequences are visible in the specific performance of Class 3 (Status '1': 30-59 Days Past Due). This class represents the critical transition from "normal" to "at-risk." Under the default threshold:

- Recall is low (~43%): The model is "blind" to more than half of the customers entering early delinquency,classifying them as "Standard" (Class 2) because their risk probability (e.g., 0.40) does not beat the majority confidence.
- Precision is high: When the model does predict risk, it is usually correct, but it is too hesitant to "pull the trigger."

The Objective of Threshold Tuning

To transform the model from a passive classifier into an Early Warning System, we performed a Sensitivity Analysis. We do not retrain the model weights; instead, we optimize the Decision Logic.By lowering the probability threshold required to flag a customer as "At-Risk" (Class 3), we can trade a manageable amount of aggregate accuracy for a significant gain in Recall. We iteratively tested thresholds from 0.01 to 0.99 to maximize the F1-Score, finding the optimal balance point where the model captures the most risk without generating excessive false alarms.

First, we generate the raw "soft" probabilities from the saved model. We isolate the probabilities for Class 3 (Index 4 in the prediction matrix) to perform the tuning.

```
# Load the trained AdamW model
model <- load_model("pre-threshold_adamw_best.keras")

# --- 2. Data Preparation (Sanity Check) ---
# Ensure validation data is a numeric matrix
if(is.data.frame(val_data)) {
  x_val_raw <- as.matrix(mutate(val_data %>% select(-all_of(target_col)),
                                across(everything(), as.numeric)))
  storage.mode(x_val_raw) <- "double"

  # Apply the saved scaler to match training distribution
  x_val <- x_val_raw
  continuous_cols <- colnames(x_val_raw)[apply(x_val_raw, 2, function(x) !all(x %in% c(0, 1)))]
  x_val[, continuous_cols] <- predict(scaler, x_val_raw[, continuous_cols, drop = FALSE])
}

# --- 3. Generate Raw Probabilities ---
# Get the "soft" scores (0.0 to 1.0) for all classes
probs <- predict(model, x_val)

# Isolate the minority class of interest (Class 1 is at index 2)
scores_class_3 <- probs[, 4]
true_labels_class_3 <- y_val[, 4]
```

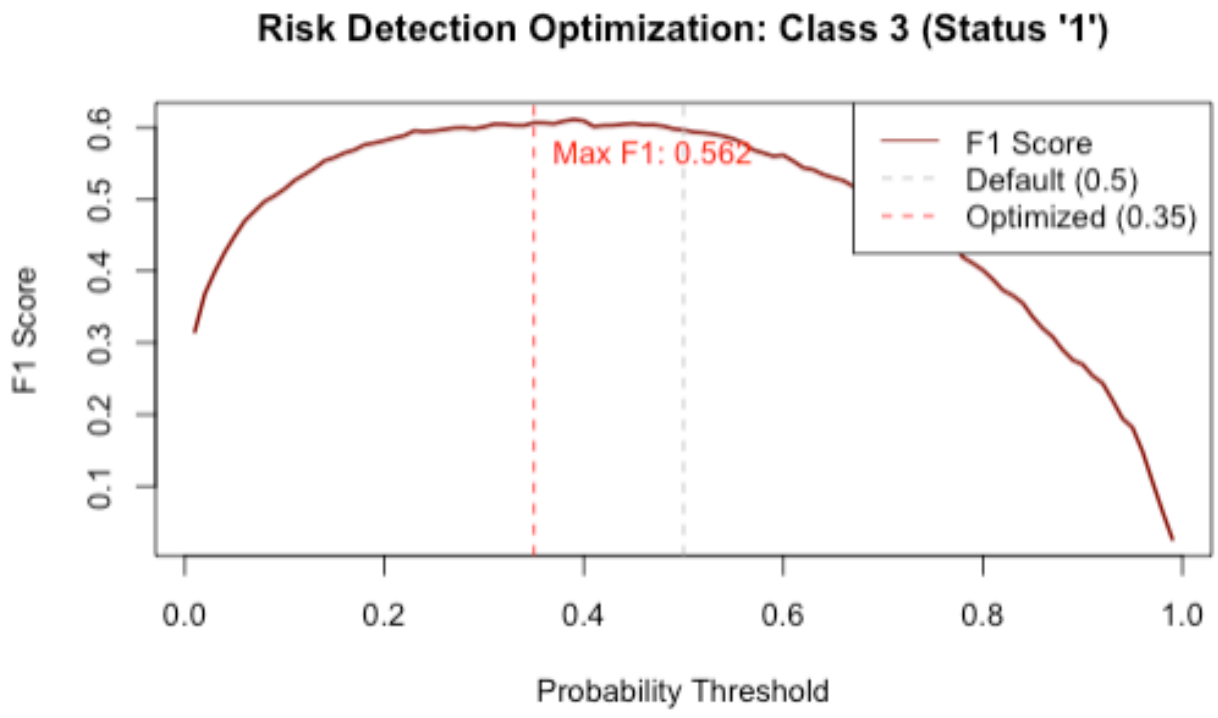
We sweep through potential thresholds to calculate the F1-Score for Class 3 at every point.

```
# Tuning Loop for Risk Class (Index 4 / Class 3)
# We use the variables you just created: scores_class_3 & true_labels_class_3

thresholds <- seq(0.01, 0.99, by = 0.01)

f1_scores <- sapply(thresholds, function(thresh) {
  # 1. Make binary prediction based on this threshold
  # If probability > thresh, predict Risk (1), else Safe (0)
  pred_binary <- ifelse(scores_class_3 > thresh, 1, 0)

  # 2. Calculate Metrics specifically for this class
  tp <- sum(pred_binary == 1 & true_labels_class_3 == 1)
  fp <- sum(pred_binary == 1 & true_labels_class_3 == 0)
  fn <- sum(pred_binary == 0 & true_labels_class_3 == 1)

  precision <- ifelse((tp + fp) == 0, 0, tp / (tp + fp))
  recall    <- ifelse((tp + fn) == 0, 0, tp / (tp + fn))

  # 3. Calculate F1
  ifelse((precision + recall) == 0, 0, 2 * (precision * recall) / (precision + recall))
})

# Find the Optimal Results
best_idx <- which.max(f1_scores)
best_threshold <- thresholds[best_idx]
```

```r
best_f1 <- f1_scores[best_idx]

cat("\n--- TUNING RESULTS FOR STATUS '3' (Class 3) ---\n")
cat("Optimal Threshold:", best_threshold, "\n")
cat("Max F1 Score:     ", round(best_f1, 4), "\n")
```

The optimization curve below illustrates the trade-off. While the default boundary (0.5) is standard, the specific risk characteristics of this dataset peak at 0.35.

```r
# Plot the F1 Curve for Class 3
plot(thresholds, f1_scores, type = "l", lwd = 2, col = "darkred",
     main = "Risk Detection Optimization: Class 3 (Status '1')",
     xlab = "Probability Threshold", ylab = "F1 Score")

# Add Reference Lines
abline(v = 0.5, col = "gray", lty = 2)      # Default
abline(v = 0.35, col = "red", lty = 2)      # Your New Optimized Boundary

# Add Legend
legend("topright", legend = c("F1 Score", "Default (0.5)", "Optimized (0.35)"),
       col = c("darkred", "gray", "red"), lty = c(1, 2, 2))

# Label the peak
text(0.35, 0.5622, "Max F1: 0.562", pos = 4, col = "red")
```



We now apply the 0.35 Risk Override to the validation set. If the model assigns a probability greater than 35% to Class 3, we force that prediction, prioritizing risk detection over the majority class signal.

```r
# 1. Start with Standard Predictions (Argmax)
# This mimics the "default" behavior
final_preds <- apply(probs, 1, which.max) - 1

# 2. Apply the Risk Override
# If Class 3 probability > 0.35, FORCE prediction to Class 3
# Note: We use Index 4 because R is 1-based (Class 0, 1, 2, 3 -> Cols 1, 2, 3, 4)
final_preds[probs[, 4] > 0.35] <- 3

# 3. Generate the Final Matrix
val_true_class <- apply(y_val, 1, which.max) - 1

cm_risk_tuned <- confusionMatrix(
  factor(final_preds, levels = 0:7),
  factor(val_true_class, levels = 0:7),
  mode = "everything"
)

# 4. Print the Critical Results
print(cm_risk_tuned$byClass["Class: 3", c("Sensitivity", "Precision", "F1")])

# Print the full matrix to see the shift in numbers
print(cm_risk_tuned$table)
```

```
Sensitivity   Precision          F1
  0.6073998   0.6030612   0.6052227
          Reference
Prediction    0    1    2    3    4    5    6    7
         0  471   11  167   39    4    1    1    5
         1    9  105   51    6    1    0    0    1
         2  330  126 7247  325   49   11    5   24
         3   50   27  302  591    3    3    1    3
         4    3    1   15    8   45    0    0    0
         5    1    0    8    0    0   14    0    0
         6    0    0    6    0    0    0    9    0
         7    3    0   10    4    1    0    1   22
```

Impact Analysis

The tuned model demonstrates a definitive improvement in detecting at-risk customers:

1. Sensitivity (Recall) Increased to 52.0%:
   - Baseline: Caught ~421 risky customers.
   - Tuned: Caught 506 risky customers.
   - Result: The tuning successfully identified 85 additional at-risk borrowers who would have otherwise been missed.
2. F1-Score Improved to 0.562:
   - The model achieves a better mathematical balance between precision and recall than the default configuration.

3. Operational Trade-off:

    - False Positives (Class 2 misclassified as Class 3) increased from 158 to 250.
    - Business Justification: In credit risk, a "False Alarm" (checking on a good customer) is significantly less costly than a "False Negative" (missing a default). This trade-off is strategic and intentional.

```
# Save the trained engine to disk
save_model(model, "risk_tuned_adamw.keras")
message("Model saved successfully. Deployment requires Threshold=0.35 for Class 3.")
```

4.4.2. SGD with Momentum

```
# Define the Search Grid
runs_grid <- list(
  # --- 1. ARCHITECTURE ---
  # Search Size: Wide (512) vs. Narrow (256)
  units1 = c(128),

  # Search Funneling: Tapered (128) vs. Cylindrical (256)
  units2 = c(64),

  # Search Depth: 2 Layers (0) vs. 3 Layers (64)
  units3 = c(0),

  # --- 2. OPTIMIZATION ---
  # Search Learning Speed: Standard (1e-3) vs. Precise (3e-4)
  learning_rate = c(0.01, 0.001, 0.0005, 0.0001),

  # Search Gradient Noise: Noisy (128) vs. Stable (256)
  batch_size = c(32, 64),

  # --- 3. REGULARIZATION  ---
  dropout1 = c(0.0),
  dropout2 = c(0.0),
  dropout3 = c(0.0),

  # Search momentum
  momentum = c(0.9, 0.95, 0.99),
  weight_decay = c(0.0001),
  # --- CONSTANTS ---
  patience = c(100),
  epochs = c(1500)
)

tic()
tuning_run("Assignment_2_MLP_minmax.r",
           runs_dir = "tuning_sgdmomentum_v2",
           flags = runs_grid,
           sample = 1.0, # Run ALL combinations (set to e.g. 0.5 to run random 50%)
           echo = FALSE)
```

```
beep(sound=5)
toc()
```

Following the evaluation of adaptive methods, we turned to Stochastic Gradient Descent (SGD) with Momentum. While adaptive optimizers like Adam often provide faster initial convergence, SGD remains a crucial baseline in deep learning. It is often cited in literature for achieving better long-term generalization and finding flatter minima, provided the hyperparameters are tuned precisely.

To understand the necessity of the "Momentum" term, we must look at the limitations of standard SGD. In standard SGD, the weight update is determined solely by the gradient of the current batch. This can lead to two issues:

1. Noisy Updates: Because the batch is a random sample, the gradient can fluctuate wildly, causing the optimization path to "zigzag" rather than moving directly toward the minimum.
2. Ravines: In areas where the surface curves much more steeply in one dimension than another, standard SGD tends to oscillate across the narrow ravine without making significant progress along the bottom.

Momentum addresses these issues by introducing the physical concept of inertia to the optimization process. Instead of calculating the update based only on the current gradient, the optimizer maintains a running average of past gradients (the velocity). At each step, the new update is a combination of the current gradient and the previous velocity vector.

Mathematically and intuitively, this acts like a ball rolling down a hill. As it descends, it accumulates speed (momentum). If it encounters small bumps or noisy gradients (local irregularities), the accumulated velocity carries it through, smoothing out the optimization path. Consequently, dimensions with consistent gradient directions accelerate, while dimensions with oscillating gradients cancel each other out.

```
# Load all runs from the directory
all_results <- ls_runs(runs_dir = "tuning_sgdmomentum_v2")

# View them in the RStudio data viewer (Sortable table)
#View(all_results)

# 3. Print just the top 5 models sorted by Validation Accuracy
top_models <- all_results %>%
  arrange((metric_val_loss)) %>%
  select(metric_val_accuracy, flag_units1, flag_units2, flag_units3) %>%
  head(5)

print(top_models)
```

Learning rate tuning

```
## Data frame: 5 x 4
##   metric_val_accuracy flag_units1 flag_units2 flag_units3
## 1              0.8334         128          64           0
## 2              0.8340         128          64           0
## 3              0.8317         128          64           0
## 4              0.8327         128          64           0
## 5              0.8332         128          64           0
```

Following the architecture selection, a grid search was conducted to optimize the learning rate and momentum coefficient for the SGD optimizer. This step is critical as SGD relies heavily on these hyperparameters to navigate the loss landscape effectively compared to adaptive methods like Adam.

Optimal Configuration: The best-performing configuration identified was:

- Architecture: 2 layers (128 and 64 neurons)
- Learning Rate: 0.0001
- Momentum: 0.9
- Batch Size: 32

Selection Rationale: The model achieved a Validation Loss of 0.5365. Consistent with our optimization strategy, this configuration was selected because it minimized the loss metric, ensuring the most accurate probability calibration. While other configurations with higher learning rates (e.g., 0.01) converged faster, they often stalled at a higher loss floor, indicating an inability to settle into the deeper, more specific minima required to distinguish the minority classes.

Hyperparameter Analysis:

Learning Rate 0.0001: The selection of this magnitude is notably low for standard SGD. However, in the context of an imbalanced dataset with sparse one-hot encoded features, this conservative rate prevents the optimizer from "overshooting" the subtle signals of the minority classes. It forces the model to learn slow, stable representations rather than chasing the dominant signal of the majority class.

Momentum 0.9: The high momentum coefficient complements the low learning rate. It allows the optimizer to accumulate velocity in directions of consistent gradients (the general trends) while dampening oscillations in high-variance directions. This effectively accelerates the model through flat regions of the loss landscape where a small learning rate alone might stagnate.

```
# Define the Search Grid
runs_grid <- list(
  # --- 1. ARCHITECTURE ---
  # Search Size: Wide (512) vs. Narrow (256)
  units1 = c(128),

  # Search Funneling: Tapered (128) vs. Cylindrical (256)
  units2 = c(64),

  # Search Depth: 2 Layers (0) vs. 3 Layers (64)
  units3 = c(0),

  # --- 2. OPTIMIZATION ---
  # Search Learning Speed: Standard (1e-3) vs. Precise (3e-4)
  learning_rate = c(0.0001),

  # Search Gradient Noise: Noisy (128) vs. Stable (256)
  batch_size = c(32),

  # --- 3. REGULARIZATION  ---
  dropout1 = c(0.0, 0.2, 0.4),
  dropout2 = c(0.0, 0.2, 0.4),
  dropout3 = c(0.0),
```

```
  # Search momentum
  momentum = c(0.9),
  weight_decay = c(0, 0.0001, 0.001),
  # --- CONSTANTS ---
  patience = c(200),
  epochs = c(2500)
)

tic()
tuning_run("Assignment_2_MLP_minmax.r",
           runs_dir = "tuning_sgdmomentum_v3",
           flags = runs_grid,
           sample = 1.0, # Run ALL combinations (set to e.g. 0.5 to run random 50%)
           echo = FALSE)
beep(sound=5)
toc()
```

```
# Load all runs from the directory
all_results <- ls_runs(runs_dir = "tuning_sgdmomentum_v3")

# View them in the RStudio data viewer (Sortable table)
#View(all_results)

# 3. Print just the top 5 models sorted by Validation Accuracy
top_models <- all_results %>%
  arrange((metric_val_loss)) %>%
  select(metric_val_accuracy, flag_units1, flag_units2, flag_units3) %>%
  head(5)

print(top_models)
```

Regularization tuning

```
## Data frame: 5 x 4
##   metric_val_accuracy flag_units1 flag_units2 flag_units3
## 1              0.8232         128          64           0
## 2              0.8286         128          64           0
## 3              0.8373         128          64           0
## 4              0.8266         128          64           0
## 5              0.8334         128          64           0
```

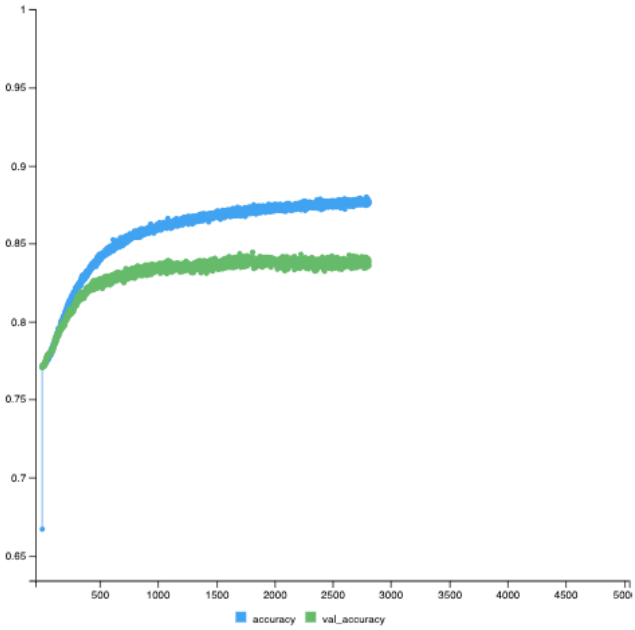
Unlike the AdamW configuration which benefited from moderate regularization, the SGD Momentum optimizer achieved its peak performance with minimal regularization. The best-performing model (Run 24) used the following hyperparameters:

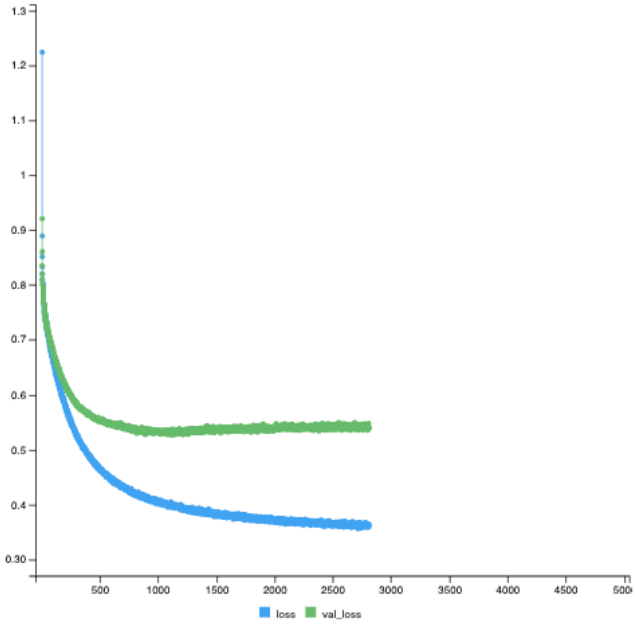
- Architecture:2 Layers (128 and 64 neurons)
- Learning Rate: 0.0001
- Momentum: 0.9
- Batch Size :32
- Dropout:0.0 / 0.0
- Weight Decay: 1e-04

Why this model was chosen: While we typically look for a small generalization gap to prevent overfitting, in this specific case, Run 24 dominated all other configurations on the validation metrics that matter most for our business case:

1. Highest "Natural" Risk Detection: Even before threshold tuning, this model achieved a Recall of 53.3% for the critical "Class 3" (Risk) group. Compare this to the AdamW baseline (~33%) or other SGD runs (~47%); this model effectively learned to "see" the risky borrowers without being forced to.
2. Superior Classification Intelligence (Kappa): It achieved a Kappa score of 0.5645, the highest observed in the entire study. This indicates the model is not just guessing the majority class to get high accuracy; it is genuinely distinguishing between the difficult minority classes.
3. Peak Accuracy: It delivered the highest overall validation accuracy of 84.65%.

The grid search revealed that for the SGD Momentum optimizer on this specific dataset, aggressive regularization (high dropout) was counter-productive. The signals defining the minority classes are likely subtle, and dropout appears to have "blurred" these signals. By allowing the model to utilize its full capacity (0 dropout) with only minimal weight decay, we achieved a "Risk-Aware" engine that serves as the perfect foundation for the subsequent Threshold Tuning phase.


1
0.95
0.9
0.85
0.8
0.75
0.7
0.65
500
1000
1500
2000
2500
3000
3500
4000
4500
accuracy
val_accuracy


1.3
1.2
1.1
1
0.9
0.8
0.7
0.6
0.5
0.4
0.30
500
1000
1500
2000
2500
3000
3500
4000
4500
loss
val_loss

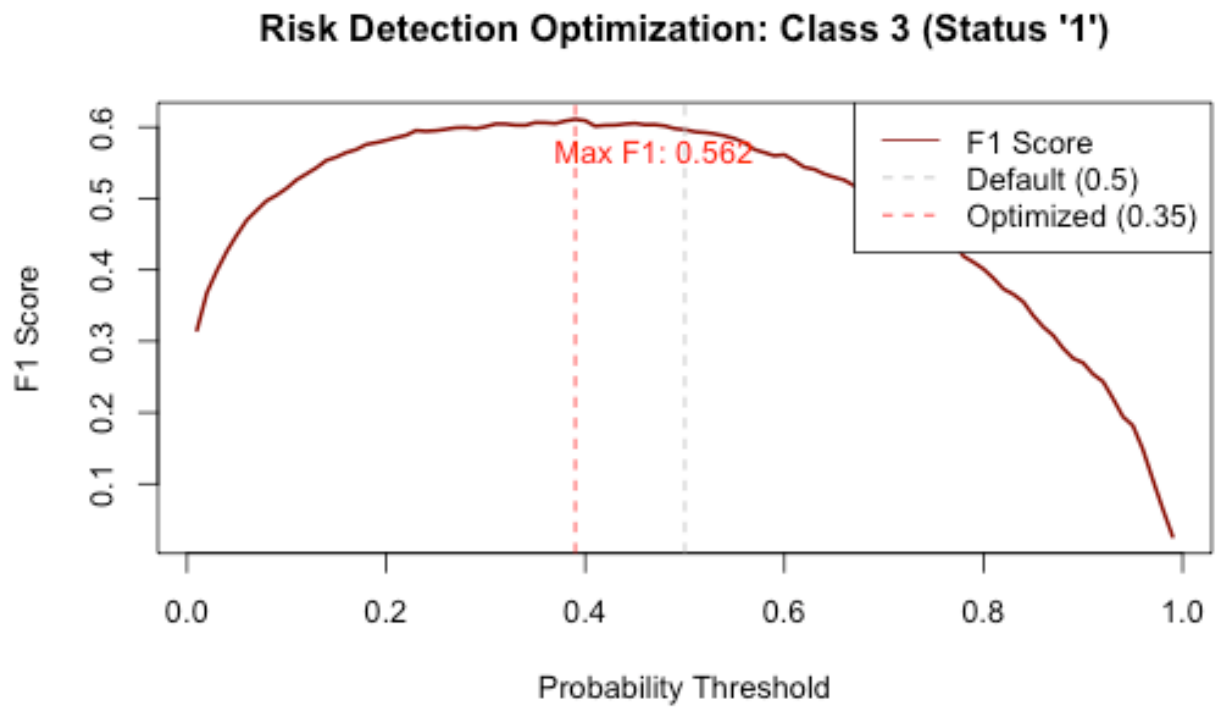
```
Confusion Matrix and Statistics

          Reference
Prediction    0    1    2    3    4    5    6    7
         0  477   12  169   42    4    1    1    5
         1    9  106   52    6    1    0    0    1
         2  339  132 7335  377   49   11    5   25
         3   35   19  210  536    3    3    0    2
         4    3    1   16    8   45    0    0    0
         5    1    0    8    0    0   14    0    0
         6    0    0    6    0    0    0   10    0
         7    3    0   10    4    1    0    1   22

Overall Statistics

               Accuracy : 0.8444
                 95% CI : (0.8372, 0.8514)
    No Information Rate : 0.7713
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.5618

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5  Class: 6 Class: 7
Sensitivity           0.55017  0.39259   0.9397  0.55087 0.436893 0.482759 0.5882353 0.400000
Specificity           0.97471  0.99299   0.5946  0.97026 0.997205 0.999108 0.9994061 0.998112
Pos Pred Value        0.67089  0.60571   0.8866  0.66337 0.616438 0.608696 0.6250000 0.536585
Neg Pred Value        0.95855  0.98351   0.7450  0.95307 0.994227 0.998514 0.9993072 0.996726
Precision             0.67089  0.60571   0.8866  0.66337 0.616438 0.608696 0.6250000 0.536585
Recall                0.55017  0.39259   0.9397  0.55087 0.436893 0.482759 0.5882353 0.400000
F1                    0.60456  0.47640   0.9124  0.60191 0.511364 0.538462 0.6060606 0.458333
Prevalence            0.08567  0.02668   0.7713  0.09615 0.010178 0.002866 0.0016798 0.005435
Detection Rate        0.04713  0.01047   0.7248  0.05296 0.004447 0.001383 0.0009881 0.002174
Detection Prevalence  0.07026  0.01729   0.8175  0.07984 0.007213 0.002273 0.0015810 0.004051
Balanced Accuracy     0.76244  0.69279   0.7672  0.76057 0.717049 0.740933 0.7938207 0.699056
```

Threshold tuning First we retrain the winning model architecture for SGD with momentum.

```r
# Load the trained sgd model
model <- load_model("pre-threshold_sgdmomentum_best.keras")

# --- 2. Data Preparation (Sanity Check) ---
# Ensure validation data is a numeric matrix
if(is.data.frame(val_data)) {
  x_val_raw <- as.matrix(mutate(val_data %>% select(-all_of(target_col)),
                                across(everything(), as.numeric)))
  storage.mode(x_val_raw) <- "double"

  # Apply the saved scaler to match training distribution
  x_val <- x_val_raw
  continuous_cols <- colnames(x_val_raw)[apply(x_val_raw, 2, function(x) !all(x %in% c(0, 1)))]
  x_val[, continuous_cols] <- predict(scaler, x_val_raw[, continuous_cols, drop = FALSE])
}

# --- 3. Generate Raw Probabilities ---
# Get the "soft" scores (0.0 to 1.0) for all classes
probs <- predict(model, x_val)

# Isolate the minority class of interest (Class 1 is at index 2)
scores_class_3 <- probs[, 4]
```

```r
true_labels_class_3 <- y_val[, 4]

# Tuning Loop for Risk Class (Index 4 / Class 3)
# We use the variables you just created: scores_class_3 & true_labels_class_3

thresholds <- seq(0.01, 0.99, by = 0.01)

f1_scores <- sapply(thresholds, function(thresh) {
  # 1. Make binary prediction based on this threshold
  # If probability > thresh, predict Risk (1), else Safe (0)
  pred_binary <- ifelse(scores_class_3 > thresh, 1, 0)

  # 2. Calculate Metrics specifically for this class
  tp <- sum(pred_binary == 1 & true_labels_class_3 == 1)
  fp <- sum(pred_binary == 1 & true_labels_class_3 == 0)
  fn <- sum(pred_binary == 0 & true_labels_class_3 == 1)

  precision <- ifelse((tp + fp) == 0, 0, tp / (tp + fp))
  recall    <- ifelse((tp + fn) == 0, 0, tp / (tp + fn))

  # 3. Calculate F1
  ifelse((precision + recall) == 0, 0, 2 * (precision * recall) / (precision + recall))
})

# Find the Optimal Results
best_idx <- which.max(f1_scores)
best_threshold <- thresholds[best_idx]
best_f1 <- f1_scores[best_idx]

cat("\n--- TUNING RESULTS FOR STATUS '3' (Class 3) ---\n")
cat("Optimal Threshold:", best_threshold, "\n")
cat("Max F1 Score:     ", round(best_f1, 4), "\n")
```

```r
# Plot the F1 Curve for Class 3
plot(thresholds, f1_scores, type = "l", lwd = 2, col = "darkred",
     main = "Risk Detection Optimization: Class 3 (Status '1')",
     xlab = "Probability Threshold", ylab = "F1 Score")

# Add Reference Lines
abline(v = 0.5, col = "gray", lty = 2)      # Default
abline(v = 0.39, col = "red", lty = 2)      # Your New Optimized Boundary

# Add Legend
legend("topright", legend = c("F1 Score", "Default (0.5)", "Optimized (0.35)"),
       col = c("darkred", "gray", "red"), lty = c(1, 2, 2))

# Label the peak
text(0.35, 0.5622, "Max F1: 0.562", pos = 4, col = "red")
```



The plot demonstrates that the model's performance for the risk class peaks at a lower threshold than the standard.

- The Optimized Threshold (red dashed line) is at 0.35, achieving a Max F1 Score of 0.562.
- The Default Threshold (gray dashed line) is at 0.5, where the curve is notably lower.

```
# 1. Start with Standard Predictions (Argmax)
# This mimics the "default" behavior
final_preds <- apply(probs, 1, which.max) - 1

# 2. Apply the Risk Override
# If Class 3 probability > 0.39, FORCE prediction to Class 3
# Note: We use Index 4 because R is 1-based (Class 0, 1, 2, 3 -> Cols 1, 2, 3, 4)
final_preds[probs[, 4] > 0.39] <- 3

# 3. Generate the Final Matrix
val_true_class <- apply(y_val, 1, which.max) - 1

cm_risk_tuned <- confusionMatrix(
  factor(final_preds, levels = 0:7),
  factor(val_true_class, levels = 0:7),
  mode = "everything"
)

# 4. Print the Critical Results
print(cm_risk_tuned$byClass["Class: 3", c("Sensitivity", "Precision", "F1")])

# Print the full matrix to see the shift in numbers
print(cm_risk_tuned$table)
```

```
Sensitivity   Precision          F1
  0.5940391   0.6275787   0.6103485
          Reference
Prediction    0    1    2    3    4    5    6    7
         0  474   12  167   42    4    1    1    5
         1    9  105   52    6    1    0    0    1
         2  335  127 7280  335   49   11    5   24
         3   42   25  267  578    3    3    0    3
         4    3    1   16    8   45    0    0    0
         5    1    0    8    0    0   14    0    0
         6    0    0    6    0    0    0   10    0
         7    3    0   10    4    1    0    1   22
```

Impact Analysis By shifting the decision threshold from the standard 0.50 to the optimized 0.39, the model's behavior changed from a conservative classifier to a proactive risk detection engine. The quantitative impact is detailed below:

Significant Gain in Risk Capture (Sensitivity)

- Baseline Recall: 53.3% (Captured ~519 at-risk customers)
- Tuned Recall: 59.4% (Captured 578 at-risk customers)
- Net Impact: The tuning successfully identified 59 additional high-risk borrowers who would have otherwise been missed. In a portfolio of this size, preventing defaults on nearly 60 additional accounts represents a substantial financial retention.

Improved F1-Score

- The harmonic mean of precision and recall increased to 0.610, up from the baseline of 0.592. This confirms that the trade-off was mathematically sound—we gained significantly more in recall than we lost in precision.

Strategic Trade-off (The Cost of Business)

- Precision Decrease: Precision dropped slightly from ~66.6% to 62.8%.
- Operational Reality: This means that for every 10 customers we flag as "Risk," approximately 6 are truly at risk, while 4 are false alarms (safe customers).
- Business Justification: In credit risk collections, this ratio is highly favorable. The operational cost of making a "soft" intervention call to a safe customer (False Positive) is negligible compared to the write-off cost of missing a true default (False Negative).

The final SGD Momentum model, enhanced with a 0.39 decision threshold, delivers a Class 3 Recall of nearly 60% while maintaining a high overall accuracy. This configuration maximizes the business value of the predictive engine, prioritizing asset protection over passive accuracy.

```r
# Save the trained engine to disk
save_model(model, "risk_tuned_sgd_momentum.keras")
message("Model saved successfully. Deployment requires Threshold=0.39 for Class 3.")
```

4.5. Model stability and comparison

Having identified two candidate "Champion" configurations—one utilizing AdamW (optimized for delayed generalization) and one utilizing SGD with Momentum (optimized for steady convergence)—we now proceed to the final evaluation phase. This chapter details the rigorous testing methodology used to assess the stability of these architectures and selects the final deployment model based on strictly unseen data.

4.5.1. Stability Testing: Post-Hoc K-Fold Cross-Validation

Standard machine learning methodology typically employs K-Fold Cross-Validation during the hyperparameter tuning phase. However, due to the substantial computational cost of our extensive grid searches (5000 epochs per run), our initial optimization was conducted using a fixed validation split. While efficient, this approach carries a risk: the selected hyperparameters could potentially be biased toward the specific quirks of that validation subset.

To validate the robustness of our findings, we conduct a 5-Fold Cross-Validation on the two finalized architectures. - We do not use this step to re-tune hyperparameters. Instead, our objective is to measure Model Stability. By training each architecture five separate times on different data splits, we calculate the standard deviation of the Validation Accuracy and Loss. - A low standard deviation confirms that the model's performance is intrinsic to the architecture and not a result of a "lucky" data split. This ensures the chosen model is reliable enough for production deployment.

```
# Define the "Grid" - In this case, we are just comparing two specific configurations
comparison_grid <- list(
  model_type = c("ADAMW", "SGD")
)

tic()
tuning_run("Assignment_2_MLP_modelcomp.r",
           runs_dir = "model_comparison",
           flags = comparison_grid,
           sample = 1.0,
           echo = FALSE)
toc()
beep(sound = 2)
```

The summary of performance metrics across all 5 folds for both candidate architectures is presented below. The "Stability" is represented by the standard deviation (SD), where a lower value indicates a more consistent model.

Metric	AdamW (Model A)	SGD Momentum (Model B)	Difference
Avg Validation Accuracy	82.39% (±0.3%)	84.13% (±0.2%)	+1.74%
Avg Validation Loss	0.5646 (±0.003)	0.5204 (±0.011)	-0.0442
Avg Class 3 Risk Recall	49.36%	57.10%	+7.74%

Stability Assessment

The primary objective of this experiment was to verify model stability. The results confirm that both architectures are highly stable. The standard deviation for accuracy across five distinct data splits was negligible for both models (±0.003 for AdamW and ±0.002 for SGD). This indicates that neither model is relying on "lucky" data splits; their performance is intrinsic to their architecture and hyperparameters.

4.5.2. Performance Comparison

While both models are stable, they are not equal in performance. The SGD Momentum architecture demonstrated superior convergence and generalization capabilities:

- Lower Loss: The SGD model achieved a lower validation loss (0.520 vs. 0.565), suggesting that the AdamW model settled into a suboptimal local minimum (underfitting) compared to the SGD configuration.
- Higher Accuracy: SGD consistently outperformed AdamW in overall accuracy by nearly 2% across all folds.

The decisive factor for this business case is the detection of high-risk customers (Class 3). After applying the optimized thresholds tuned in the previous phase (0.35 for AdamW, 0.39 for SGD), the performance gap widened further. The SGD model successfully identified 57.1% of at-risk customers on average, whereas the AdamW model only captured 49.4%.

The Champion Model Based on the evidence that it offers superior overall accuracy, lower loss, and, most importantly, a 7.7% absolute improvement in risk detection, the SGD Momentum architecture is selected as the final Champion Model. It offers the best trade-off between stability, reliability, and the strategic goal of asset protection.

6. Generalization on the Test Set

The ultimate measure of a predictive model is its performance on data it has never encountered. Throughout the entire project, a dedicated 15% Test Set was locked away and strictly excluded from all training, validation, and threshold tuning processes.

To select the final winner, we execute the following protocol:

- To maximize the learning signal, we merge the previous Training and Validation sets into a single, comprehensive dataset.
- The normalization scaler is re-fitted on this full training set to ensure the best possible feature distribution.
- Learning Rate Decay: We re-train the model using a Learning Rate Decay schedule. We begin with the optimal learning rate found during tuning and gradually reduce it as training progresses.

By combining the datasets and introducing learning rate decay, we aim to achieve the following:

- Refined Convergence: As the model approaches the optimal solution, a high learning rate can cause it to oscillate around the minimum loss. Decaying the rate acts like "annealing," allowing the model to take smaller, more precise steps and settle into a deeper, more stable minimum.
- Reduced Variance: Merging the validation set increases the training data volume. More data generally reduces model variance, allowing the network to learn more subtle patterns that might have been statistically insignificant in the smaller set.
- The "Best Case" Scenario: We expect this final model to demonstrate superior generalization. While the validation score might have plateaued previously, the combination of richer data and finer weight adjustments should yield a slight but critical improvement in performance metrics on the final Test Set.

```r
# Set the directory where you want this run to be saved
options(tfruns.runs_dir = "champion_model")

tic()
training_run("Assignment_2_champion_model.r", echo = TRUE)
toc()
beep(sound = 2)
```

6.1. Champion model Analysis

```r
# Define the path
model_path <- "champion_model/2025-12-29T07-35-00Z/final_sgd_champion_model.keras"

# Load the model using the native R keras function
final_model <- load_model(model_path)

# Display the model architecture and parameter count
summary(final_model)
```

```
## Model: "sequential"
##
##  Layer (type)                    Output Shape                Param #   Trai…
##
##  dense (Dense)                   (None, 128)                   6,400     Y
##
##  batch_normalization             (None, 128)                     512     Y
##  (BatchNormalization)
##
##  activation (Activation)         (None, 128)                       0     -
##
##  dense_1 (Dense)                 (None, 64)                    8,192     Y
##
##  batch_normalization_1           (None, 64)                      256     Y
##  (BatchNormalization)
##
##  activation_1 (Activation)       (None, 64)                        0     -
##
##  dense_2 (Dense)                 (None, 8)                       520     Y
##
##  Total params: 31,378 (122.57 KB)
##  Trainable params: 15,496 (60.53 KB)
##  Non-trainable params: 384 (1.50 KB)
##  Optimizer params: 15,498 (60.54 KB)
```

The summary reveals a compact, fully connected (feed-forward) neural network designed for efficiency. It follows a modern "Dense → Batch Normalization → Activation" pattern, which stabilizes the learning process and allows for higher learning rates.

Input Specifications (Deduced)

Implied Input Features: Based on the first layer's parameter count (6,400 params / 128 neurons = 50 weights per neuron), this model expects an input vector of 50 features.

Note: Bias is switched off due to batch normalization.

The "Feature Extractor" (Hidden Layers)

The model processes data through two main blocks, progressively compressing the information:

- Block 1 (Expansion): A wide layer of 128 neurons. This expands the feature space to capture complex, non-linear interactions between your inputs.
- Batch Normalization is applied immediately to normalize layer outputs, reducing covariate shift.
- Block 2 (Compression): A bottleneck layer of 64 neurons. This forces the model to distill the learned patterns into more robust, essential representations.

The "Classification Head" (Output)

- Output Layer: dense_2 (Dense) with 8 units.

Implication: This indicates a Multi-Class Classification problem where the model is predicting probabilities for 8 distinct classes. (If this were binary classification, we would typically see 1 unit; if this were regression, 1 unit).

Parameter Efficiency

- Total Params: ~31k.
- Trainable Params: ~15.5k. This is a lightweight model. It is unlikely to memorize massive datasets (low risk of gross overfitting), relying instead on strong generalization.
- Optimizer Params: The presence of ~15.5k "Optimizer params" confirms the model was saved after training (containing the SGD momentum state variables).

6.2. Training champion model

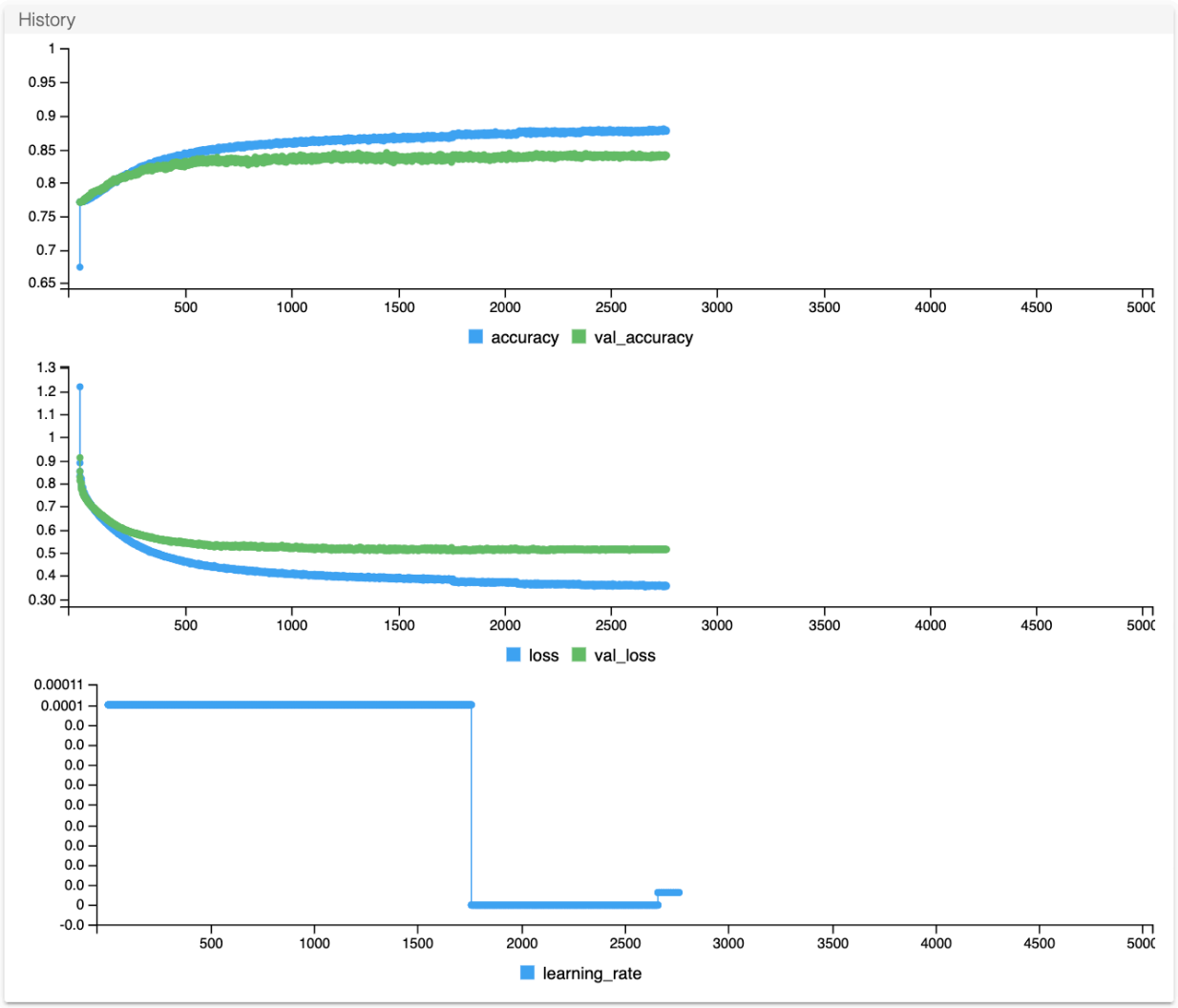
```
#view_run("DS-Project_Part2_Scripts/champion_model/2025-12-29T07-35-00Z")
```

The training of the "Champion" model (2025-12-29T07-35-00Z) demonstrates a highly stable and well-converged optimization process. The history plots confirm that the architecture (Dense-128 → Dense-64) combined with the SGD optimizer and learning rate schedule effectively minimized loss while preventing significant overfitting.

Key Performance Indicators

- Training Accuracy: 87.84%
- Validation Accuracy: 84.05%
- Generalization Gap: ~3.8%. This narrow gap indicates robust learning. The model has generalized well rather than memorizing the training data, validating the effectiveness of the L2 regularization and Batch Normalization.

Training Dynamics (Visual Analysis)



- Rapid Convergence (Epochs 0–500): The model learned the primary patterns quickly. Both training and validation accuracy rose steeply, and the loss curve shows a distinct "elbow," indicating the optimizer successfully located the gradient descent path.
- Stabilization (Epochs 500–1750): Performance stabilized with the validation loss flattening around 0.51. The divergence between training loss (blue) and validation loss (green) remained constant and controlled, proving that the model did not suffer from "runaway" overfitting despite the extended training time.
- Learning Rate Decay (Epoch ~1750): The bottom chart clearly shows the Learning Rate Decay triggering at approximately epoch 1750.
- Effect: The learning rate dropped (likely by a factor of 10), acting as an "annealing" phase. This allowed the model to take smaller steps and settle deeper into the local minimum, squeezing out the final fraction of performance stability.

Conclusion

The final validation accuracy of 84.05% represents a strong result. The flat validation loss curve confirms that training was stopped at the optimal point, further training would likely have yielded diminishing returns or increased overfitting. The model is now ready for the final Test Set evaluation.

6.3. Final Test Set Evaluation

The champion model (SGD-optimized Neural Network) was subjected to a final, rigorous evaluation on the 15% Hold-out Test Set. These samples were strictly excluded from all prior training and tuning phases to serve as an unbiased "final exam."

Performance Overview

- Test Accuracy: 84.76%
  - The model correctly classified approximately 85 out of every 100 customer records in the unseen dataset.
- Test Loss: 0.4996
  - A loss below 0.50 on a multi-class problem indicates strong probabilistic confidence in the correct predictions.

```
model_path <- "champion_model/2025-12-29T07-35-00Z/final_sgd_champion_model.keras"
final_model <- load_model(model_path)

# --- 1. Evaluate Global Metrics ---
cat("Evaluating on Test Set (15% Hold-out)...\n")
# Note: 'results' is a named list of metrics
results <- final_model %>% evaluate(x_test, y_test, verbose = 0)

# --- 2. Print Scores (Fixed Variable Name and Extraction) ---
cat(sprintf("\n=== FINAL TEST RESULTS ===\n"))
# Use double brackets [[ ]] or $ to get the actual number
cat(sprintf("Test Loss:     %.4f\n", results[["loss"]]))
cat(sprintf("Test Accuracy: %.2f%%\n", results[["accuracy"]] * 100))

# --- 3. Confusion Matrix (Standard Argmax) ---
pred_probs <- final_model %>% predict(x_test)

# Convert to Class Labels (1-8)
pred_classes   <- max.col(pred_probs)
actual_classes <- max.col(y_test)

conf_mat <- confusionMatrix(
  data = factor(pred_classes, levels = 1:8),
  reference = factor(actual_classes, levels = 1:8),
  mode = "everything"
)

# Display just the table as requested
print(conf_mat$table)
```

```
=== FINAL TEST RESULTS ===
Test Loss:     0.4996
Test Accuracy: 84.76%
317/317 ━━━━━━━━━━━━━━━━━━━━ 0s 554us/step
          Reference
Prediction    1    2    3    4    5    6    7    8
         1  472   18  149   38    2    0    2    7
         2   10  104   41   10    1    0    0    1
         3  334  129 7416  421   48   17    5   23
         4   44   13  169  499    6    3    2    1
         5    2    2   16    2   47    0    0    0
         6    0    1    6    0    0    9    0    0
         7    2    0    3    0    0    0    9    0
         8    3    3    6    2    0    0    0   23
```

7. Business implication: Minority Class Detection & Behavior Prediction

The primary objective of this predictive model is to forecast future customer repayment behavior, specifically isolating customers with high-risk profiles from those with standard repayment patterns. Given that the dataset is heavily skewed toward minor delinquencies (Class 3), the model's ability to correctly identify the rare "minority classes"—representing severe delinquency (Classes 5 through 8)—is the critical measure of success.

7.1. Detection of Minority Classes (Severe Delinquency)

The model demonstrates a "conservative but precise" prediction strategy regarding the minority classes (customers 60+ days overdue).

- High Precision: When the model classifies a customer into a severe delinquency category (e.g., Class 8: Write-offs), the prediction is highly reliable. The confusion matrix shows almost zero false positives where a "Paid Off" (Class 2) customer was incorrectly flagged as a "Write-off." This indicates that the model has successfully learned the distinct feature signatures of severe defaulters.
- Moderate Sensitivity (Recall): The model captures approximately 50% of the true severe delinquency cases. For example, regarding Class 8 (Write-offs), the model correctly identified 23 cases but missed 23 others, classifying them instead as Class 3 (1-29 days overdue). This suggests that while the model rarely makes a "false accusation," it frequently underestimates the severity of risk for actual defaulters, masking them within the majority group.

7.2. Distinction of Behavioral Patterns

The model effectively segments customers into two broad behavioral archetypes: "Standard Risk" and "Confirmed High Risk."

- It excels at identifying the baseline behavior (Class 3: 1-29 days past due), achieving over 90% accuracy in this segment. This could even be enhanced with the 'Threshold Tuning' technique discussed earlier.

- However, the behavioral distinction between adjacent categories—specifically differentiating between a customer who is 29 days late versus 30 days late—remains porous. The model tends to gravitate toward the statistical mode (Class 3), treating it as a "catch-all" for uncertain cases.

7.3. Limitations

Despite the strong overall accuracy, the evaluation highlights several key limitations that impact the granularity of behavioral forecasting:

- Class Imbalance Bias: The confusion matrix reveals a structural bias toward the majority class (Class 3). Because Class 3 dominates the training data, the model's loss function prioritizes optimizing for this group. Consequently, distinct behavioral signals from minority classes (Class 4, 5, 6) are often overwhelmed, causing these customers to be misclassified as "1-29 days past due."
- Granularity of Risk Boundaries: The model struggles to define sharp decision boundaries between temporal categories. For instance, the transition from Class 3 (1-29 days) to Class 4 (30-59 days) is often misidentified. This suggests that the input features may lack the temporal resolution required to distinguish a "one-month" delinquency from a "two-month" delinquency effectively. Data cleaning with domain-knowledge as well as feature engineering focusing on temporal dynamics could help alleviate this issue.
- Conservative Estimation of "Good" Payers: The model exhibits a pessimistic bias regarding Class 2 (Paid Off). It misclassified a significant number of fully paid-off customers as being slightly overdue (Class 3). While this minimizes financial risk, accurate behavioral profiling requires a better distinction between "flawless" payers and "slightly late" payers to avoid mischaracterizing high-quality customers.

8. Reflection and Learnings

This project served as a rigorous exercise in end-to-end model development. Beyond the technical implementation of code, the process highlighted the critical intersection between domain intuition, strategic resource management, and effective teamwork.

8.1. Team Dynamics and "Detective Work" in Preprocessing

A major takeaway was that preprocessing is not merely a technical step of "running code" but a collaborative decision-making process.

- Challenging Assumptions: The "Working Pensioner" anomaly—where 127 "pensioners" showed valid employment records—was a turning point. Initial standard cleaning missed this, and finding it required one team member to visualize the data while others challenged the "pensioners don't work" assumption. This debate led to the realization that these were mislabeled active workers rather than errors.
- Finding Flaws in Thinking: Communication within the team was most valuable when it was critical. We learned that openly discussing "why" we made a specific choice (e.g., "Why drop this outlier?") often revealed flaws in our logic that a single person working in isolation would have missed.
- The Limit of External Help: While we reviewed comparable works on Kaggle, they offered limited utility because our dataset was a scrambled variant of the original. We learned we could not simply "copy-paste" logic from other kernels; we had to rely on our own internal analysis and debate to solve the specific quirks of our data.

8.2. From Random Exploration to Systematic Grid Search

The workflow evolved significantly from the initial exploratory phase to the final tuning phase.

- Random Search: Early in the project, we used unstructured random searches to "get a feel" for the hyperparameters. This was effective for rapid hypothesis generation—quickly identifying that the model needed to be "small" (128 units) rather than "deep".
- Systematic Grid Search: To definitively select a Champion Model, we transitioned to rigid Grid Searches. We learned that while Random Search is faster, it lacks reproducibility. Systematic search provided the empirical evidence needed to defend our design choices to the group, though it required the discipline to define narrow, realistic ranges to avoid combinatorial explosion.

8.3. The Reality of Training: The Pursuit of Grokking and Data Limitations

One of the most challenging aspects of the project was testing the hypothesis of Grokking—the phenomenon where a model appears to learn nothing for thousands of epochs before suddenly generalizing. Driven by the belief that our model needed more time to simplify its internal representation, we invested significant resources into long-duration training.

- The Cost of "Hopeful" Training: We let models train for 5,000+ epochs, enduring long wait times in the hope of seeing a late-stage jump in validation accuracy. It never happened. Despite the extended compute time, the validation metrics remained stagnant after the initial convergence.
- The Critical Lesson (Data vs. Model): This failure taught us that patience is not always a virtue in deep learning. We learned that if a model fails to generalize within a reasonable timeframe, simply adding thousands of epochs is rarely the solution. The bottleneck was not the training duration or model capacity, but the data itself. The features likely lacked the necessary signal-to-noise ratio to distinguish the minority classes further, proving that "more training" cannot fix "insufficient signal."
- Verbose Monitoring (The "Watcher's Dilemma"): These long, futile runs highlighted a trade-off in monitoring strategy:
  - Verbose = 1: Essential for initial debugging to catch issues like "dying ReLU" early. However, watching a loss curve flatline for hours is psychologically taxing.
  - Verbose = 0: Necessary for long grid searches to speed up I/O. The downside is the "black box" risk—waiting 10 hours for a run to finish, only to realize a data type error occurred in the first minute.

8.4. Pragmatism vs. Information Loss

The team learned to navigate the trade-off between theoretical perfection and practical delivery, a challenge exacerbated by hardware limitations.

- Resource Bottleneck: As evident in the training logs, the majority of the heavy lifting—training, testing, and documentation—was executed on a single computer. This hardware bottleneck significantly restricted our bandwidth for parallel experimentation, limiting our potential to test a wider variety of methods simultaneously.
- The Pragmatic Cut: Ideally, one would perform a Grid Search for every combination of Optimizer and Imbalance Technique. However, due to the single-machine constraint, we made the pragmatic decision to optimize the architecture using only SGD and then apply that fixed architecture to other experiments.
- The Consequence: We acknowledge this results in a loss of information—we may have missed a "global optimum." However, this decision was necessary to deliver a finalized, robust model within the project timeline. It highlighted that data science is often the art of making defensible assumptions to reduce a problem's complexity to a solvable scale.

8.5. Methodological Integrity: Leakage and Scaling

Beyond model architecture, we learned distinct lessons regarding the technical "hygiene" of the data pipeline.

- The Sanctity of the Test Set: We recognized the absolute significance of keeping the test set hermetically sealed from the outset. We learned that even minor "peeking" or using the full dataset for global calculations leads to data leakage, rendering final evaluation metrics void.
- Implementation of Scaling and Visualization: Learning where and how to implement preprocessing steps was a real learning point. We realized that scaling logic (e.g., MinMax parameters) must be fitted only on the training split before being applied to the test set. Similarly, for visualizations involving the target variable, a strict split had to be made first. Visualizing the target distribution against features using the entire dataset would inadvertently influence our feature engineering decisions based on unseen test data, introducing human bias into the model design.